



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA U
NOVOM SADU



Mastilović Radoslav

**Implementacija CLOUD tehnologija za razvoj aplikacije
za upravljanje forumskim temama i komentarima na
primeru Reddita**

DIPLOMSKI RAD
- Osnovne akademske studije –

Novi Sad, 2024.

	UNIVERZITET U NOVOM SADU • FAKULTET TEHNIČKIH NAUKA 21000 NOVI SAD, Trg Dositeja Obradovića 6	Datum:
	ZADATAK ZA IZRADU DIPLOMSKOG (BACHELOR) RADA	List:

(Podatke unosi predmetni nastavnik – mentor)

Vrsta studija:	☐ Osnovne akademske studije
Studijski program:	Primenjeno softversko inženjerstvo
Rukovodilac studijskog programa:	vanr. prof. dr Selakov Aleksandar

Student:	Radoslav Mastilović	Broj indeksa:	PR106/2020
Oblast:	Razvoj Cloud aplikacija u pametnim mrežama		
Mentor:	prof. dr Srdjan Vukmirović		

NA OSNOVU PODNETE PRIJAVE, PRILOŽENE DOKUMENTACIJE I ODREDBI STATUTA FAKULTETA IZDAJE SE ZADATAK ZA DIPLOMSKI RAD, SA SLEDEĆIM ELEMENTIMA:

- problem – tema rada;
- način rešavanja problema i način praktične provere rezultata rada, ako je takva provera neophodna;
- literatura

NASLOV DIPLOMSKOG (BACHELOR) RADA:

Implementacija CLOUD tehnologija za razvoj aplikacije za upravljanje forumskim temama i komentarima na primeru Reddita

TEKST ZADATKA:

Cilj projekta je implementacija sistema za kreiranje i upravljanje forumskim temama i komentarima sa zasebnim servisima za slanje notifikacija i proveru dostupnosti sistema. Korisnički interfejs će biti Web Role aplikacija koja omogućava registraciju, logovanje, izmenu profila, kreiranje postova i komentara, pretragu i sortiranje postova, favorizovanje postova, upvote/downvote funkcionalnost za postove i komentare...

Notifikacije će se slati korisnicima pretplaćenim na određene teme putem NotificationService-a. HealthMonitoringService će redovno proveravati dostupnost servisa i slati obaveštenja administratorima putem emaila, a za obaveštavanje će se koristiti **SignalR** za real-time komunikaciju.

Rukovodilac studijskog programa:	Mentor rada:

Primerak za: ☐ - Studenta; ☐ - Mentora

Sadržaj

1. Uvod.....	1
2. Korišćenje tehnike i tehnologije.....	3
2.1. Microsoft Visual Studio [2022].....	3
2.2. JavaScript, HTML, CSS	5
2.3. Microsoft Azure Cloud Service	8
2.4. Microsoft Azure Storage Service.....	9
2.4.1. Microsoft Azure Compute Emulator	11
2.4.2. Microsoft Azure Storage Emulator.....	12
3. Specifikacija sistema	14
3.1. WebRole services	14
3.2. WorkerRole services	17
3.3. Ostale komponente sistema i tok podataka	23
4. Prikaz implementiranog sistema.....	25
4.1. Prikaz AdminTools sa bazom podataka i SignalR	25
4.2. Prikaz Login stranice	28
4.3. Prikaz Registration stranice sa bazom podataka	28
4.4. Prikaz Profile stranice sa bazom podataka	30
4.5. Prikaz Home-Reddit stranice sa bazom podataka	31
4.6. Prikaz PostPage stranice sa bazom podataka	36
5. Implementacija sistema	41
5.1. WebRole komponenta	41
5.2. RedditService i Common komponenta	51
5.3. NotificationService komponenta	59
5.4. HealthMonitoringService komponenta	64
5.5. AdminTools i SignalR biblioteka.....	69
6. Zaključak	75
Literatura	77
Biografija	79
Ključna dokumentacijska informacija	81
Key words documentation.....	83

1. Uvod

Savremeno doba karakteriše brz razvoj informacionih tehnologija i široka dostupnost interneta. To je dovelo do značajnih promena u načinu na koji komuniciramo, radimo i pristupamo informacijama. U okviru ovog tehnološkog napretka, CLOUD tehnologije su se izdvojile kao ključni faktor u omogućavanju skalabilnih, fleksibilnih i pouzdanih aplikacija. Upotreba CLOUD servisa omogućava organizacijama i programerima da efikasno upravljaju resursima, poboljšaju performanse i smanje operativne troškove.

CLOUD tehnologije, kao što su Microsoft Azure, omogućavaju razvoj distribuiranih sistema koji mogu obraditi veliki broj korisničkih zahteva, skalirati resurse prema potrebi i obezbediti visoku dostupnost aplikacija. Korišćenjem ovih tehnologija, moguće je brzo i efikasno implementirati složene poslovne procese i aplikacije koje zadovoljavaju sve veću potražnju za online uslugama.

Kroz rad na različitim projektima, stečena su znanja i veštine u korišćenju Microsoft Azure servisa za razvoj distribuiranih sistema. Implementirane su različite vrste servisa, kao što su WebRole, WorkerRole, i različite vrste skladišta kao što su Table Storage, Blob Storage i Queue Storage. Ove tehnologije omogućavaju kreiranje aplikacija koje su visoko dostupne, skalabilne i otporne na greške.

Osim tehničkih aspekata, važan deo razvoja ovakvih sistema je i razumevanje potreba korisnika i način na koji će oni koristiti aplikaciju. Zbog toga je fokus ovog rada na razvoju aplikacije koja će korisnicima omogućiti da na jednostavan i intuitivan način kreiraju, pretražuju i upravljaju forumskim temama i komentarima.

Tema rada je **“Implementacija CLOUD tehnologija za razvoj aplikacije za upravljanje forumskim temama i komentarima na primeru Reddita”**. Osnovni cilj aplikacije je omogućiti korisnicima da jednostavno kreiraju, pretražuju i upravljaju temama i komentarima, uz visok nivo pouzdanosti i performansi.

Aplikacija se sastoji od sledećih ključnih komponenti, čiji će detaljniji opis biti prikazan u narednim poglavljima: AdminTools (Console Application), Common (Class Library), HealthMonitoringService (WorkerRole Service), NotificationService (WorkerRole Service), Reddit (Azure Cloud Service), RedditService (WorkerRole Service) i WebRole (WebRole Service).

Korišćenje Microsoft Azure platforme omogućava visoku dostupnost, skalabilnost i pouzdanost sistema. Azure servisi kao što su Table Storage, Queue Storage i Blob Storage omogućavaju efikasno upravljanje i skladištenje podataka, dok Azure platforma omogućava lako upravljanje Web/Worker role servisima.

Dalji tekst biće predstavljen kroz šest poglavlja. U drugom poglavlju će biti opisane tehnologije korišćene pri implementaciji rešenja. Treće poglavlje će obuhvatiti specifikaciju funkcionalnosti sistema, uključujući detaljan opis svakog servisa i njihove međusobne interakcije. Četvrto poglavlje će detaljno opisati prikaz sistema i njegovo podešavanje. Peto poglavlje će prikazati implementirani sistem. U poslednjem, šestom poglavlju, biće izveden zaključak sa diskusijom o mogućnostima za proširenje aplikacije.

2. Korišćene tehnike i tehnologije

Razvoj savremene Cloud aplikacije predstavlja kompleksan proces koji zahteva pažljivu integraciju različitih tehnika i tehnologija. Ove tehnologije omogućavaju ne samo efikasnu implementaciju već i visoku skalabilnost, pouzdanost i sigurnost sistema, što su ključni faktori za postizanje uspeha u dinamičnom i zahtevnom svetu CLOUD razvoja. U ovom poglavlju ćemo detaljno opisati različite tehnike i tehnologije koje su korišćene prilikom izrade našeg rešenja.

Od alata za razvoj koji ubrzavaju i pojednostavljaju proces kodiranja i testiranja, preko programskih jezika koji omogućavaju kreiranje dinamičnih i interaktivnih korisničkih interfejsa, do naprednih Cloud servisa koji obezbeđuju skalabilnost infrastrukture i efikasno upravljanje podacima – svaka od ovih komponenti ima vitalnu ulogu u uspešnom razvoju aplikacije.

Prvo, fokusiraćemo se na Microsoft Visual Studio 2022, integrisano razvojno okruženje koje nudi robustan skup alata za razvoj, debugovanje i testiranje aplikacija. Ovaj alat omogućava programerima da efikasno upravljaju kodom, detektuju i ispravljaju greške te integrišu različite sisteme za kontrolu verzija, čime se značajno poboljšava produktivnost i kvalitet konačnog rešenja.

Takođe, analiziraćemo ključne programske jezike kao što su JavaScript, HTML i CSS. JavaScript je esencijalan za kreiranje dinamičnih i interaktivnih elemenata na veb stranicama, dok HTML i CSS pružaju strukturu i stilizaciju, omogućavajući izradu vizuelno privlačnih i funkcionalnih korisničkih interfejsa. Kombinacija ovih jezika omogućava izradu responzivnih i intuitivnih prijatnih aplikacija koje zadovoljavaju savremene standarde i očekivanja korisnika.

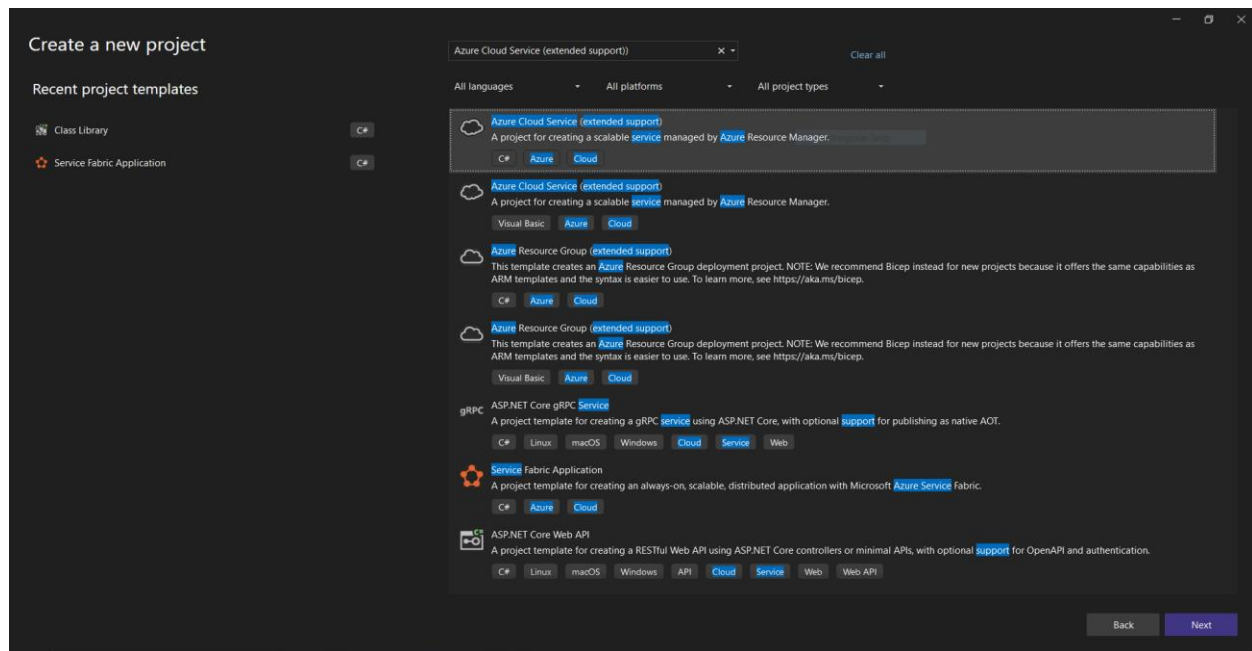
Poseban akcenat će biti stavljen na CLOUD tehnologije koje pruža Microsoft Azure. Azure Cloud Service omogućava fleksibilno i skalabilno okruženje za pokretanje aplikacija, dok Azure Storage Service Explorer olakšava upravljanje skladištem podataka, pružajući sigurno i pouzdano skladištenje velikih količina podataka. Ove tehnologije omogućavaju nesmetan rad aplikacije, upravljanje opterećenjem i obezbeđuju kontinuitet poslovanja čak i u uslovima visokog opterećenja.

Na kraju, ovaj pregled korišćenih tehnika i tehnologija pružiće sveobuhvatan uvid u izbor i primenu tehničkih alata i servisa koji su omogućili kreiranje efikasne, pouzdane i skalabilne Cloud aplikacije.

2.1. Microsoft Visual Studio [2022]

Microsoft Visual Studio 2022 je najnovija verzija integrisanog razvojnog okruženja (IDE) koje je razvila kompanija Microsoft. Ovo okruženje je ključno za razvoj različitih vrsta softverskih rešenja, uključujući veb stranice, veb aplikacije, veb usluge i mobilne aplikacije. Verzija 2022 donosi brojne inovacije i unapređenja koja značajno poboljšavaju iskustvo programera, povećavajući efikasnost i produktivnost razvoja.

Jedna od najznačajnijih novina u Visual Studio 2022 je prelazak na 64-bitnu arhitekturu, što omogućava glavnom procesu Visual Studio-a da koristi više od 4 GB RAM-a. Ovo je ključno za rad sa velikim projektima, jer eliminiše probleme nedostatka memorije kod kompleksnih i obimnih projekata, pružajući stabilnije i brže razvojno okruženje.



Slika 2.1. – Microsoft Visual Studio [2022]

U ovom projektu, koji je kreiran u Visual Studio 2022 kao Azure Cloud Service sa (extended support) projekat (Slika 2.1.), koristi se Azure Resource Manager za upravljanje skalabilnim servisima, što omogućava efikasno rukovanje resursima i skaliranje aplikacije prema potrebama korisnika (projekte koje smo kreirali u C# smo nabrojali u prvoj sekciji). Projekat je razvijen koristeći C# programski jezik, a za web rolu je korišćen ASP.NET MVC okvir, što je omogućilo efikasnu organizaciju i upravljanje aplikacijom.

Ključne karakteristike Visual Studio [2022]:

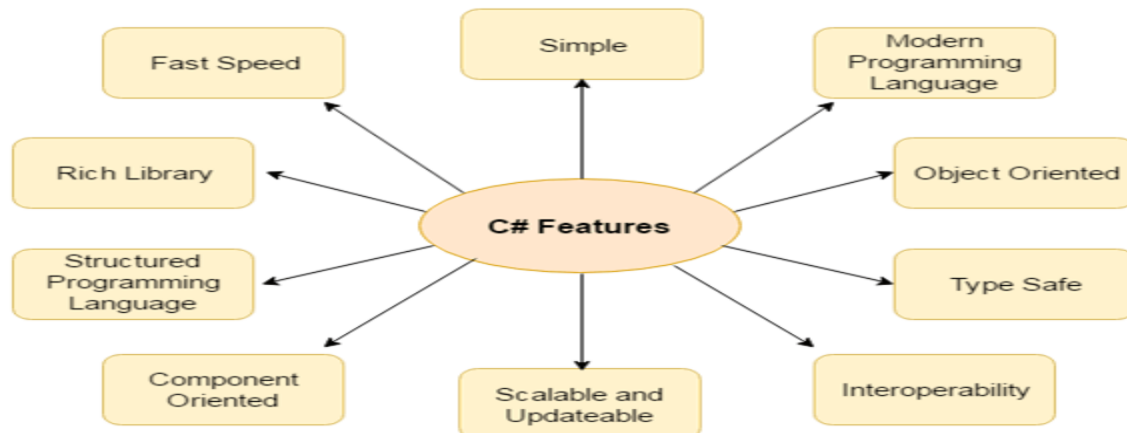
- IntelliSense i refaktoriranje koda: Automatsko dovršavanje i lakše restrukturiranje koda.
- Program za otklanjanje grešaka: Detaljna analiza i ispravljanje grešaka.
- Profilisanje koda i alati za analizu performansi: Identifikacija uskih grla i optimizacija performansi.
- Dizajner za izgradnju GUI aplikacija: Brzo i jednostavno kreiranje korisničkih interfejsa.
- Podrška za različite programske jezike: Podrška za širok spektar jezika, uključujući C#, C++, Python, JavaScript i druge.
- Integracija sa sistemima kontrole verzija: Integracija sa Git i Subversion za efikasno upravljanje kodom.
- Azure DevOps integracija: Upravljanje životnim ciklusom razvoja softvera i automatizacija procesa isporuke.

C# Programsko Okruženje:

C# je moderan, objektno orijentisan programski jezik, poznat po jednostavnosti, brzini i sigurnosti. Neke od ključnih karakteristika C# uključuju (Slika 2.1.1.):

- Jednostavnost: Lak za učenje i korišćenje.
- Brzina: Brz i efikasan u izvršavanju koda.

- Objektno orijentisan: Podržava OOP paradigme kao što su enkapsulacija, nasleđivanje i polimorfizam.
- Sigurnost tipova: Sprečava greške vezane za tipove podataka.
- Interoperabilnost: Može se integrisati sa drugim jezicima i platformama.
- Skalabilnost i ažuriranje: Pogodan za razvoj skalabilnih aplikacija koje je lako ažurirati



Slika 2.1.1 – Karakteristike C#

2.2. JavaScript, HTML, CSS

JavaScript:

JavaScript je skriptni jezik koji se koristi za izradu dinamičnih i interaktivnih elemenata na veb stranicama. Od svog nastanka, JavaScript se razvio u jedan od najvažnijih jezika za veb razvoj, omogućavajući programerima da dodaju funkcionalnosti koje nadmašuju statičke mogućnosti HTML-a i CSS-a. U ovom projektu, JavaScript je korišćen na WebRole komponenti u folderu Scripts i u AdminTools (Console Application) komponenti za potrebe implementacije SignalR biblioteke otvorenog koda, na koju ćemo se kasnije više posvetiti u narednim poglavljima. Primer JavaScript koda dat je na slici 2.2.1.

```

// JavaScript za validaciju polja i potvrdu prijave
function ValidateLogin() {
    const email = document.getElementById("email").value.trim();
    const password = document.getElementById("password").value.trim();

    if (email.length === 0) {
        alert("Polje za email mora biti popunjeno!");
        return false;
    }
    else if (!/^[a-zA-Z0-9@.]*$/i.test(email) || !email.includes('@') || !email.includes('.') ) {
        alert("Email adresa nije validna!");
        return false;
    }
    else if (password.length === 0) {
        alert("Polje za lozinku mora biti popunjeno!");
        return false;
    }
    else if (password.length < 6) {
        alert("Lozinka nije validna! Lozinka mora imati minimalno 6 karaktera.");
        return false;
    }
    else {
        alert("Prijavili ste se!");
        return true;
    }
}

function ValidateAndSubmit() {
    if (ValidateLogin()) {
        document.querySelector("form").submit();
    }
}
  
```

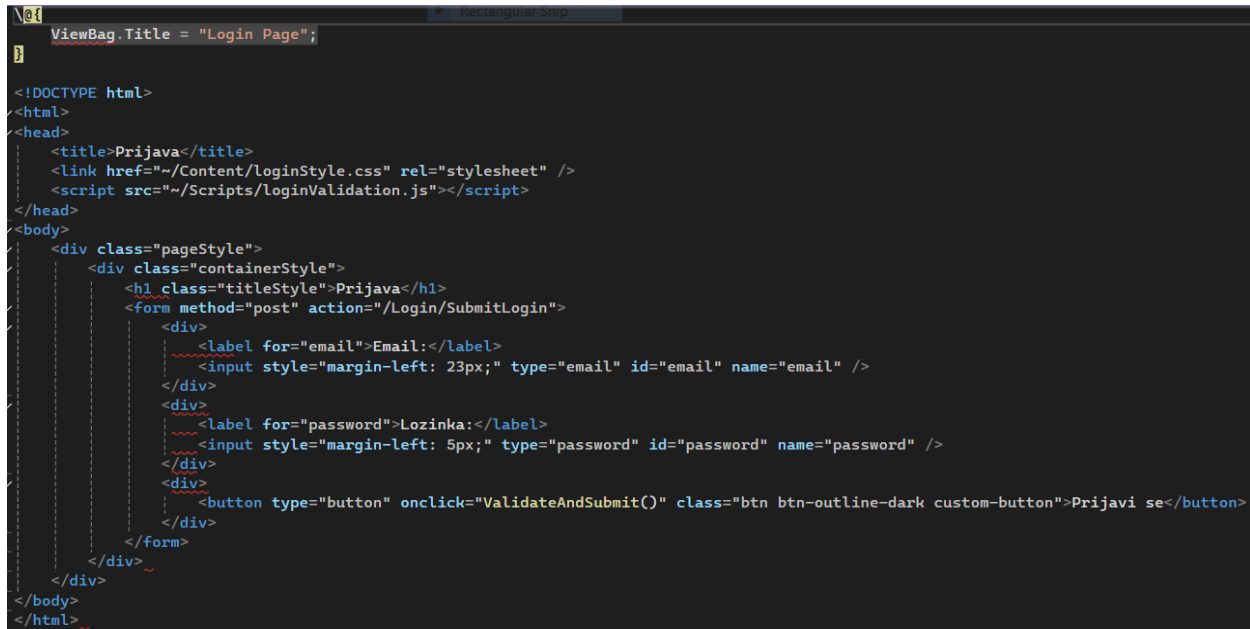
Slika 2.2.1 – JavaScript kod primjer

Ovaj JavaScript kod je korišćen za validaciju stranice Login i nazvan je loginValidation.js.

- Ključne karakteristike JavaScripta:
 - Interaktivnost: JavaScript omogućava kreiranje interaktivnih elemenata na veb stranicama, kao što su meniji, dijaloški okviri, validacija obrazaca i animacije. Ova interaktivnost značajno poboljšava korisničko iskustvo.
 - Manipulacija DOM-om: JavaScript omogućava dinamičku manipulaciju Document Object Model-a (DOM), što znači da se sadržaj stranice može menjati u realnom vremenu bez potrebe za ponovnim učitavanjem stranice.
 - Podrška za različite biblioteke i okvire: JavaScript ekosistem uključuje brojne biblioteke i okvire kao što su jQuery, React, Angular i Vue.js, koje olakšavaju i ubrzavaju razvoj složenih veb aplikacija.
 - Asinhrono programiranje: JavaScript podržava asinhronne operacije kroz callback funkcije, promese i async/await sintaksu, omogućavajući efikasno upravljanje operacijama koje traju duže, kao što su HTTP zahtevi.

HTML:

HTML (HyperText Markup Language) je osnovni jezik za izradu veb stranica. On definiše strukturu i sadržaj veb stranica koristeći jednostavne oznake (tagove). HTML je temeljna tehnologija na kojoj počiva veb razvoj i služi kao kostur svake veb stranice. U ovom projektu, HTML se koristi u WebRole u folderu Views i folderima unutar njega potrebnim za svaku stranicu za prikaz.



```

ViewBag.Title = "Login Page";

<!DOCTYPE html>
<html>
<head>
<title>Prijava</title>
<link href="~/Content/loginStyle.css" rel="stylesheet" />
<script src="~/Scripts/loginValidation.js"></script>
</head>
<body>
<div class="pageStyle">
<div class="containerStyle">
<h1 class="titleStyle">Prijava</h1>
<form method="post" action="/Login/SubmitLogin">
<div>
<label for="email">Email:</label>
<input style="margin-left: 23px;" type="email" id="email" name="email" />
</div>
<div>
<label for="password">Lozinka:</label>
<input style="margin-left: 5px;" type="password" id="password" name="password" />
</div>
<div>
<button type="button" onclick="ValidateAndSubmit()" class="btn btn-outline-dark custom-button">Prijava se</button>
</div>
</form>
</div>
</div>
</body>
</html>

```

Slika 2.2.2. – HTML kod primjer

Primer html koda dat je na slici 2.2.2. i on se koristi za implementaciju i prikaz Login stranice, na slici je prikazano kako se poziva funkcija ValidateAndSubmit() iz JavaScript koda koji je dat na slici 2.2.1. kao i CSS kod dat na slici 2.2.3.

- Ključne karakteristike HTML-a:
 - Struktura stranice: HTML koristi oznake za definisanje različitih elemenata stranice kao što su naslovi, paragrafi, liste, tabele, slike i linkovi. Ove oznake definišu kako će sadržaj biti organizovan i prikazan na stranici.
 - Semantički HTML: HTML5 uvodi semantičke oznake kao što su <header>, <footer>, <article> i <section>, koje poboljšavaju čitljivost koda i olakšavaju rad pretraživačima i pomoćnim tehnologijama.
 - Multimedijalna podrška: HTML5 donosi napredne funkcionalnosti za ugradnju multimedijalnih sadržaja kao što su video i audio, omogućavajući bogatije korisničko iskustvo bez potrebe za dodatnim dodacima.

CSS:

CSS (Cascading Style Sheets) je jezik za opisivanje stilova i izgleda HTML elemenata na veb stranici. CSS omogućava programerima da odvoje sadržaj od prezentacije, što doprinosi boljoj organizaciji i lakšem održavanju koda. U ovom projektu, CSS se koristi u WebRole u folderu Content.

```

.pageStyle {
    text-align: center;
    background-image: url('/Images/Backgrounds/login.jpg');
    background-size: cover;
    background-position: center;
    height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
}

.containerStyle {
    text-align: center;
    background-color: white;
    width: 350px;
    height: 200px;
    border-radius: 8px;
}

.titleStyle {
    margin-top: 15px;
    font-family: 'Times New Roman';
    font-weight: bold;
    color: black;
}

label {
    font-family: 'Times New Roman';
    font-weight: bold;
    font-size: 20px;
    color: black;
}

input {
    margin-top: 5px;
    font-family: 'Times New Roman';
    font-weight: bold;
    color: black;
}

.custom-button {
    margin-top: 10px;
}

```

Slika 2.2.3. – CSS kod primjer

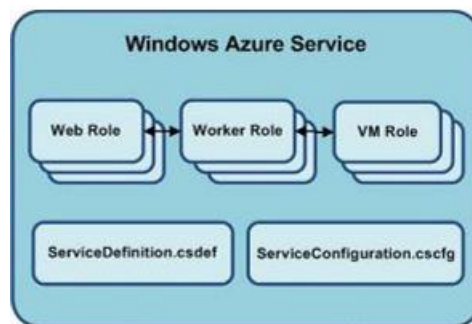
Slika 2.2.3. je primer CSS koda koji je korišten za implementaciju stranice Login.

- Ključne karakteristike CSSa:
 - Stilizacija stranice: CSS omogućava definisanje stilova za različite HTML elemente putem selektora, uključujući boje, fontove, margine, padding, pozicije i dimenzije. Ovi stilovi se primenjuju kroz selektore i pravila.
 - Responsivni dizajn: CSS podržava tehnike responsivnog dizajna kao što su medija upiti, fleksibilne mreže (flexbox) i CSS Grid, omogućavajući da veb stranice budu prilagođene različitim uređajima i veličinama ekrana.

- Animacije i tranzicije: CSS omogućava kreiranje jednostavnih animacija i tranzicija, koje poboljšavaju vizuelni efekat i interaktivnost veb stranica, bez potrebe za korišćenjem JavaScript-a.

2.3. Azure Cloud Service (extended support)

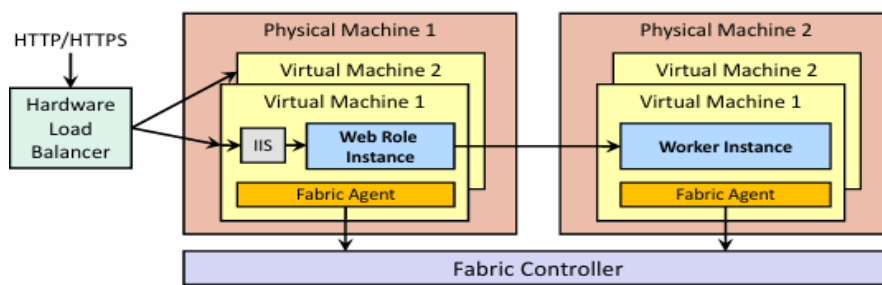
Azure Cloud Service je robustna platforma koja omogućava razvoj, implementaciju i upravljanje aplikacijama u oblaku. Koristeći Azure Cloud Service, programeri mogu brzo i efikasno razvijati skalabilne i pouzdane aplikacije, prilagođene različitim potrebama i opterećenjima. Azure Cloud Service pruža podršku za različite tipove servisa, kao što su WebRole i WorkerRole (Slika 2.2.1.), koji omogućavaju razdvajanje frontend i backend logike aplikacije.



Slika 2.3.1. Windows Azure Service

Ključne karakteristike Azure Cloud Service-a, zbog čega je korišten za implementaciju:

- Skalabilnost i fleksibilnost: Azure Cloud Service omogućava horizontalnu i vertikalnu skalabilnost aplikacija. Horizontalna skalabilnost postiže se dodavanjem novih instanci servisa, dok se vertikalna skalabilnost ostvaruje promenom konfiguracije resursa svake instance.
- WebRole i WorkerRole: WebRole se koristi za hostovanje veb aplikacija i API-ja, dok WorkerRole omogućava izvršavanje pozadinskih zadataka i obradu podataka. Ova podela omogućava efikasnu raspodelu posla i bolju upravljivost sistema (Slika 2.2.2.).

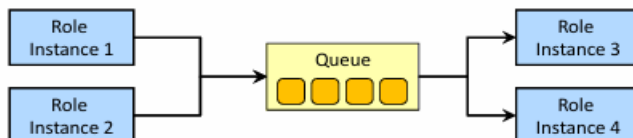


- **Web Role:** predstavlja jednu .NET application koja se pokreće u IIS:
 - Web Application ili Web Service (WCF)
- **Worker Role:** Servisi koji služe za proračun
 - Obično su izolovani od spoljnog sveta
- **Fabric Agent** skuplja podatke o radu sistema (broj korisnika, kvarova, ...)

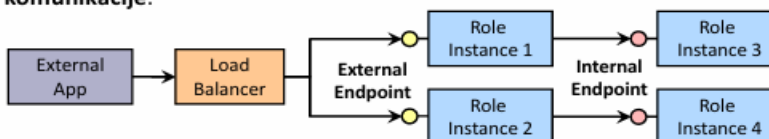
Slika 2.3.2. Arhitektura WebRole i WorkerRole servisa

U ovom projektu je korišćena komunikacija izmedju WebRola i WokerRola preko HTTP(S)-a. Postoje komunikacije preko Queue-ova (redova čekanja) i preko TCP-a (Slika 2.3.3.).

- Role komuniciraju na asinhroni način pomoću **queue-ova**.
 - To je preporučeni način komunikacije za pouzdani prenos poruka



- Role mogu da komuniciraju i direktno **upotrebom TCP ili HTTP(S) komunikacije**.

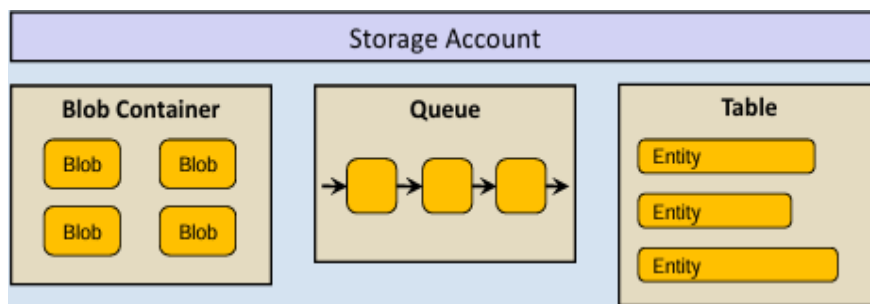


Slika 2.3.3. Komunikacija izmedju Web/Worker rola

- Visoka dostupnost i pouzdanost: Azure Cloud Service obezbeđuje visoku dostupnost i pouzdanost aplikacija putem redundantne infrastrukture i automatskog oporavka od grešaka. Servisi su dizajnirani da budu otporni na kvarove i brzo se oporavljaju u slučaju problema.
- Integracija sa drugim Azure servisima: Azure Cloud Service se jednostavno integriše sa drugim Azure servisima, kao što su Azure Storage, Azure SQL Database i Azure Active Directory, što omogućava sveobuhvatno rešenje za razvoj i upravljanje aplikacijama.
- Upravljanje putem Azure Portala: Azure Portal omogućava jednostavno upravljanje, monitoring i konfiguraciju Azure Cloud Service-a. Kroz Azure Portal, programeri mogu lako pratiti performanse, upravljati resursima i konfiguracijama, i brzo reagovati na promene u opterećenju.

2.4. Azure Storage Service

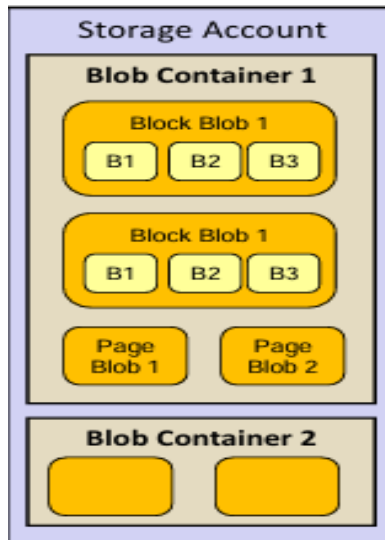
Azure Cloud Storage je servis za skladištenje podataka u oblaku koji nudi sigurnost, pouzdanost i skalabilnost. Ovaj servis je korišćen u projektu za efikasno čuvanje i upravljanje podacima. Ključne karakteristike Azure Cloud Storage-a uključuju:



Slika 2.4.1. Arhitektura skladišta

Blob Storage:

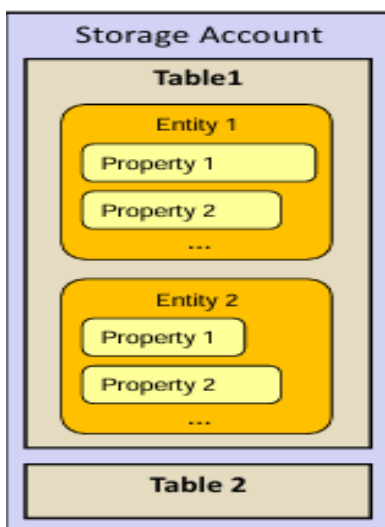
Služi za čuvanje nestruktuiranih podataka, kao što su slike, video zapisi i dokumenti. Blob Storage je organizovan u kontejnere, koji mogu biti javni ili privatni. Postoje Block i Page tipovi blobova. Blob Storage u ovom projektu se koristi za smeštanje slika korisnika i slike postova što će biti prikazano u narednim poglavljima.



Slika 2.4.2. Arhitektura Blob Storage-a

Table Storage:

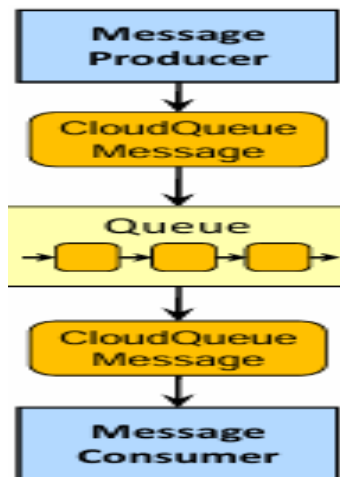
Omogućava čuvanje struktuiranih podataka u obliku entiteta. Svaki entitet sadrži skup svojstava (properties) i ne zahteva fiksnu šemu, što omogućava veću fleksibilnost u organizaciji podataka. U ovom projektu je korišćeno za smeštanje svih ostalih podataka u aplikaciji koje ne smeštaju Queue i Blob Storage što će biti prikazano u narednim poglavljima.



Slika 2.4.3. Arhitektura Table Storage-a

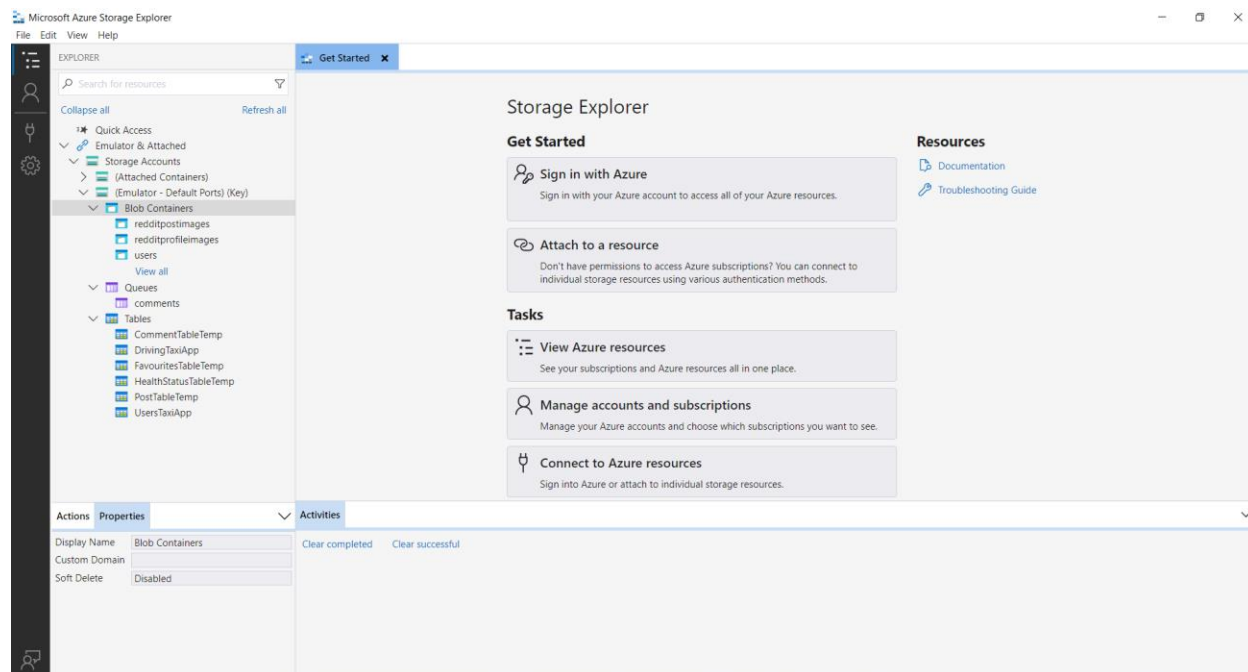
Queue Storage:

Omogućava pouzdanu asinhronu razmenu poruka između različitih delova sistema. Queue Storage podržava FIFO (First In, First Out) sistem za redove čekanja, što omogućava efikasno upravljanje porukama. U ovom projektu korišćeno je za smeštanje podatka o komentarima na postove od strane korisnika koji se kasnije šalju vlasniku komentara koji je favorizovao taj post što će biti prikazano u narednim poglavljima.



Slika 2.4.4. Arhitektura Queue Storage-a

Visoka pouzdanost i skalabilnost: Azure Cloud Storage nudi visoku pouzdanost putem repliciranja podataka na više nodova u različitim fault domenima. Takođe, omogućava lako skaliranje i pristup velikim količinama podataka. Na slici 2.4.5. prikazan je izgled baze podataka aplikacije.



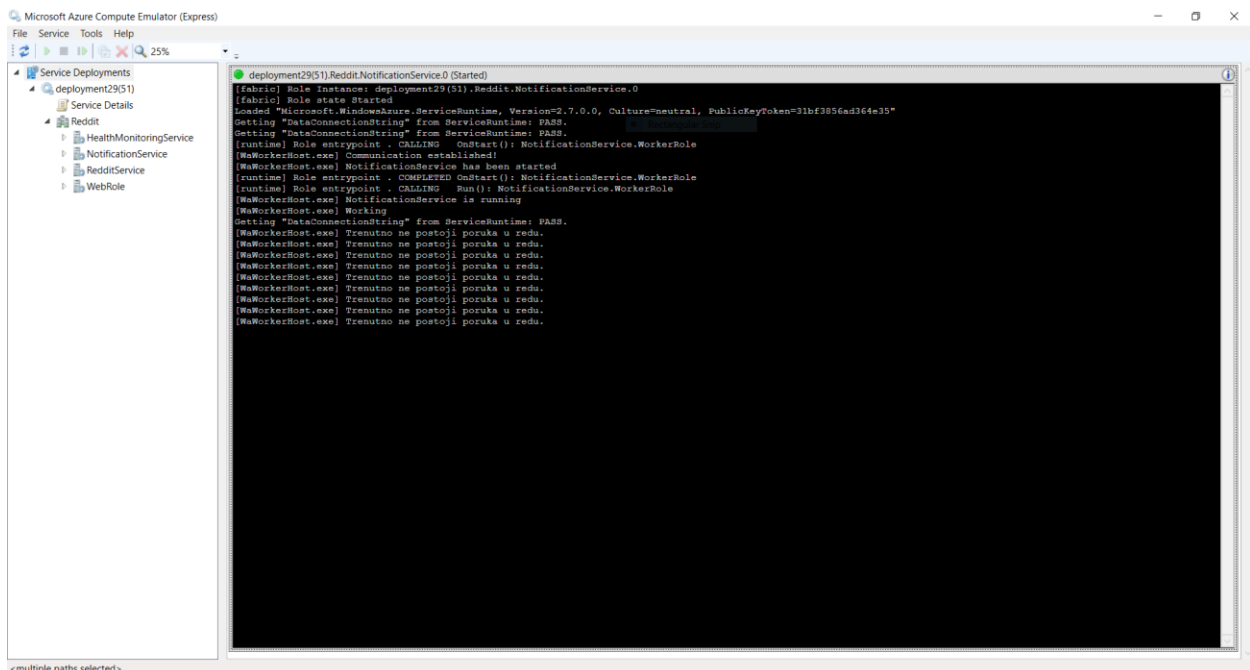
Slika 2.4.5. Baza podataka aplikacije u Microsoft Azure Storage Explorer-u

2.4.1. Microsoft Azure Compute Emulator

Alatka koja omogućava programerima lokalno testiranje i debugiranje Azure aplikacija pre nego što ih implementiraju u Azure oblak. Ovaj emulator simulira Azure okruženje, pružajući razvojni alat koji oponaša ponašanje aplikacija u oblaku.

Slika 2.4.1. prikazuje radno okruženje Microsoft Azure Compute Emulator-a sa nekoliko pokrenutih servisa. Ovo uključuje HealthMonitoringService, NotificationService, RedditService i WebRole. Prikaz stanja servisa i logova pomaže programerima da prate performanse i identifikuju potencijalne probleme u aplikacijama pre nego što ih implementiraju u Azure oblak.

Microsoft Azure Compute Emulator omogućava razvojnom timu efikasno testiranje, debugiranje i optimizaciju aplikacija, kako bi bile spremne za proizvodno okruženje u oblaku.



Slika 2.4.1. Microsoft Azure Compute Emulator

Ključne karakteristike Microsoft Azure Compute Emulator-a:

- **Simulacija Azure okruženja:** Azure Compute Emulator omogućava programerima da lokalno testiraju svoje aplikacije u okruženju koje oponaša Azure oblak. Ovo omogućava brže iteracije i testiranje bez potrebe za stalnim implementacijama u oblak.
- **Debugiranje i monitoring:** Emulator pruža sveobuhvatne mogućnosti za debugiranje i monitoring aplikacija, omogućavajući programerima da identifikuju i ispravljaju greške pre nego što aplikaciju postave u proizvodno okruženje.

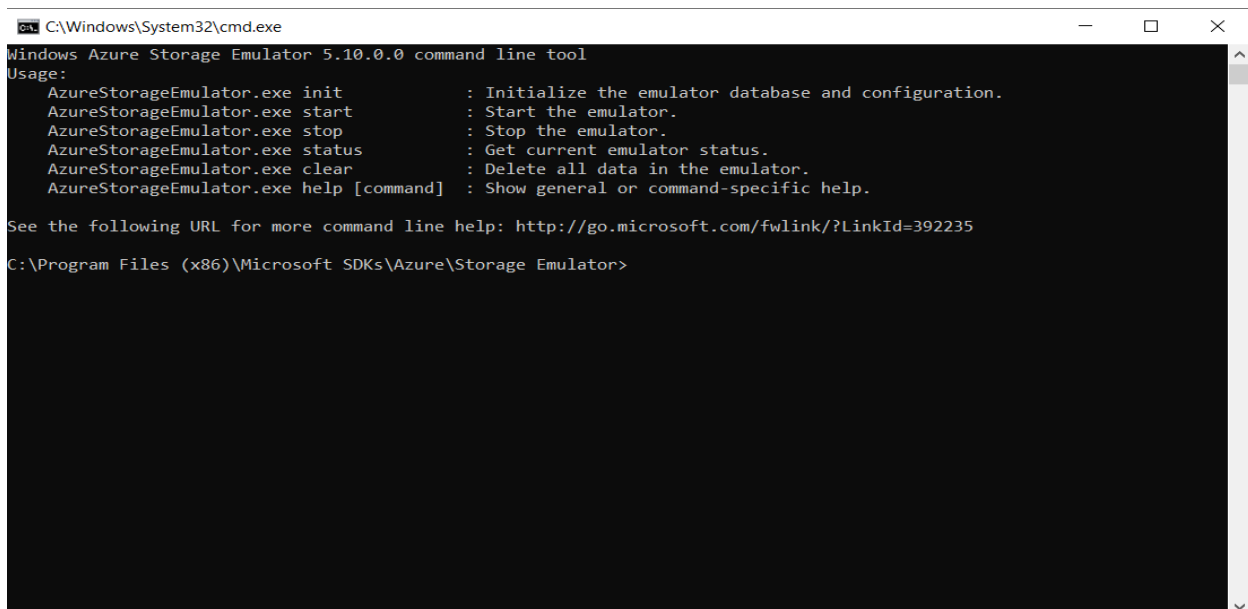
- Upravljanje servisima: Kroz emulaciju, programeri mogu upravljati servisima kao što su WebRole i WorkerRole, testirajući kako različiti delovi aplikacije funkcionišu zajedno pre implemenacije na Azure.
- Lokalni razvoj: Koristeći Compute Emulator, razvojni timovi mogu raditi na aplikacijama lokalno, bez potrebe za stalnim povezivanjem sa Azure oblakom, što smanjuje troškove i povećava produktivnost.
- Prikaz stanja servisa: Emulator prikazuje detaljne informacije o stanju servisa, uključujući logove, status instanci i performanse, što omogućava dubinsku analizu i optimizaciju aplikacija.

2.4.2. Microsoft Azure Storage Emulator

Microsoft Azure Storage Emulator omogućava programerima lokalno testiranje i debugiranje Azure Storage aplikacija pre implementacije u Azure oblak. Ovaj emulator pruža funkcionalnost koja oponaša ponašanje stvarnih Azure Storage servisa, omogućavajući razvoj i testiranje aplikacija bez potrebe za povezivanjem na Azure.

Na slici 2.4.1. prikazana je komandna linija Microsoft Azure Storage Emulator-a sa osnovnim komandama za upravljanje emulatorom, uključujući inicijalizaciju, pokretanje, zaustavljanje i proveru statusa. Ove komande omogućavaju programerima da efikasno upravljaju emuliranim Azure Storage okruženjem tokom razvoja i testiranja aplikacija.

Microsoft Azure Storage Emulator omogućava razvojnom timu efikasno testiranje, debugiranje i optimizaciju aplikacija koje koriste Azure Storage servise, pripremajući ih za razvojno okruženje.

The image shows a Windows command prompt window titled "C:\Windows\System32\cmd.exe". The prompt displays the "Windows Azure Storage Emulator 5.10.0.0 command line tool" usage information. The text is as follows:

```
Windows Azure Storage Emulator 5.10.0.0 command line tool
Usage:
  AzureStorageEmulator.exe init           : Initialize the emulator database and configuration.
  AzureStorageEmulator.exe start          : Start the emulator.
  AzureStorageEmulator.exe stop           : Stop the emulator.
  AzureStorageEmulator.exe status         : Get current emulator status.
  AzureStorageEmulator.exe clear          : Delete all data in the emulator.
  AzureStorageEmulator.exe help [command] : Show general or command-specific help.

See the following URL for more command line help: http://go.microsoft.com/fwlink/?LinkId=392235

C:\Program Files (x86)\Microsoft SDKs\Azure\Storage Emulator>
```

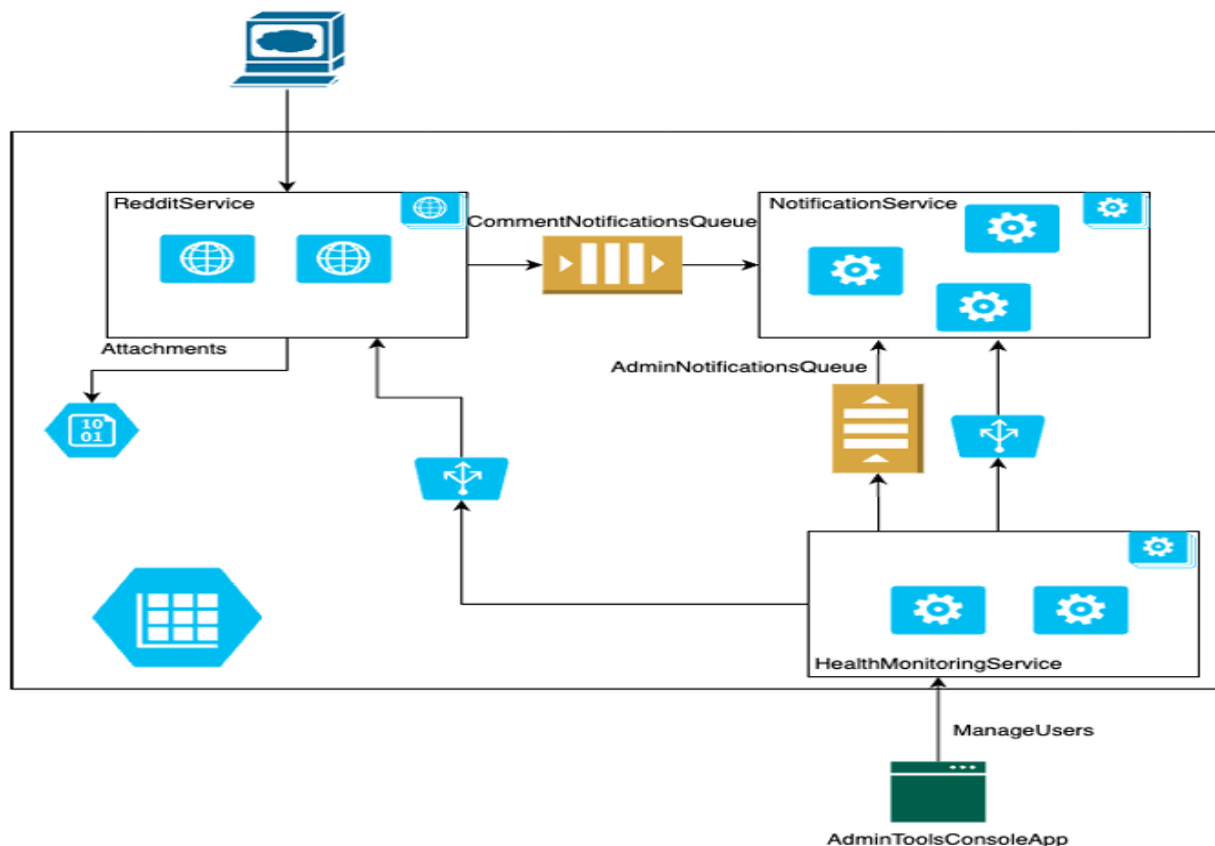
Slika 2.4.1. Microsoft Azure Compute Emulator

Ključne karakteristike Microsoft Azure Storage Emulator-a:

- Simulacija Azure Storage okruženja: Azure Storage Emulator omogućava lokalno testiranje aplikacija koje koriste Azure Storage, simulirajući ponašanje stvarnih servisa.
- Komandna linija za upravljanje: Emulator se pokreće i kontroliše putem komandne linije, omogućavajući inicijalizaciju, pokretanje, zaustavljanje, proveru statusa i brisanje podataka.
- Debugging i monitoring: Pruža sveobuhvatne mogućnosti za debugiranje i monitoring aplikacija, omogućavajući programerima da identifikuju i isprave greške pre postavljanja u porizvodno okruženje.
- Lokalni razvoj: Koristeći Storage Emulator, razvojni timovi mogu lokalno razvijati i testirati aplikacije bez potrebe za stalnim povezivanjem sa Azure oblakom, što smanjuje troškove i povećava produktivnost.

3.Specifikacija sistema

Celokupna ideja je da se implementira sistem za pravljenje različitih forumskih tema i komentara, sličan Reddit-u. Sistem treba implementirati tako da poseduje zasebne servise za slanje notifikacija (NotificationService) i proveru dostupnosti servisa (HealthMonitoringService), prikaz sistema na slici 3.0.1., komponente su:



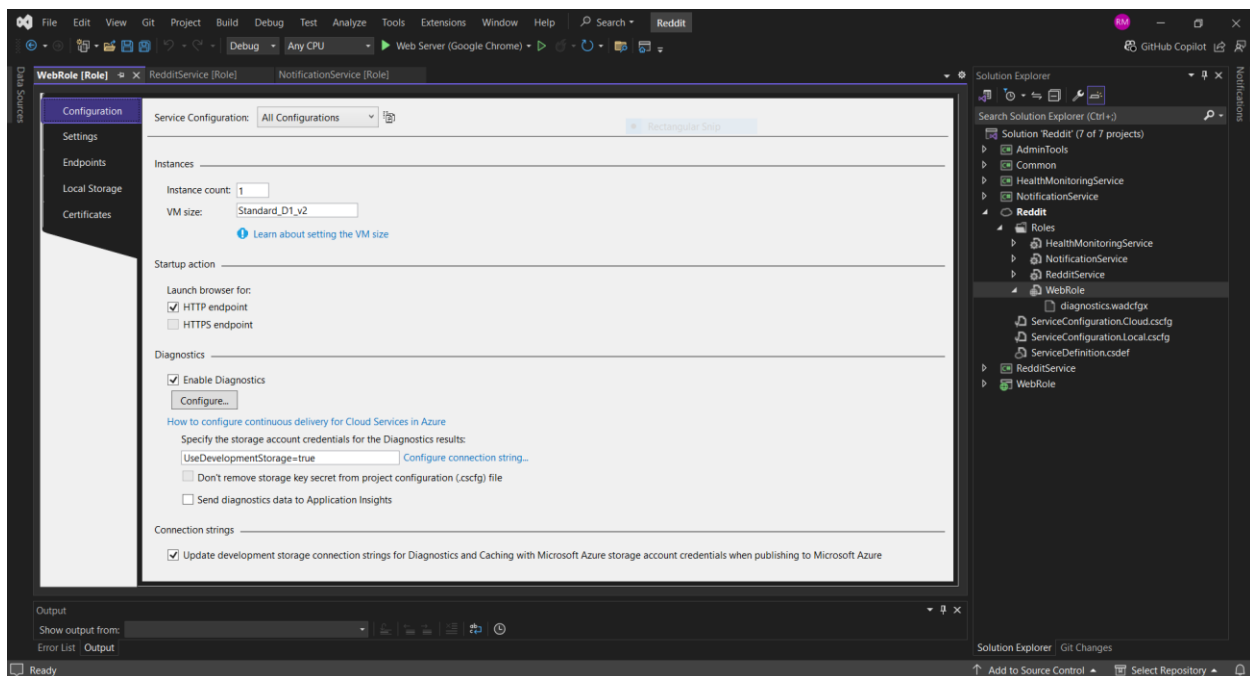
Slika 3.0.1. Specifikacija sistema za implementaciju

3.1. WebRole services

WebRole – WebRole je aplikacija koja služi kao korisnički interfejs za interakciju sa forumom:

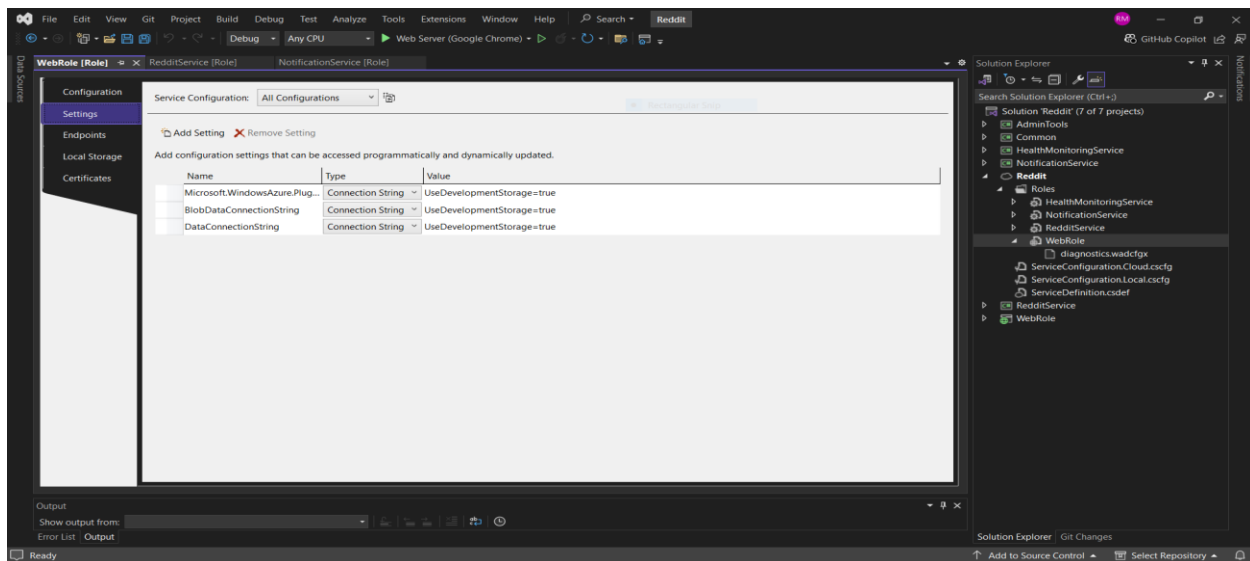
- Najjednostavnije je koristiti MVC koji treba da ima vizuelni prikaz dostupnosti RedditService servisa.
- Funkcionalnosti koje treba podržati na korisničkom interfejsu su: registracija novog korisnika, logovanje postojećeg korisnika, izmena korisničkog profila, pravljenje teme - novi post, pravljenje novog komentara od strane bilo kog korisnika na bilo koju temu, pretraga i sortiranje tema na osnovu naslova teme, brisanje teme ili ostavljenog komentara na neku temu, Upvote/Downvote za teme i Upvote/Downvote za komentara postavljene na odgovarajuće teme, to jeste na slici su prikazani kao Attachments (prilozi).

- Korisnik se registruje unošenjem: Ime, Prezime, Adresa, Grad, Država, Broj telefona, Email, Lozinka, Slika korisnika
- Korisnik se loguje putem: Email, Lozinka – koja treba da bude hešovana prilikom upisa i čitanja iz baze podataka.
- Opšti opis korisničkog iskustva: Nakon pokretanja aplikacije, korisnik je dostupan chatbot sa spiskom često postavljanih pitanja i opcijom za unos sopstevnih pitanja za pomoć. Novi korisnik, nakon što kreira nalog, dolazi na stranicu sa temama. Na ovoj stranici, inicijalno će videti prazan spisak svojih tema, sa opcijom kreiranja nove teme. Ispod ovog dela, prikazan je spisak svih postojećih tema, dok se ispod tog spiska nalazi lista omiljenih tema, sortirana prema broju pretplaćenih korisnika. U okviru dela sa svim temama, korisnik može da pregleda bilo koju temu, da se pretplati na neku temu ili da ostavi komentar. Kada bilo koji korisnik ostavi komentar na neku temu, svi pretplaćeni korisnici dobijaju obaveštenje putem mejla sa sadržajem komentara. Stranice sa temama treba paginirati kako bi bile pregledne. Takođe, potrebno je omogućiti korisnicima da prikaže slike uz svoje postove, kao i da postave svoju korisničku fotografiju.



Slika 3.1.1. WebRole Configuration

Na slici 3.1.1. prikazana je konfiguracija WebRole aplikacije unutar Azure Cloud servisa. Broj instanci je podešen na 1, što znači da će se pokrenuti jedna instanca aplikacije. Veličina VM je postavljena na Standard_D1_v2, određujući veličinu virtuelne mašine koja će biti korišćena. Pokretanje pretraživača je automatski podešeno za HTTP endpoint, omogućavajući lakši pristup aplikaciji tokom razvoja. Omogućena je dijagnostika za praćenje i analizu logova i performansi aplikacije, koristeći UseDevelopmentStorage=true za razvojno skladište. Ova podešavanja omogućavaju osnovnu konfiguraciju WebRole aplikacije za razvoj i testiranje u Azure okruženju

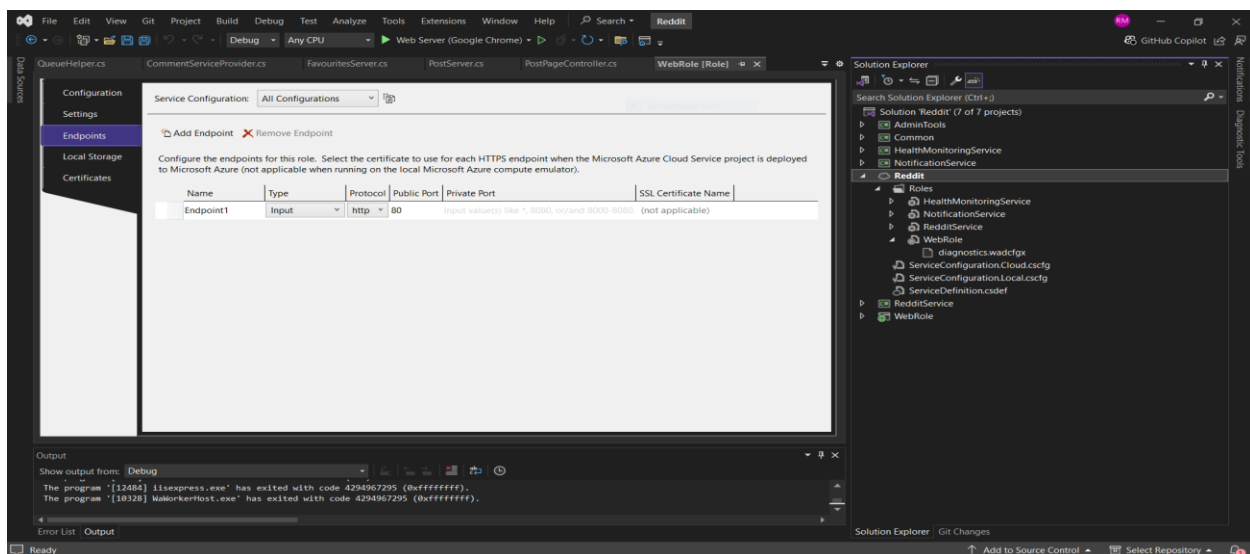


Slika 3.1.2. WebRole Settings

Na slici 3.1.2. prikazana su podešavanja različitih konekcionih stringova koje koristi WebRole unutar Azure Cloud servisa.

Konekcioni string za dijagnostiku (Microsoft.WindowsAzure.Plugins.Diagnostics.ConnectionString) je tipa Connection String i postavljen je na vrednost UseDevelopmentStorage=true, omogućavajući korišćenje lokalnog skladišta tokom razvoja. Konekcioni string za blob podatke (BlobDataConnectionString) je tipa Connection String sa vrednošću UseDevelopmentStorage=true, što omogućava skladištenje blob podataka lokalno tokom razvoja. Konekcioni string za ostale podatke (DataConnectionString) je postavljen na UseDevelopmentStorage=true, što omogućava lokalno skladištenje podataka tokom razvoja. Ova podešavanja olakšavaju testiranje i razvoj bez korišćenja produkcionog skladišta, što je detaljno opisano u narednim poglavljima.

Ove postavke omogućavaju WebRole aplikaciji da koristi razvojno skladište za različite tipove podataka, olakšavajući testiranje i debugovanje bez korišćenja produkcionog skladišta.



Slika 3.1.3. WebRole Endpoints

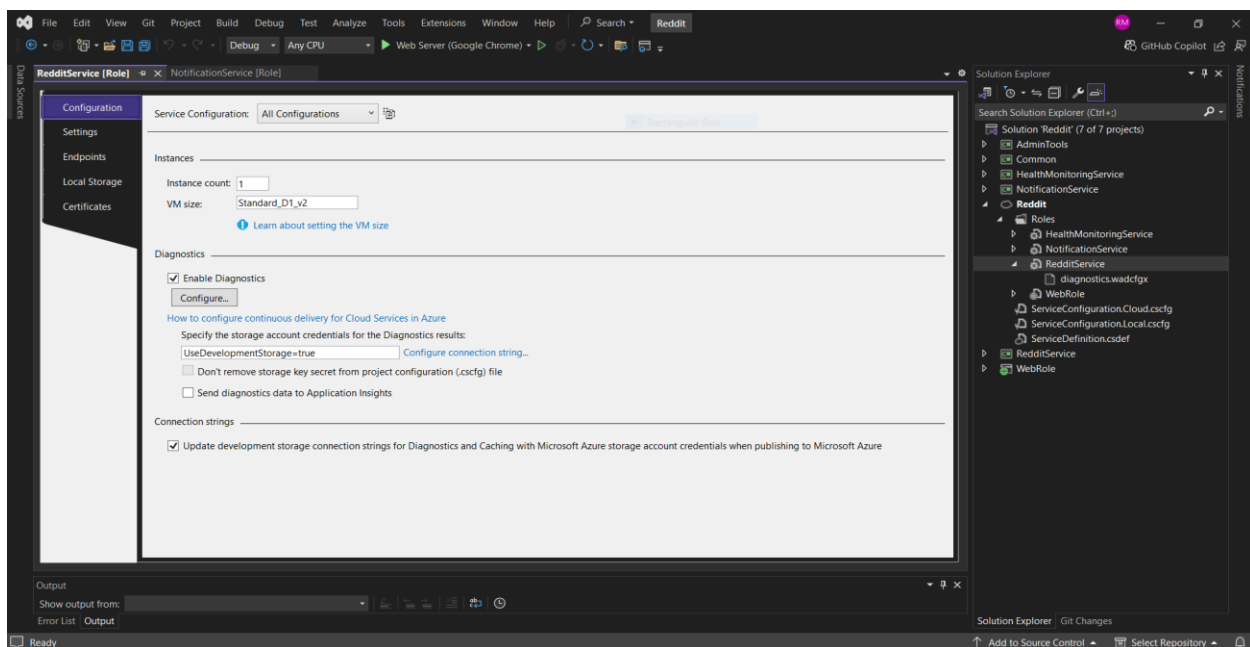
Na slici 3.1.3. prikazane su postavke endpoint-ova koje koristi WebRole unutar Azure Cloud servisa..

Ova postavka omogućava WebRole aplikaciji da prihvata HTTP zahteve na portu 80. To znači da će aplikacija biti dostupna putem standardnog HTTP porta, omogućavajući korisnicima pristup aplikaciji putem web pretraživača. SSL sertifikat nije primenjen u ovoj konfiguraciji, što znači da komunikacija neće biti šifrovana putem HTTPS-a.

3.2. WorkerRole services

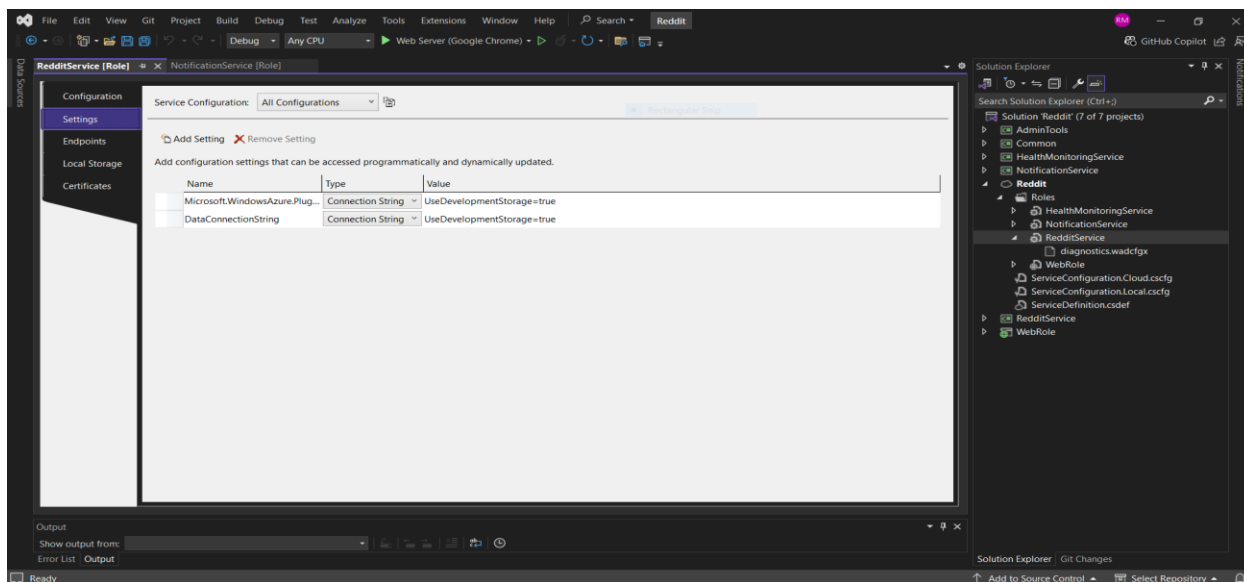
RedditService - Servisni sloj za sve korisničke zahteve:

- Zaseban Worker Role servis koji obavlja sve pozadinske operacije
- Obrađuje korisničke zahteve koje prima iz WebRole preko Common projekta, koji mu obezbeđuje da obavlja i sve ostale zahteve vezane za ostale komponente u projektu, skladišti podatke u bazu.



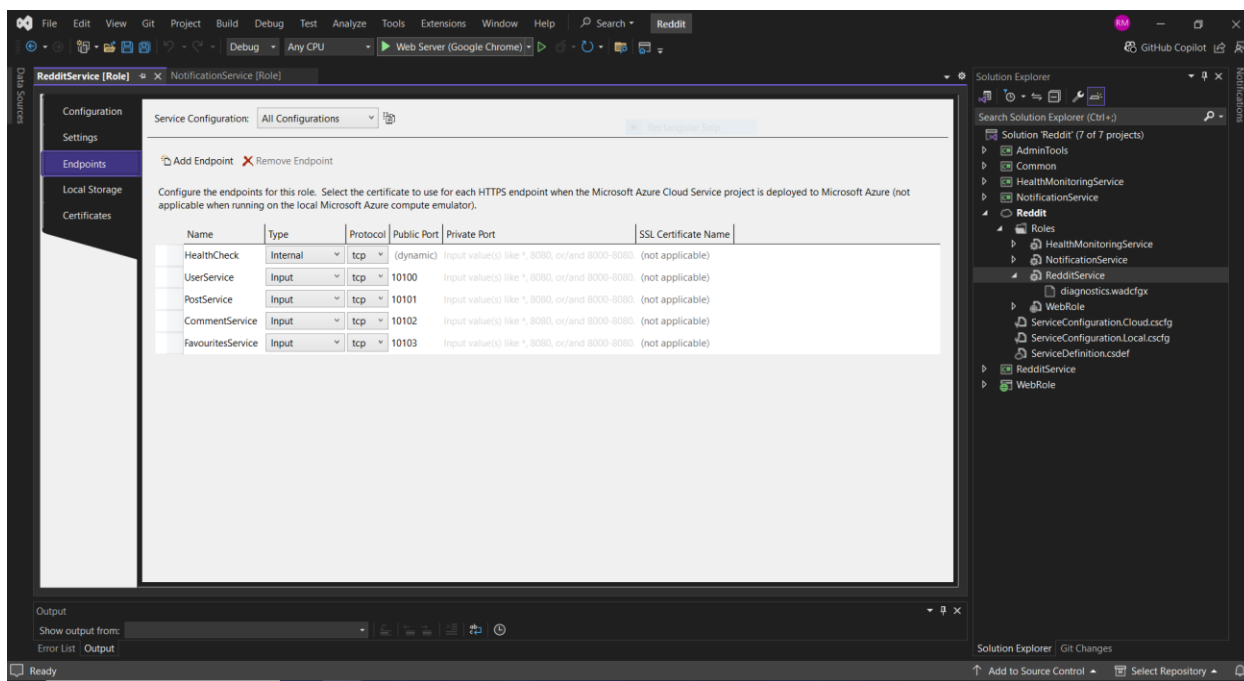
Slika 3.2.1. RedditService Configuration

Slika 3.2.1. prikazuje konfiguraciju za RedditService. Ova konfiguracija osigurava da RedditService koristi jednu instancu virtuelne mašine sa odgovarajućom veličinom, uz omogućenu dijagnostiku radi praćenja performansi i problema tokom razvoja.



Slika 3.2.2. RedditService Settings

Slika 3.2.2 prikazuje podešavanja koja omogućavaju korišćenje lokalnog razvojno skladišta za dijagnostiku i skladištenje podataka, olakšavajući testiranje i razvoj bez potrebe za korišćenjem produkcionog skladišta.



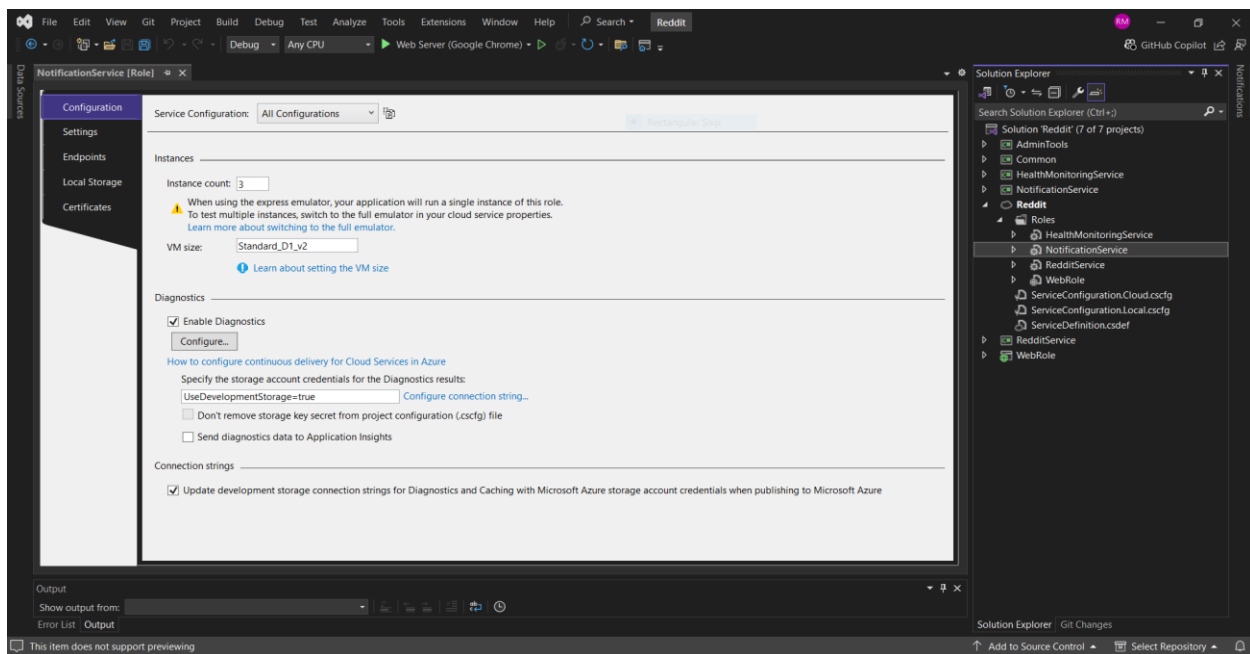
Slika 3.2.3. RedditService Endpoints

Slika 3.2.3. prikazuje podešavanja endpoint-ova za RedditService koje definišu različite servise poput HealthCheck, UserService, PostService, CommentService, i FavouritesService, svaki sa svojim odgovarajućim portovima i protokolima, obezbeđujući organizovanu i bezbednu komunikaciju između servisa.

Ova konfiguracija omogućava RedditService-u da efikasno upravlja korisničkim zahtevima, skladišti podatke i obezbeđuje potrebne servise za celokupan sistem.

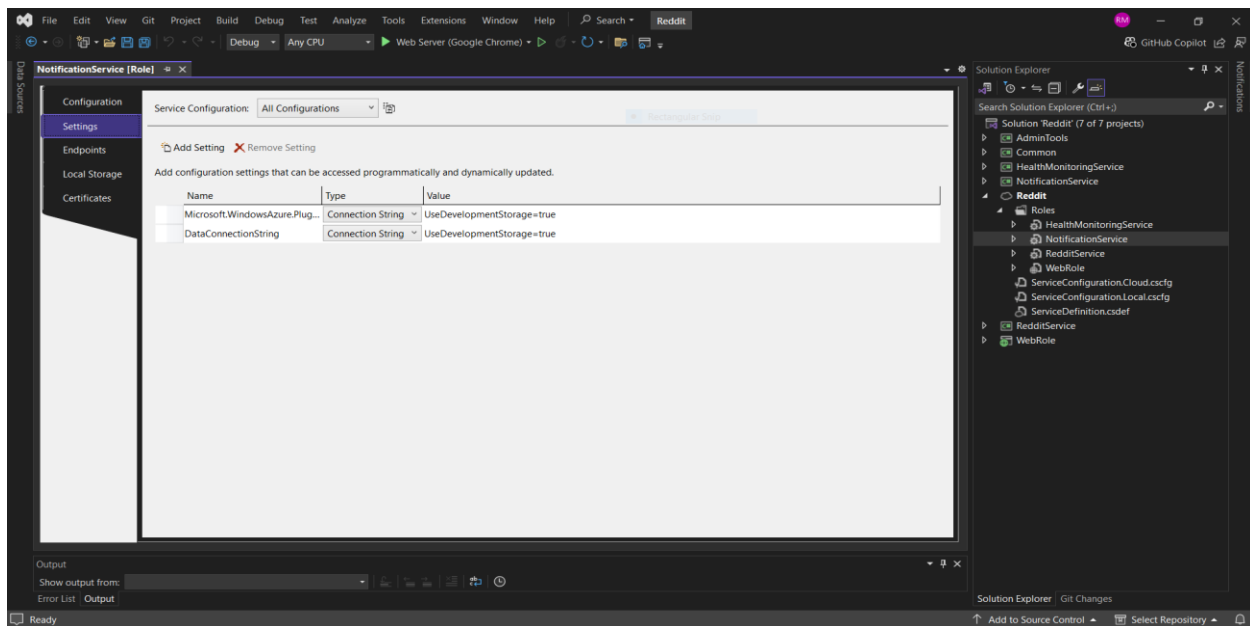
NotificationService - Slanje mejla: Zaseban Worker Role servis za slanje mejlova pretplaćenim korisnicima, ne sme da poremeti rad sistema. Slanje mejla je implementirano kroz sledeće elemente:

- Zaseban Worker Role servis pod nazivom - NotificationService - 3 instance
- Prilikom postavljanja novog komentara na temu - RedditService rola šalje poruku u red (naziv reda comments) gde ubacuje i ID komentara
- NotificationService prilikom čitanja iz reda iz Azure tabele dobija sve podatke o komentaru i ID komentara kao i sve pretplaćene korisnike (njihove mejlove) na tu temu za koju je komentar vezan i šalje mejlove svim korisnicima da je napravljen novi komentar sa tekstom komentara (takođe izvučen tekst komentara)
- Persistovane informacije o svakoj poslatoj grupi notifikacija | datum-vreme | id-komentara | broj-poslatih-mejlova |



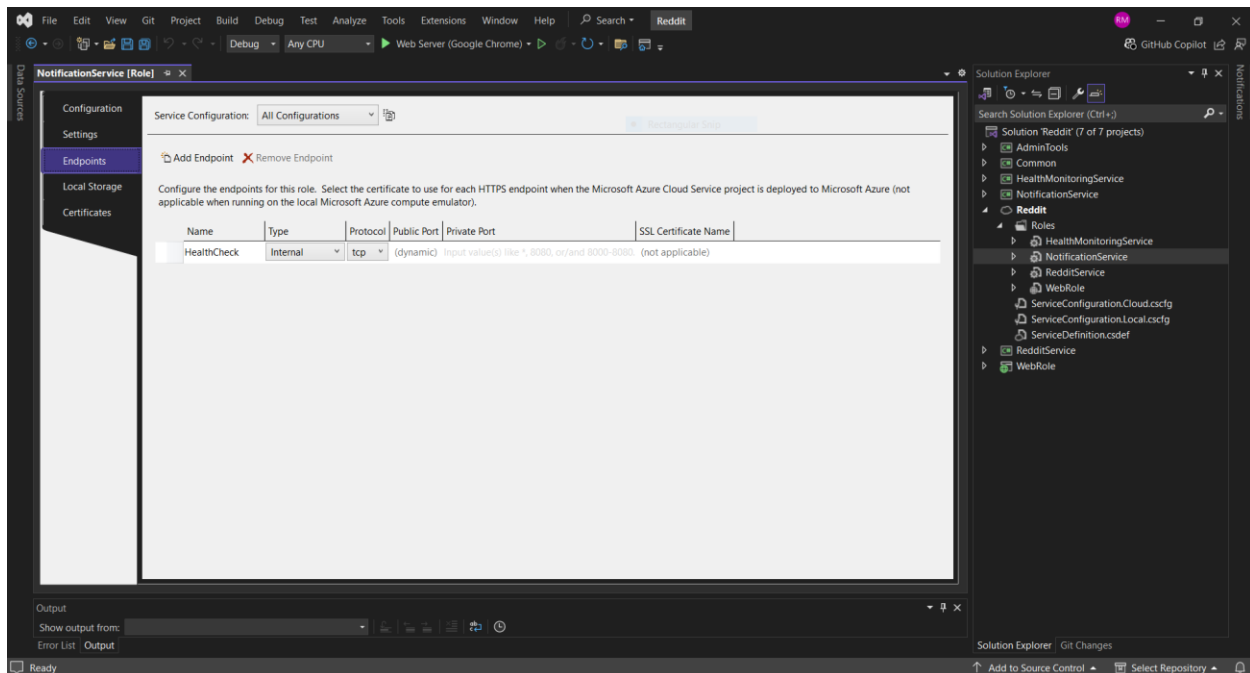
Slika 3.2.4. NotificationService Configuration

Slika 3.2.4. prikazuje konfiguraciju NotificationService-a koje uključuje postavljanje broja instanci na 3, korišćenje Standard_D1_v2 veličine virtualne mašine, i omogućenu dijagnostiku sa lokalnim razvojnim skladištem.



Slika 3.2.5. NotificationService Settings

Slično kao i kod RedditService—a Slika 3.2.5. prikazuje podešavanja konekcionih stringova koje uključuju korišćenje lokalnog razvojno skladišta za dijagnostiku i skladištenje podataka, olakšavajući testiranje i razvoj bez potrebe za korišćenjem produkcionog skladišta.



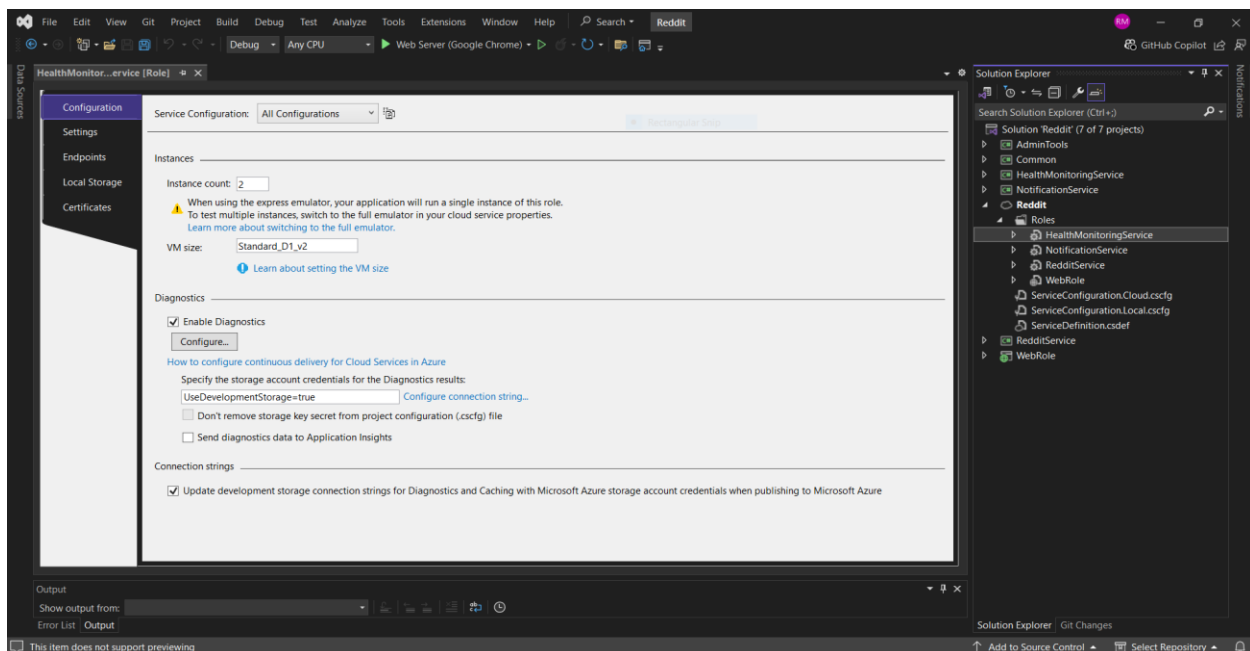
Slika 3.2.6. NotificationService Endpoints

Slika 3.2.6. prikazuje postavke endpoint-ova za NotificationService koje definišu HealthCheck endpoint kao interni sa dinamičkim TCP portom, obezbeđujući proveru zdravlja sistema.

Ova konfiguracija omogućava NotificationService-u da efikasno upravlja slanjem mejlova pretplaćenim korisnicima, obezbeđujući stabilnost i neometan rad sistema.

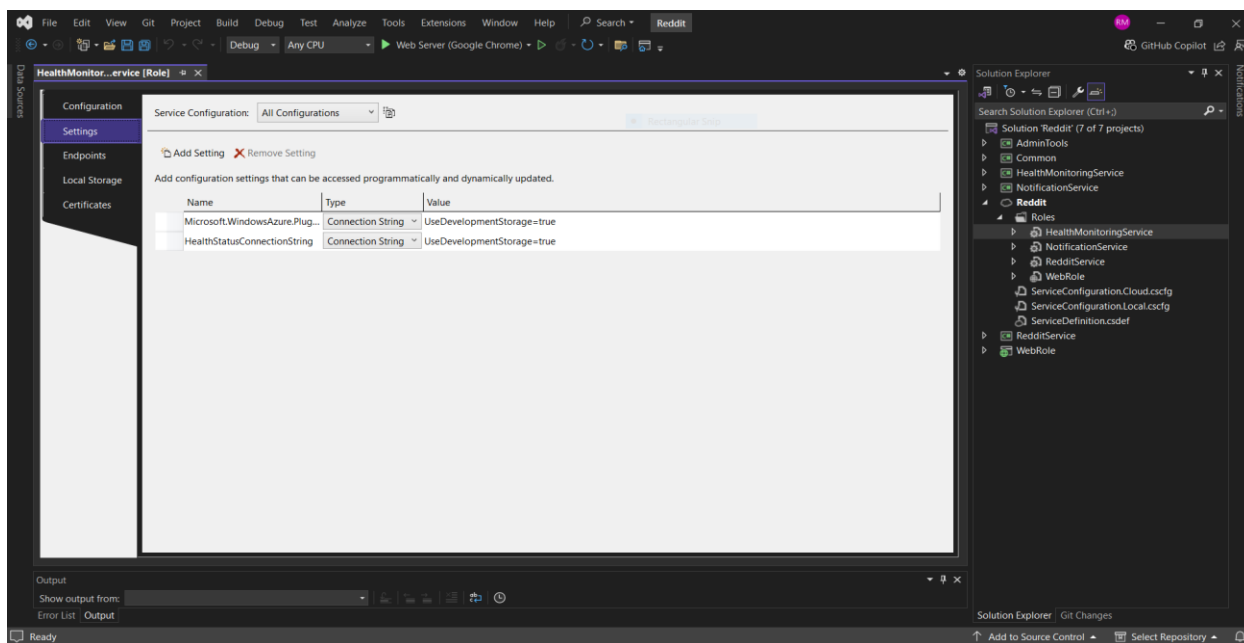
HealthMonitoringService - Implementacija provere dostupnosti sistema:

- Zaseban Worker Role servis pod nazivom - HealthMonitoringService - 2 instance
- Svakih 1-5 sekundi šalje zahtev ka RedditService i NotificationService /health-monitoring endpoint gde ukoliko sam zahtev prođe smatra se da je sve kako treba, a ukoliko ne prođe šalje se mejl na mejl adrese koje je moguće urediti kroz AdminTools.
- Svedjedno da li je zahtev prošao ili nije, neophodno je upisati poruku u posebnu tabelu HealthCheck sa trenutnim datumom i vremenom i statusom da li je uspelo ili nije, npr. 2023-11-03:12:23.33.3333_OK ili 2023-11-03:12:23.33.3333_NOT_OK



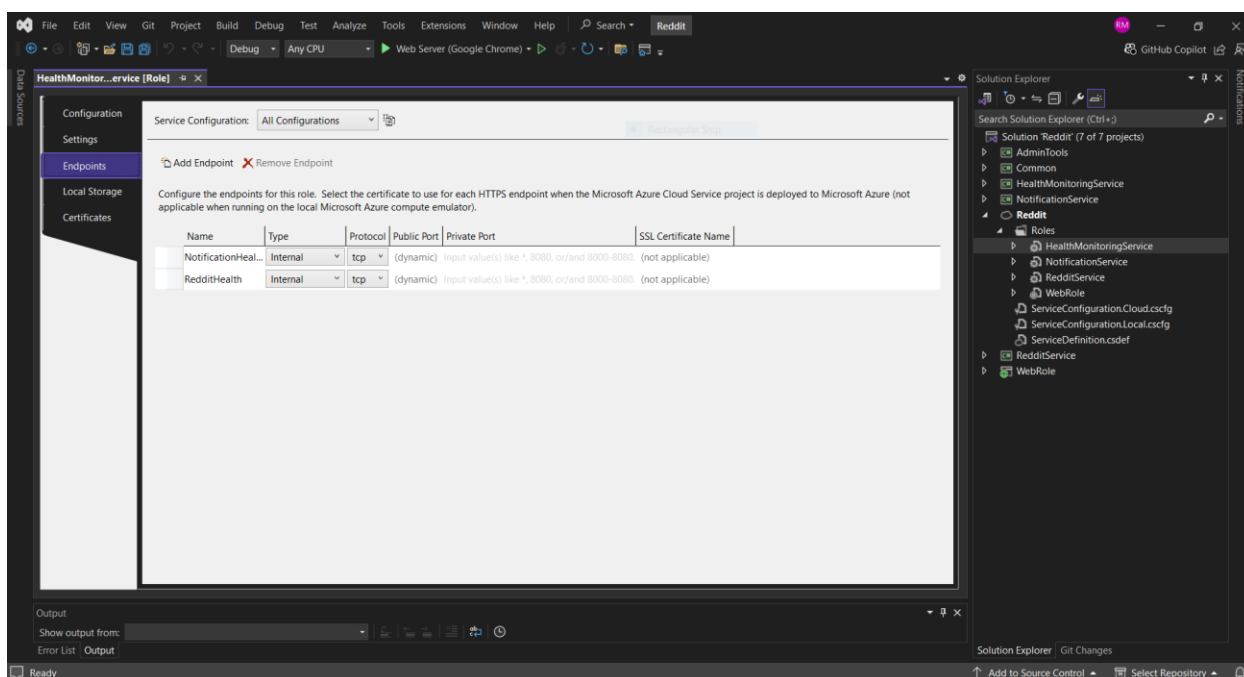
Slika 3.2.7. HealthMonitoringService Configuration

Slika 3.2.7 prikazuje konfiguraciju HealthMonitoringService-a koja uključuje postavljanje broja instanci na 2, korišćenje Standard_D1_v2 veličine virtualne mašine, i omogućenu dijagnostiku sa lokalnim razvojnim skladištem, kao što je prikazano na slici ispod.



Slika 3.2.8. HealthMonitoringService Settings

Slika 3.2.8. prikazuje podešavanja konekcionih stringova uključujući korišćenje lokalnog razvojnog skladišta za dijagnostiku i HealthStatusConnectionString, olakšavajući testiranje i razvoj bez potrebe za korišćenjem produkcionog skladišta.



Slika 3.2.9. HealthMonitoringService Endpoints

Slika 3.2.9. prikazuje postavke endpoint-ova za HealthMonitoringService koja definiše dva interna endpoint-a za proveru zdravlja: NotificationHealth i RedditHealth, oba sa dinamičkim TCP portovima, obezbeđujući proveru zdravlja sistema.

Ova konfiguracija omogućava HealthMonitoringService-u da efikasno prati dostupnost sistema, šalje notifikacije administratorima u slučaju problema i beleži status svakog zahteva u tabelu HealthCheck.

3.3. Ostale komponente sistema i tok podataka

AdminTools – konzolna aplikacija:

- Služi da se admin uloguje uz autentifikaciju na sistem, da omogući dodavanje/brisanje/ažuriranje i prikaz svih mejlova. Poruka se šalje na uneti mejl/mejlove u slučaju da neki od servisa koji su u sklopu projekta iznenada padnu - real-time slanje mejlova, koristi se softverska biblioteka otvorenog koda SignalR, što omogućava momentalno obaveštavanje korisnika o statusu sistema.

Common – Zajednički Class Library projekat, projekat koji sadrži sve zajedničke komponente neophodne za rad različitih servisa u sistemu.

Tok Podataka:

- WebRole prima korisničke zahteve, obrađuje ih i prikazuje korisnički interfejs. Podaci o korisnicima i postovima se šalju RedditService-u za obradu i skladištenje.
- RedditService obrađuje pozadinske zadatke, skladišti podatke u bazu i upravlja priložima. Pri kreiranju novog komentara, podaci o komentaru kao i ID komentara se šalju u Azure Storage Explorer bazu
- NotificationService čita podatke o komentaru iz baze kao na osnovu ID komentara, povlači podatke o pretplaćenim korisnicima i šalje im email notifikacije.
- HealthMonitoringService redovno proverava dostupnost servisa i beleži rezultate provere. U slučaju problema, šalje notifikacije administratorima putem konzolne aplikacije AdminTools koja koristi SignalR biblioteku

Ova arhitektura osigurava modularnost i skalabilnost sistema, omogućavajući efikasnu obradu korisničkih zahteva i pravovremeno slanje notifikacija, kao i stalnu proveru dostupnosti servisa.

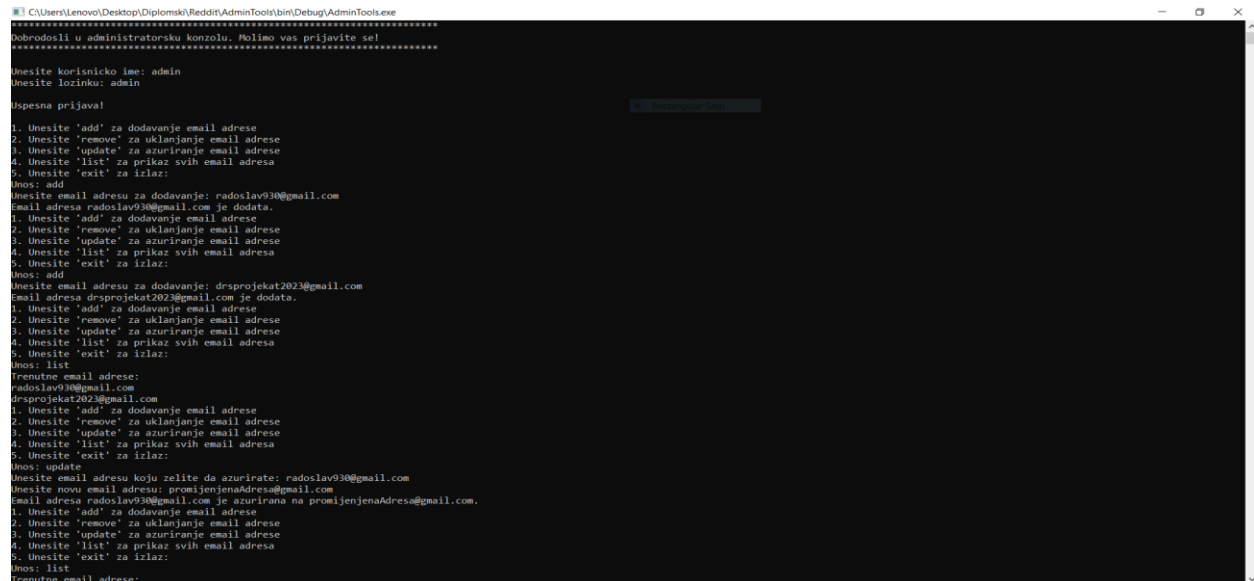
4. Prikaz implementiranog sistema

U ovom poglavlju će se detaljno prikazati funkcionalnosti i implementacija rešenja putem testnih slučajeva. Cilj je da se objasni kako sistem funkcioniše, koje su prednosti postignute implementacijom ovih rešenja, i kako korisnici mogu koristiti aplikaciju od trenutka njenog pokretanja. Prikaz će biti organizovan tako da prati redosled koraka koje korisnik preduzima prilikom korišćenja aplikacije, čime će se pružiti sveobuhvatan uvid u korisničko iskustvo i tehničke aspekte sistema.

4.1. Prikaz AdminTools sa bazom podataka i SignalR

Ovaj segment prikazuje kako AdminTools omogućava administratorima da efikasno upravljaju obaveštavanjem o statusu servisa i reagovanjem na potencijalne probleme u radu sistema.

Kada korisnik pokrene aplikaciju, prvo što treba da uradi je da pokrene novu instancu Console Application AdminTools gde će se otvoriti konzola za prikaz određenih opcija.



```

C:\Users\lenovo\Desktop\Diplomski\Reddit\AdminTools\bin\Debug\AdminTools.exe
*****
Dobrodošli u administratorsku konzolu. Molimo vas prijavite se!
*****

Unesite korisničko ime: admin
Unesite lozinku: admin

Uspesna prijava!

1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: add
Unesite email adresu za dodavanje: radoslav93@gmail.com
Email adresa radoslav93@gmail.com je dodata.
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: add
Unesite email adresu za dodavanje: drsprojekat2023@gmail.com
Email adresa drsprojekat2023@gmail.com je dodata.
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: list
Trenutne email adrese:
radoslav93@gmail.com
drsprojekat2023@gmail.com
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: update
Unesite email adresu koju zelite da azurirate: radoslav93@gmail.com
Unesite novu email adresu: promijenjenaAdresa@gmail.com
Email adresa radoslav93@gmail.com je azurirana na promijenjenaAdresa@gmail.com.
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: list
Trenutne email adrese:

```

Slika 4.1.1. AdminTools Administratorska konzola - add/list/update opcije

Korisnik, da bi se prijavio kao administrator sistema, treba da unese korisničko ime admin i lozinku admin. Nakon prijave, korisnik može koristiti sledeće opcije: add za dodavanje email adrese, remove za brisanje email adrese, update za ažuriranje email adrese, list za prikaz liste email adresa i exit za izlaz iz administratorske konzole. Sve validacije za unos su takođe implementirane, tako da korisnik mora tačno uneti sve podatke. Slika 4.1.1 prikazuje testne slučajeve administratorske konzole za add, list i update opcije.

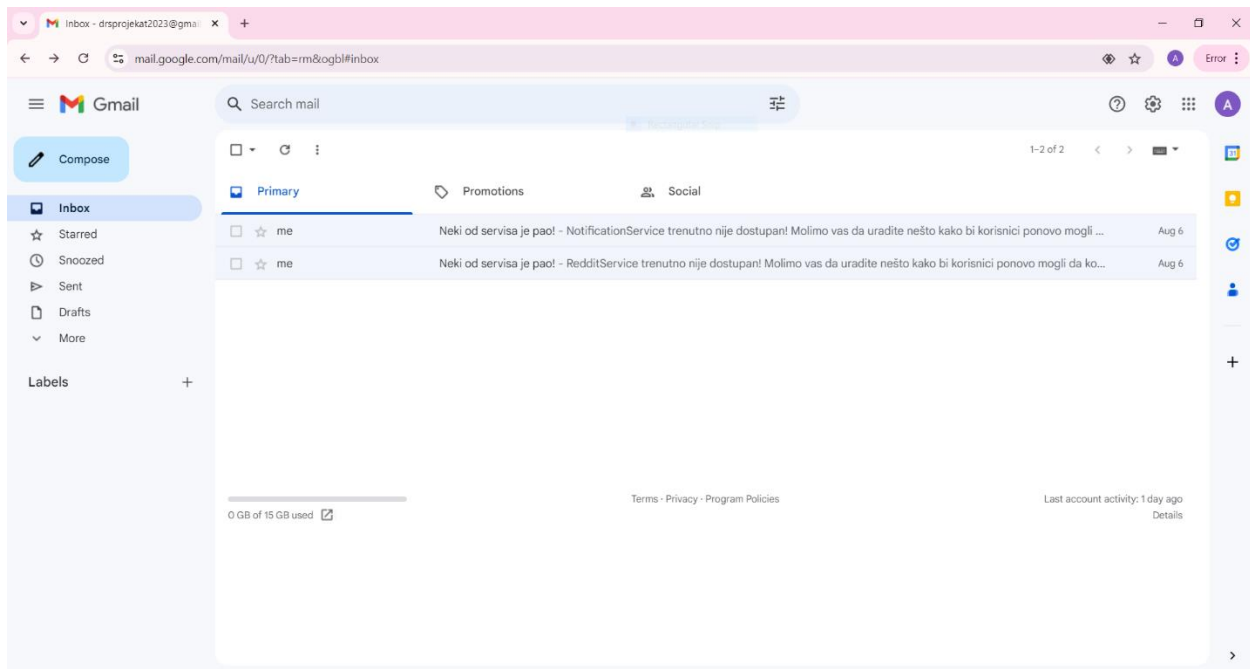
```

C:\Users\Lenovo\Desktop\Diplomski\Reddit\AdminTools\bin\Debug\AdminTools.exe
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: update
Unesite email adresu koju zelite da azurirate: radoslav930@gmail.com
Unesite novu email adresu: promijenjenaAdresa@gmail.com
Email adresa radoslav930@gmail.com je azurirana na promijenjenaAdresa@gmail.com.
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: list
Trenutne email adrese:
promijenjenaAdresa@gmail.com
drsprojekt2023@gmail.com
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: remove
Unesite email adresu za uklanjanje: promijenjenaAdresa@gmail.com
Email adresa promijenjenaAdresa@gmail.com je uklonjena.
1. Unesite 'add' za dodavanje email adrese
2. Unesite 'remove' za uklanjanje email adrese
3. Unesite 'update' za azuriranje email adrese
4. Unesite 'list' za prikaz svih email adresa
5. Unesite 'exit' za izlaz:
Unos: exit
Izlazak iz menadzera email adresa.
Server je dostupan na: http://localhost:8080
Obavestava se admin da RedditService nije dostupan!
Email uspesno poslat na drsprojekt2023@gmail.com!
Obavestava se admin da NotificationService nije dostupan!
Email uspesno poslat na drsprojekt2023@gmail.com!

```

Slika 4.1.2. AdminTools Administratorska konzola remove/exit/pad servisa

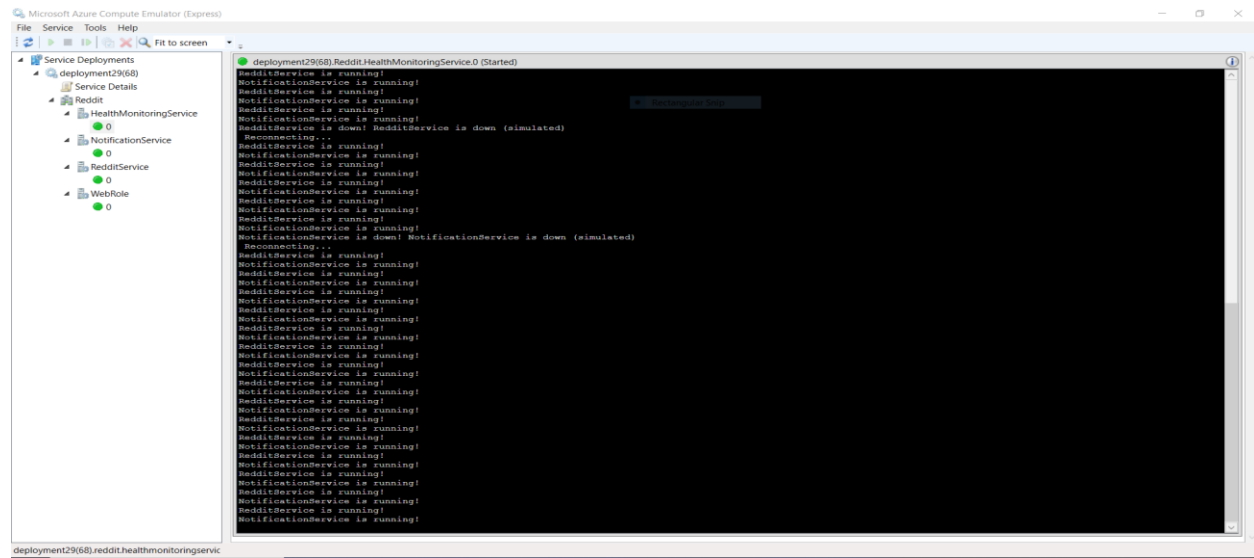
Kada korisnik izađe iz administratorske konzole, email adrese će biti aktivne za obaveštavanje korisnika u slučaju pada nekog od servisa. Server je dostupan na prikazanom localhost-u – zbog korištenja SignalR biblioteke. Slika 4.1.2 prikazuje testne slučajeve korišćenja administratorske konzole za remove, exit opcije i simulaciju pada servisa gde se ispisuje da je korisnik uspešno obavešten na unešeni email.



Slika 4.1.3. AdminTools email obavijestjenje

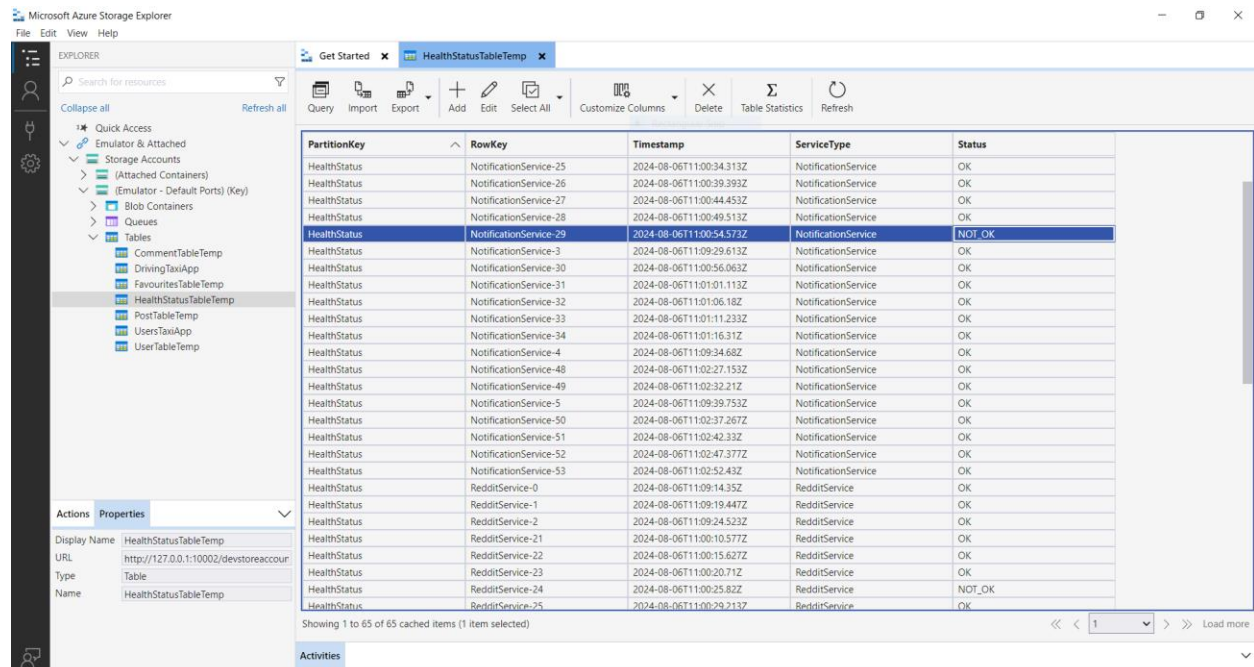
Slika 4.1.3. prikazuje da je poruka koja je poslata korištenjem SignalR biblioteke stigla na adresu u oba slučaja pada servisa i korisnika obavijestila o kvaru.

Ovo služi kao što je navedeno u specifikaciji: admin se loguje na sistem i unosi email adrese na koje će biti poslate obaveštenja u slučaju pada servisa (RedditService ili NotificationService). Kada se kvar otkloni, sistem nastavlja sa nesmetanim radom.



Slika 4.1.4. Microsoft Azure Compute Emulator prikaz ispisa pada servisa

U Microsoft Azure Compute emulatoru se prvo ispisuje pad servisa zbog lakšeg ispravka grešaka (Slika 4.1.4.).



Slika 4.1.5. Prikaz ispisa pada servisa u bazi podataka

Na slici 4.1.5 prikazano je kako HealthMonitoringService svakih 1-5 sekundi šalje zahtev ka RedditService i NotificationService. Ako zahtevi prođu, u bazu HealthStatusTableTemp upisuju se trenutni Timestamp i

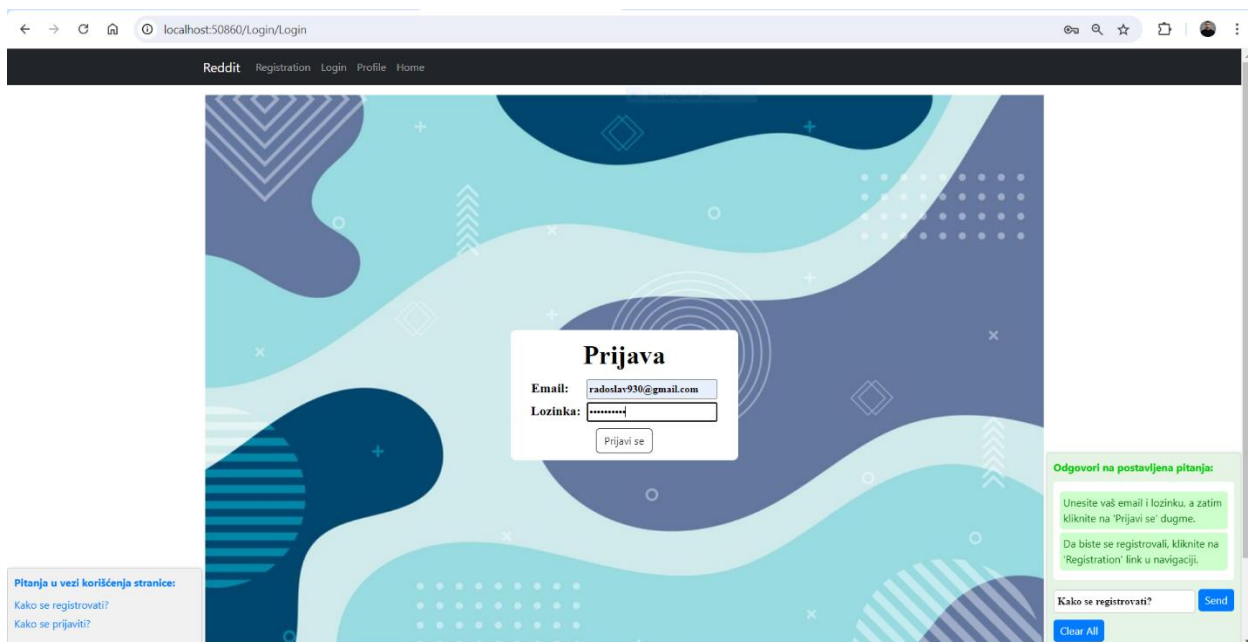
status OK. Ako neki od servisa padne i zahtev ne prođe, upisuje se trenutni Timestamp i status NOT_OK. Ova informacija se upisuje u realnom vremenu.

4.2. Prikaz Login stranice

Ako servisi funkcionišu bez grešaka, prva stranica koju korisnik vidi je Login stranica. Ukoliko korisnik nije registrovan na sistemu, neće moći da se prijavi, pa prvo prelazi na Register stranicu, koja je opisana u poglavlju 4.3. Nakon što završi registraciju, korisnik se vraća na Login stranicu gde unosi svoju jedinstvenu email adresu i lozinku kako bi pristupio sistemu, odnosno Home stranici, koja je opisana u poglavlju 4.5.

Na Login stranici se takođe nalazi chatbot koji olakšava korisnicima snalaženje na stranici. Sa leve strane korisnik može da klikne na često postavljana pitanja, dok sa desne strane dobija odgovore na ta pitanja. Korisnik takođe može direktno postaviti pitanje na desnoj strani i dobiti odgovor.

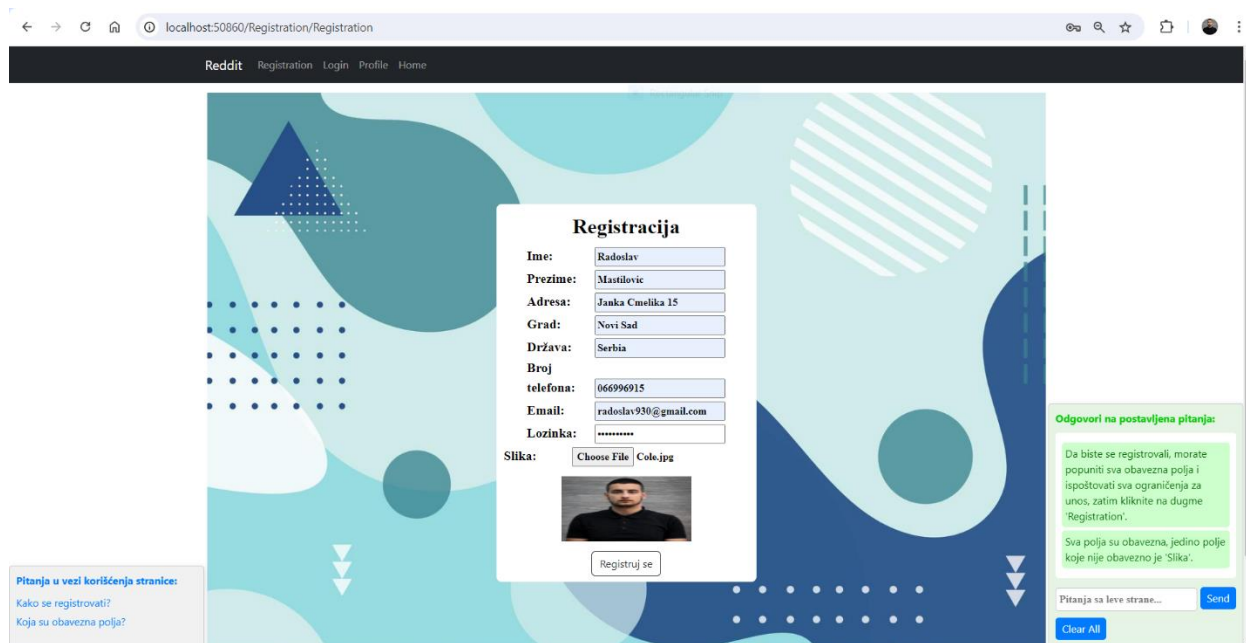
Slika 4.2.1 prikazuje Login stranicu sa chatbot-om koji pomaže korisnicima da lakše koriste stranicu.



Slika 4.2.1. Login stranica sa chatbot-om

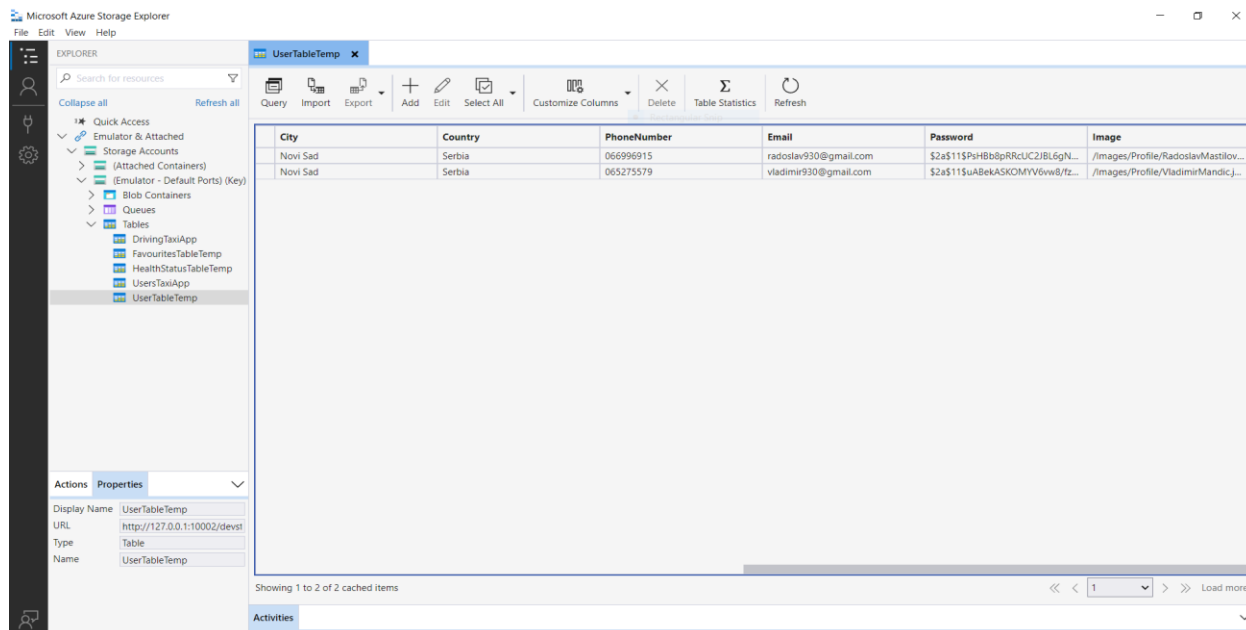
4.3. Prikaz Register stranice sa bazom podataka

Ukoliko korisnik nema nalog i prešao je na Register stranicu, na toj stranici vrši svoju registraciju na sistem. Registracija se vrši popunjavanjem potrebnih polja koja su prikazana na slici 4.3.1. Svako polje ima posebnu validaciju, i korisnik neće moći da se registruje dok sva polja ne unese pravilno. Jedino polje koje nije obavezno je polje "Slika". Kao i na Login stranici, korisniku je dostupan chatbot za pomoć.



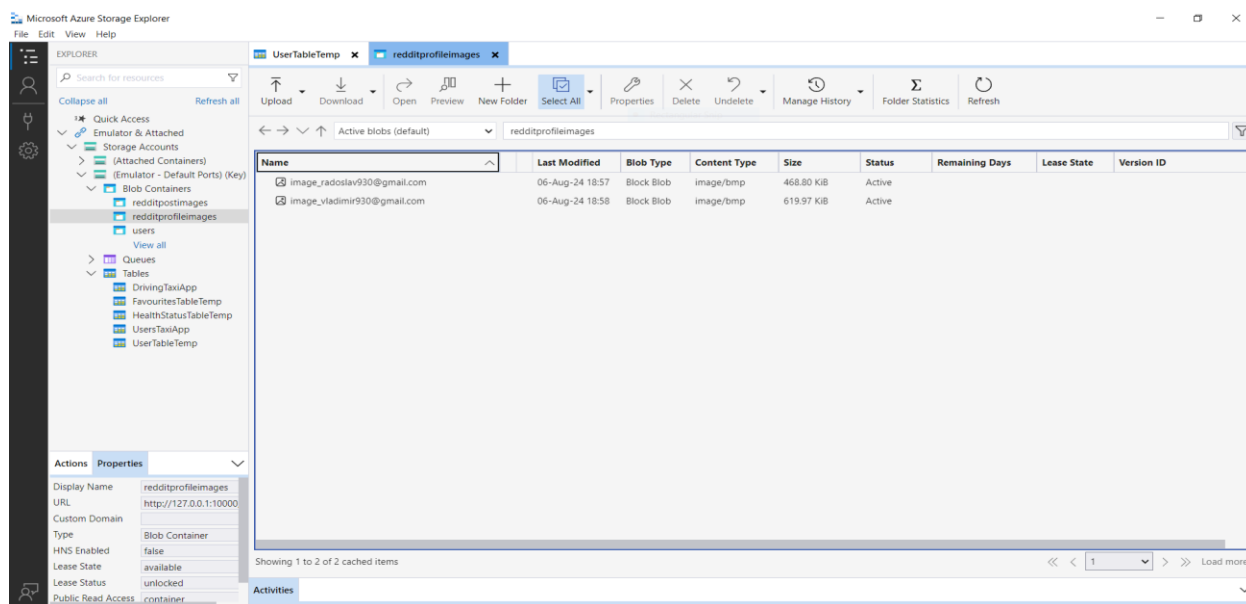
Slika 4.3.1. Register stranica sa chatbot-om

Kada korisnik popuni sva potrebna polja i uspešno se registruje na sistem, u Microsoft Azure Storage Explorer bazi, u Tables "UserTableTemp", upisuju se svi uneseni podaci (slika 4.3.2.). Ono što je važno napomenuti jeste da se prilikom unosa lozinka pre upisa u bazu hešuje, što onemogućava njeno čitanje bilo gde u sistemu, čime se korisniku obezbeđuje najveća sigurnost i privatnost. Ova hešovana lozinka će kasnije biti korišćena za validaciju na Profile stranici, što će biti objašnjeno u sledećem podnaslovu.



Slika 4.3.2. Prikaz table "UsersTableTemp" u bazi podataka

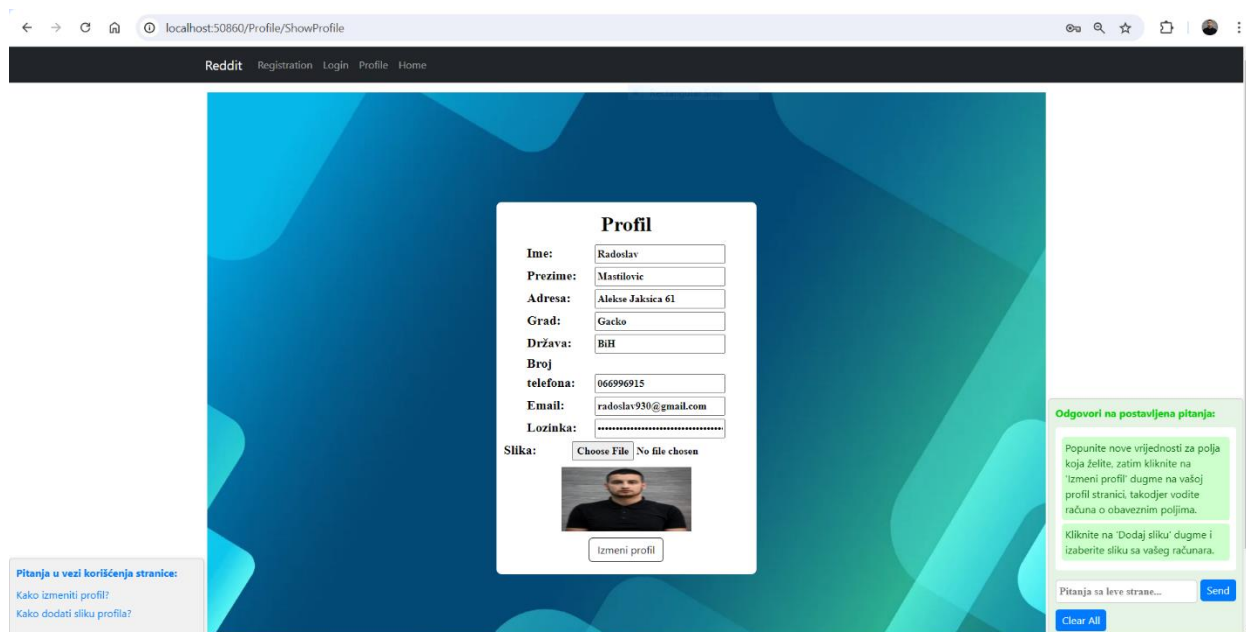
Slike koje su postavljene prilikom registracije korisnika takođe se posebno čuvaju u Blob Containers, u "redditprofileimages" kontejneru, i odatle se mogu preuzeti na računar ako je to potrebno (slika 4.3.3).



Slika 4.3.3. Prikaz kontejnera "redditprofileimages" u bazi podataka

4.4. Prikaz Profile stranice sa bazom podataka

Korisnici koji nisu registrovani i prijavljeni na sistem neće imati pristup Profile stranici. Tek nakon što se registruju i prijave na sistem, korisnici će biti preusmereni na Home-Reddit stranicu, odakle će moći da pristupe Profile stranici i menjaju svoj profil (Slika 4.4.1.). Pravila za unos podataka na Profile stranici ista su kao i na Register stranici. Kao i na ostalim stranicama, korisnicima je dostupan chatbot za pomoć. Važno je napomenuti da se lozinka iz baze čita hešovana, što osigurava njenu sigurnost.



Slika 4.4.1. Profile stranica

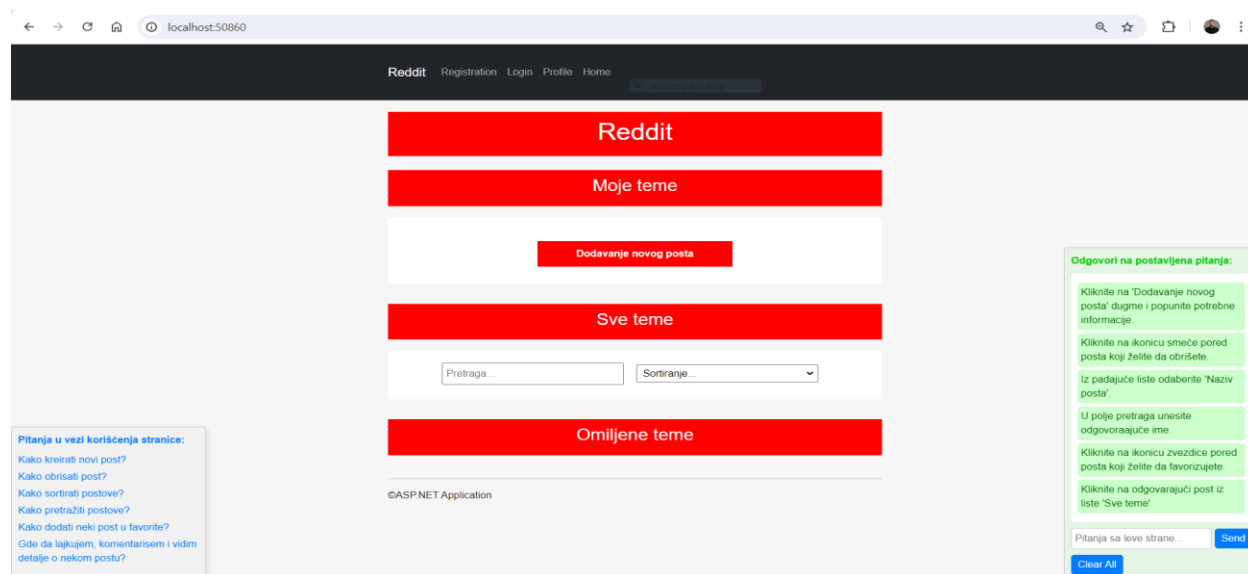
Baza podataka nakon što je korisnik izmenio polja “Adresa”, “Grad” i “Broj telefona” prikazana je na slici 4.4.2.

PartitionKey	RowKey	Timestamp	FirstName	LastName	Address	City
User	radoslav930@gmail.com	2024-08-06T17:19:33.72Z	Radoslav	Mastilovic	Alekse Jaksica 61	Gacki
User	vladimir930@gmail.com	2024-08-06T16:58:10.967Z	Vladimir	Mandic	Bulevar Despota Stefana 7	Novi

Slika 4.4.2. Prikaz tabele “UsersTableTemp” posle izmene informacija korisnika u bazi podataka

4.5. Prikaz Home-Reddit stranice sa bazom podataka

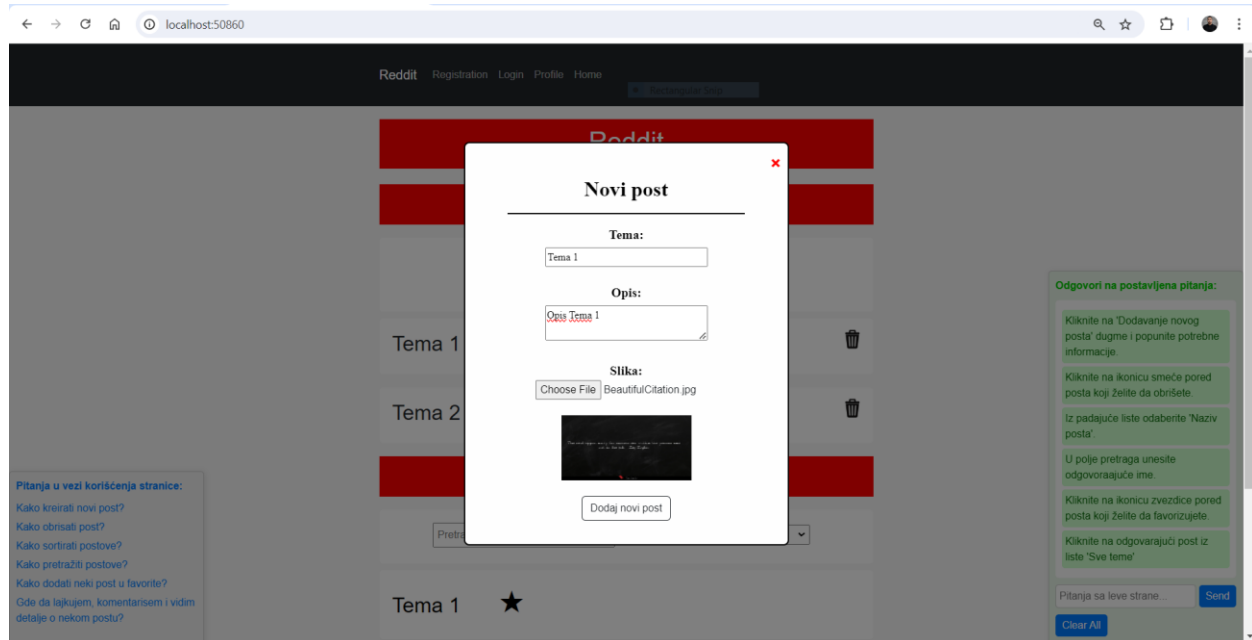
Kada se korisnici registruju i prijave na sistem, prelaze na Home-Reddit stranicu. Kao i na ostalim stranicama, tu je dostupan chatbot koji korisnicima olakšava korišćenje stranice, nudeći im odgovore na često postavljana pitanja ili omogućavajući postavljanje novih pitanja.



Slika 4.5.1. Reddit-Home stranica

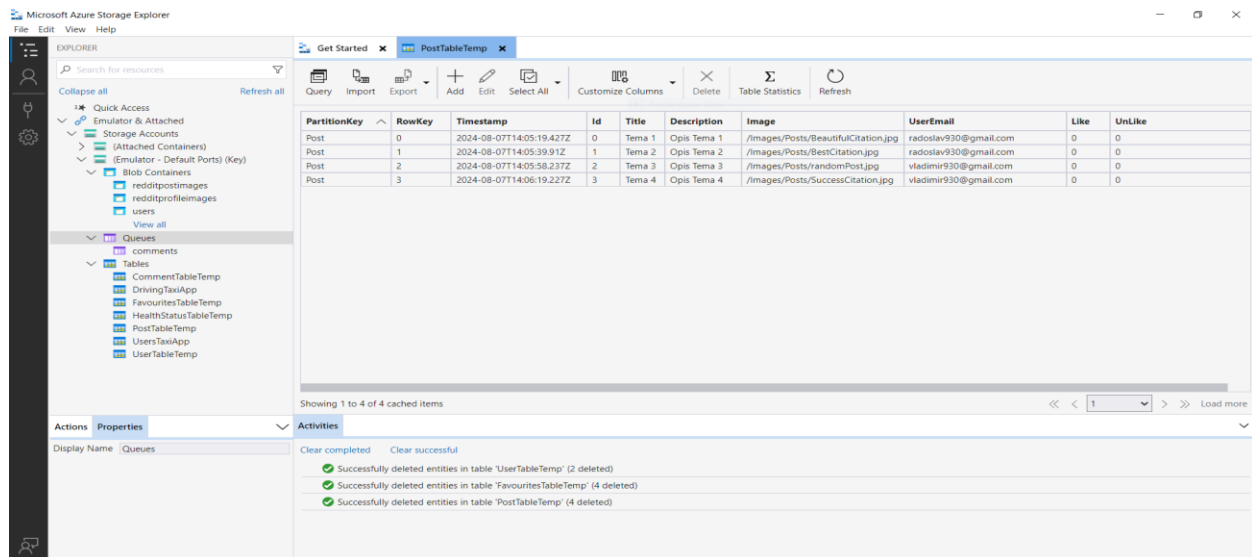
Slika 4.5.1 prikazuje Home-Reddit stranicu. Na vrhu stranice nalazi se navigacioni bar (navbar) koji omogućava prelazak na stranice Login, Registration, Home i Profile. Ispod navigacionog bara prikazana je

lista "Moje teme" koja prikazuje sve teme koje je pojedinačni korisnik dodao. Lista "Sve teme" prikazuje sve teme od svih korisnika na Reddit sistemu, sa mogućnošću sortiranja i pretrage postova prema nazivu. Lista "Omiljene teme" sadrži favorizovane teme svih korisnika i sortirana je od najviše do najmanje puta favorizovanih tema.



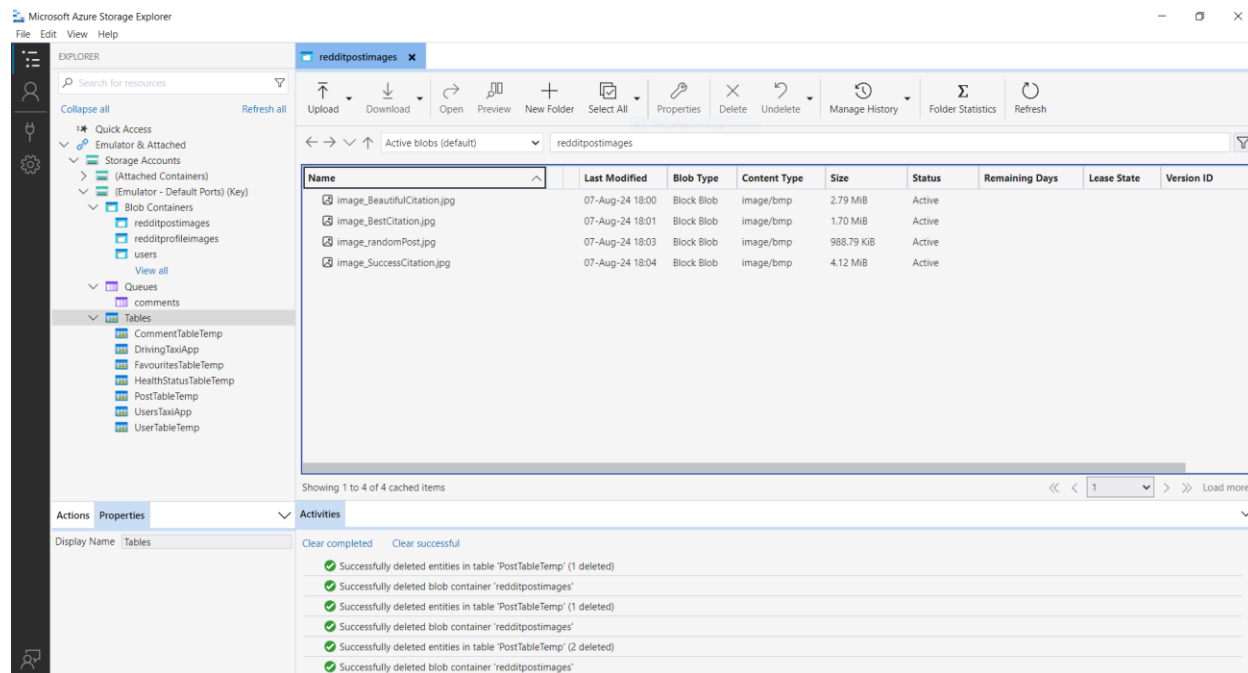
Slika 4.5.2. "Dodavanje novog posta"

Korisnici imaju mogućnost dodavanja tema. Slika 4.5.2 je prikaz kako korisnici dodaju nove teme. U listi "Moje teme" pored tema se pojavljuje ikonica za brisanje (smeće), omogućavajući korisnicima da obrišu svoje teme. U listi "Sve teme" pored tema se pojavljuje ikonica za favorizovanje (zvezdica), omogućavajući korisnicima da favorizuju teme.



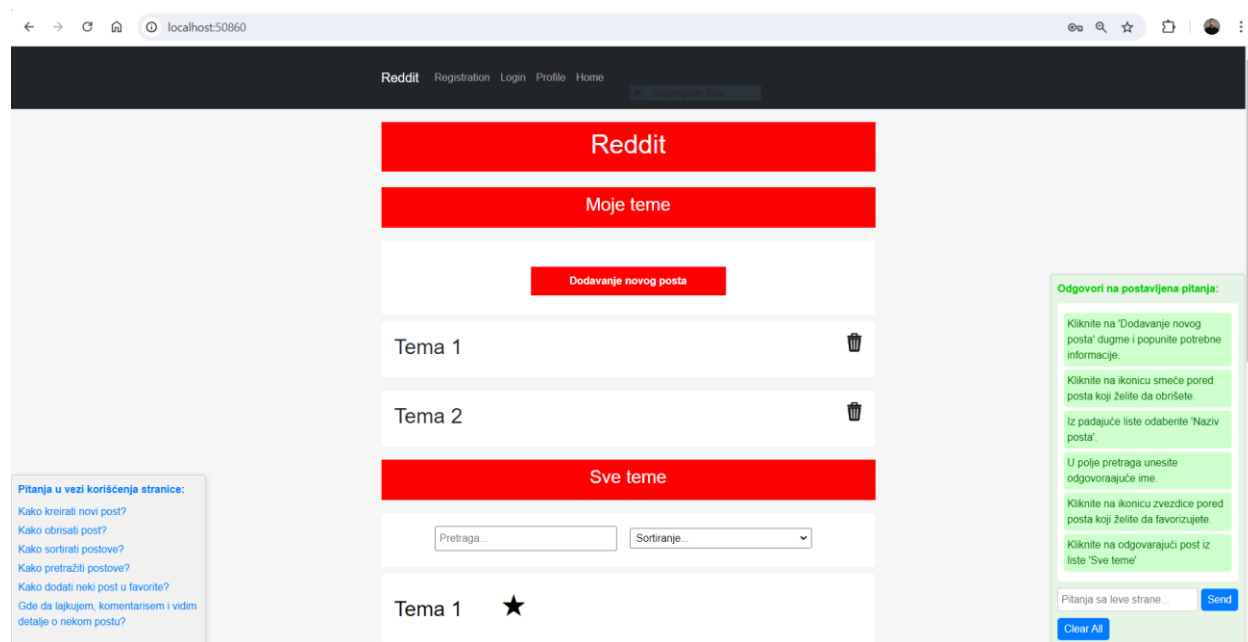
Slika 4.5.3. Tabela "PostTableTemp" u bazi podataka

Tabela “PostTableTemp,” prikazana na slici 4.5.3, sadrži sve postove koje su korisnici sistema kreirali, uz sva neophodna polja za post.



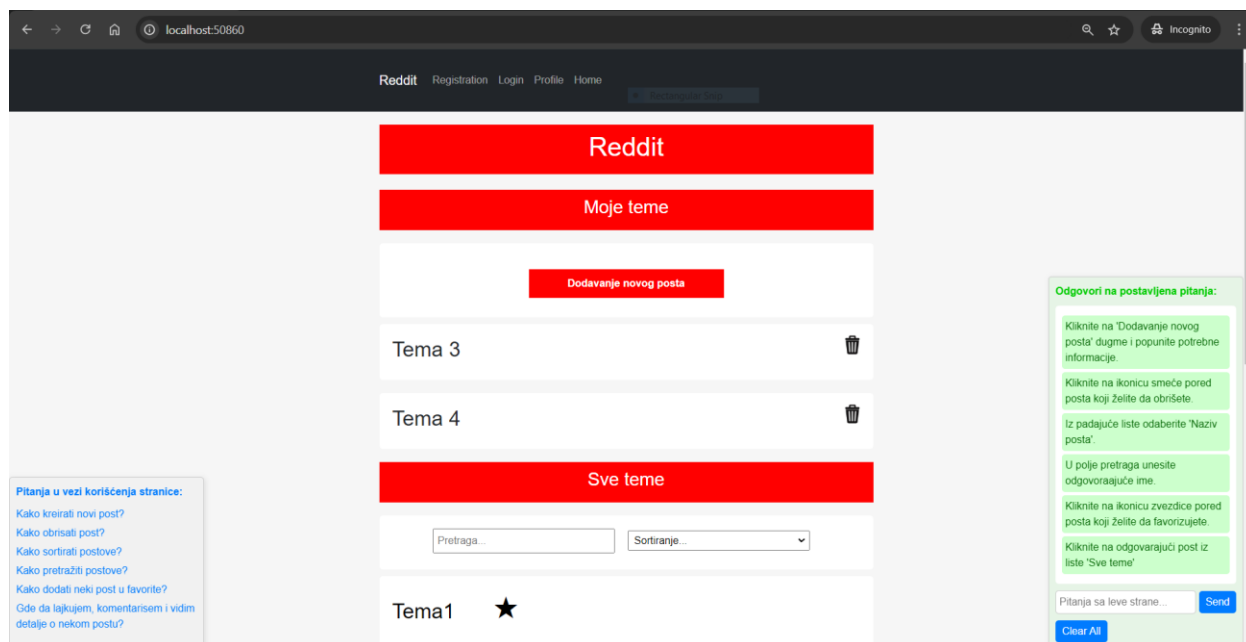
Slika 4.5.4. Blob Containers “redditpostimages” u bazi podataka

Slike dodatih postova prilikom njihovog kreiranja upisuju se u poseban Blob Container pod nazivom “redditpostimages,” što je prikazano na slici 4.5.4.



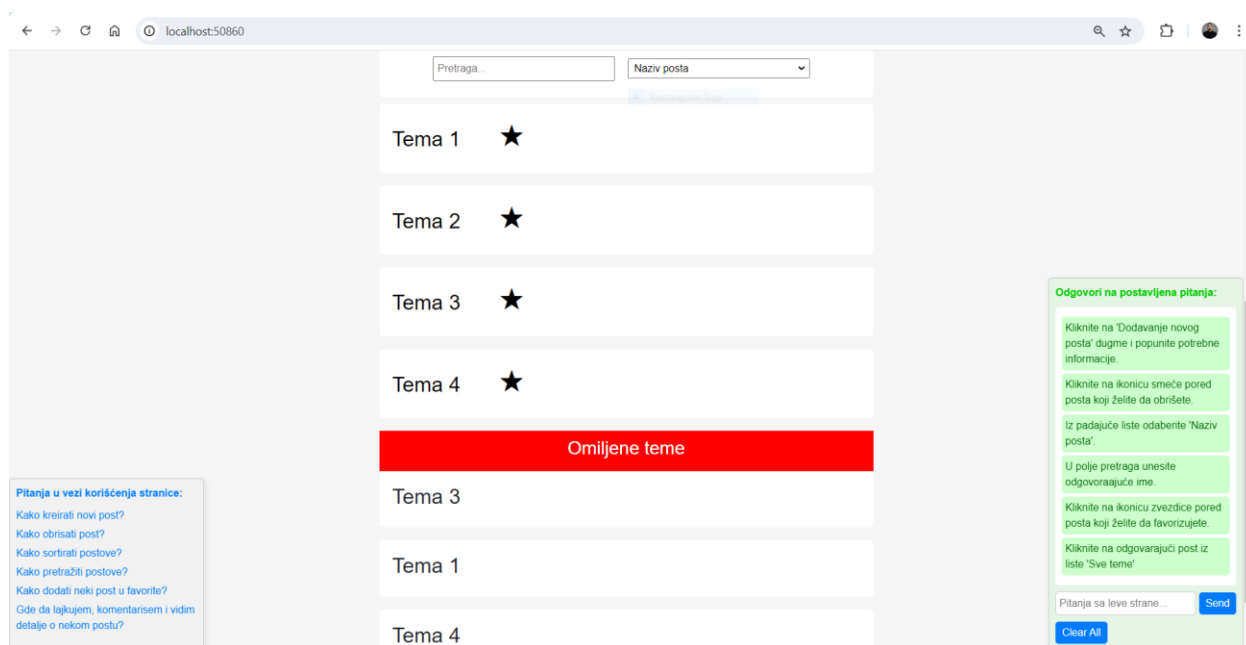
Slika 4.5.5. Liste “Moje teme” “radoslav930@gmail.com”

Izgled za korisnika sa email adresom “radoslav930@gmail.com,” koji je dodao teme “Tema 1” i “Tema 2,” odnosno njegova lista “Moje teme,” prikazan je na slici 4.5.5.



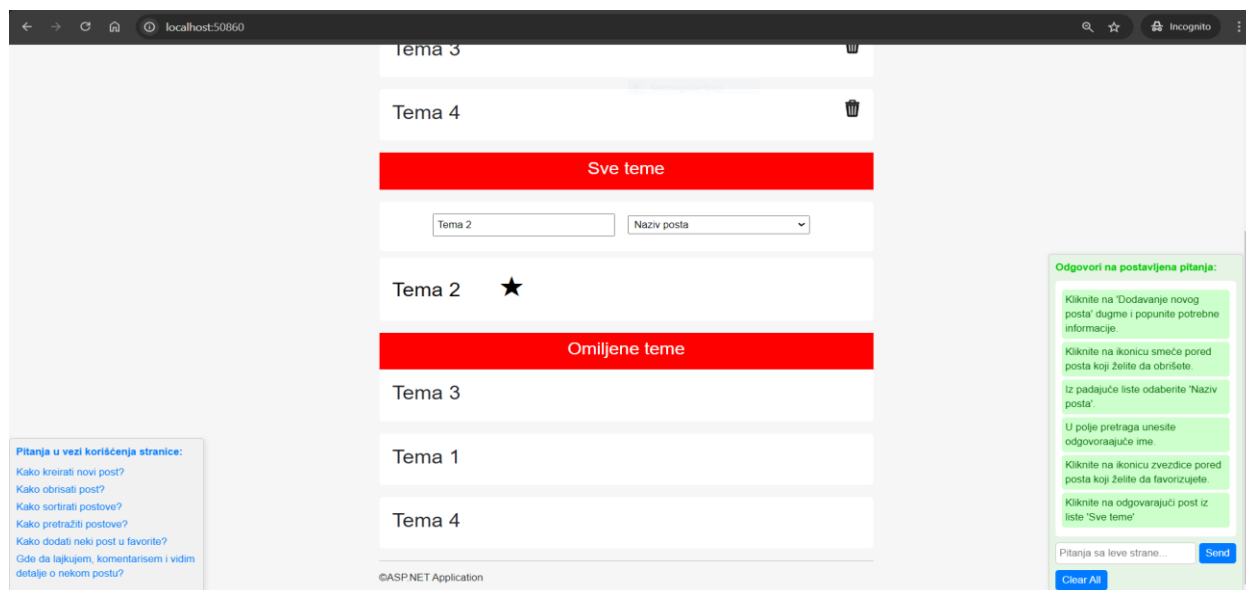
Slika 4.5.6. Lista "Moje teme" za korisnika "vladimir930@gmail.com"

Slika 4.5.6 prikazuje izgled za korisnika sa email adresom "vladimir930@gmail.com" koji je dodao teme "Tema 3" i "Tema 4", to jest njegova lista "Moje teme".



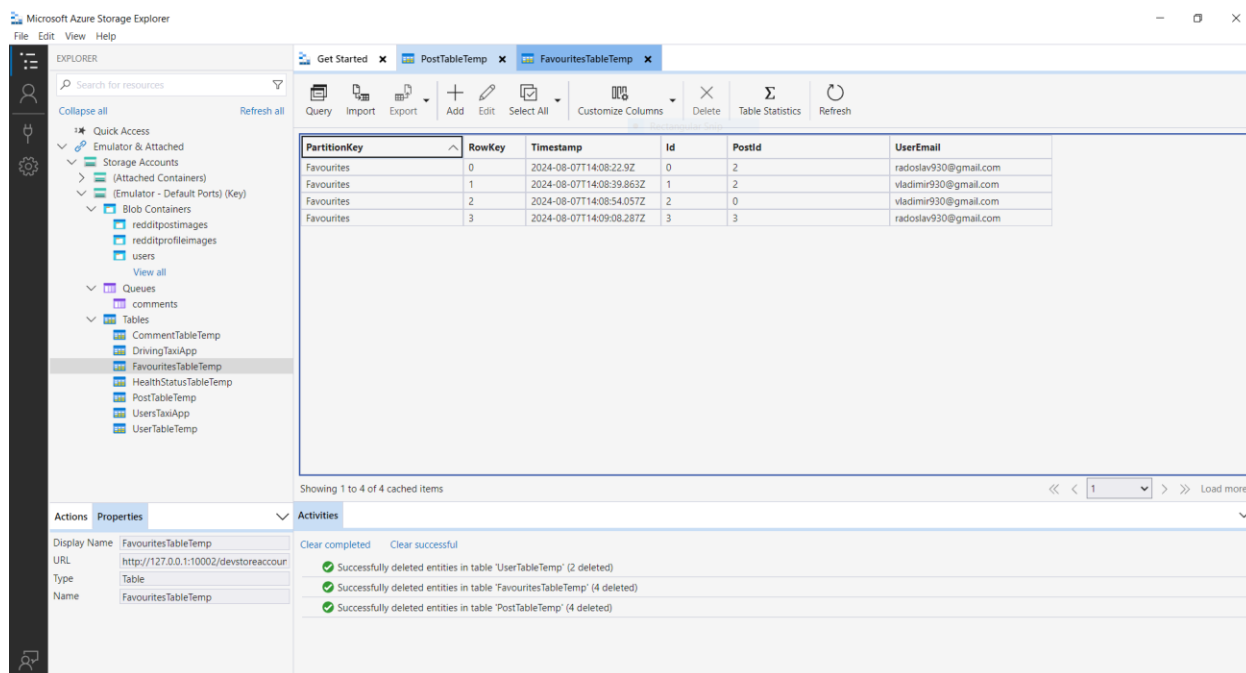
Slika 4.5.7. Liste "Sve teme" i "Omiljene teme" za korisnika "radoslav930@gmail.com"

Slika 4.5.7. prikazuje listu "Sve teme" i "Omiljene teme" za korisnika "radoslav930@gmail.com". Lista "Sve teme" sadrži sve dodate teme od korisnika, sortirane po nazivu posta, dok lista "Omiljene teme" prikazuje teme favorizovane od strane svih korisnika koja je sortirana automatski na osnovu broja favorizovanja korisnika. Podaci o favorizovanim postovima se upisuju u tabelu "FavouritesTableTemp" koja je prikazana na slici 4.5.9.



Slika 4.5.8. Liste "Sve teme" i "Omiljene teme" za korisnika "vladimir930@gmail.com"

Lista "Sve teme" i "Omiljene teme" za korisnika "vladimir930@gmail.com" prikazana je na slici 4.5.8. Lista "Sve teme" sadrži sve dodate teme od korisnika, pri čemu je prikazana pretraga teme pod nazivom "Tema 2." Lista "Omiljene teme" je ista za sve korisnike, kao što je ranije opisano, te automatski prikazuje iste postove.



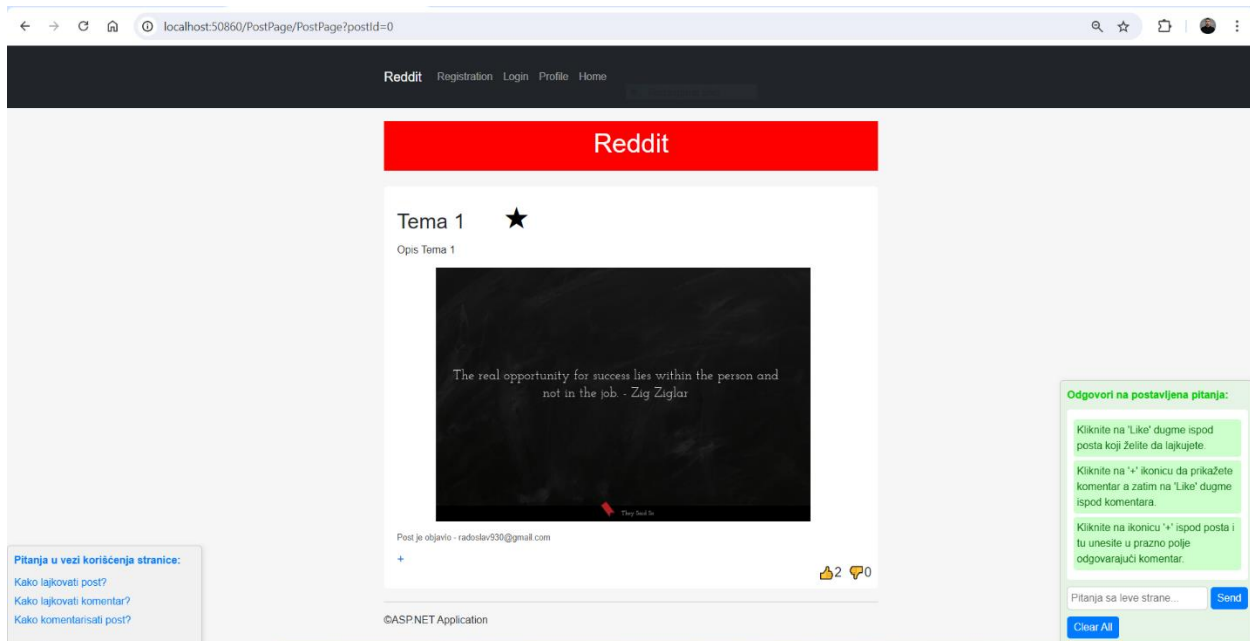
Slika 4.5.9. Tabela "FavouritesTableTemp" u bazi podataka

Tabela "FavouritesTableTemp" prikazana je na slici 4.5.9 i sadrži sve informacije o favorizovanim postovima. Na osnovu tih informacija, postovi se iščitavaju, zatim sortiraju, i na kraju prikazuju u listi "Omiljene teme" za svakog korisnika, poredani od najviše do najmanje puta favorizovanih tema.

4.6. Prikaz ProfilePage stranice sa bazom podataka

Ovaj segment prikazuje funkcionalnosti PostPage stranice, uključujući mogućnosti lajkovanja/dislaikovanja postova i komentara, dodavanja komentara, i automatskog slanja obaveštenja vlasnicima postova o novim komentarima.

Na listi "Sve teme" korisnik sistema može da uđe na bilo koju temu. Kada otvori temu, tu se takođe nalazi chatbot koji olakšava korišćenje stranice, kao i na ostalim stranicama. Slika 4.6.1 prikazuje post sa nazivom "Tema 1". Na ovoj stranici korisnik ima mogućnost da lajkuje ili dislaikuje određeni post.



Slika 4.6.1. PostPage stranica

Kada korisnik lajkuje ili dislaikuje neki od postova, te informacije se upisuju u bazu podataka u polja Like/Dislike, kao što je prikazano na slici 4.6.2.

ID	Description	Image	UserEmail	Like	Unlike
1	Opis Tema 1	/images/Posts/BeautifulCitation.jpg	radoslav930@gmail.com	2	0
2	Opis Tema 2	/images/Posts/BestCitation.jpg	radoslav930@gmail.com	0	2
3	Opis Tema 3	/images/Posts/randomPost.jpg	vladimir930@gmail.com	2	0
4	Opis Tema 4	/images/Posts/SuccessCitation.jpg	vladimir930@gmail.com	0	1

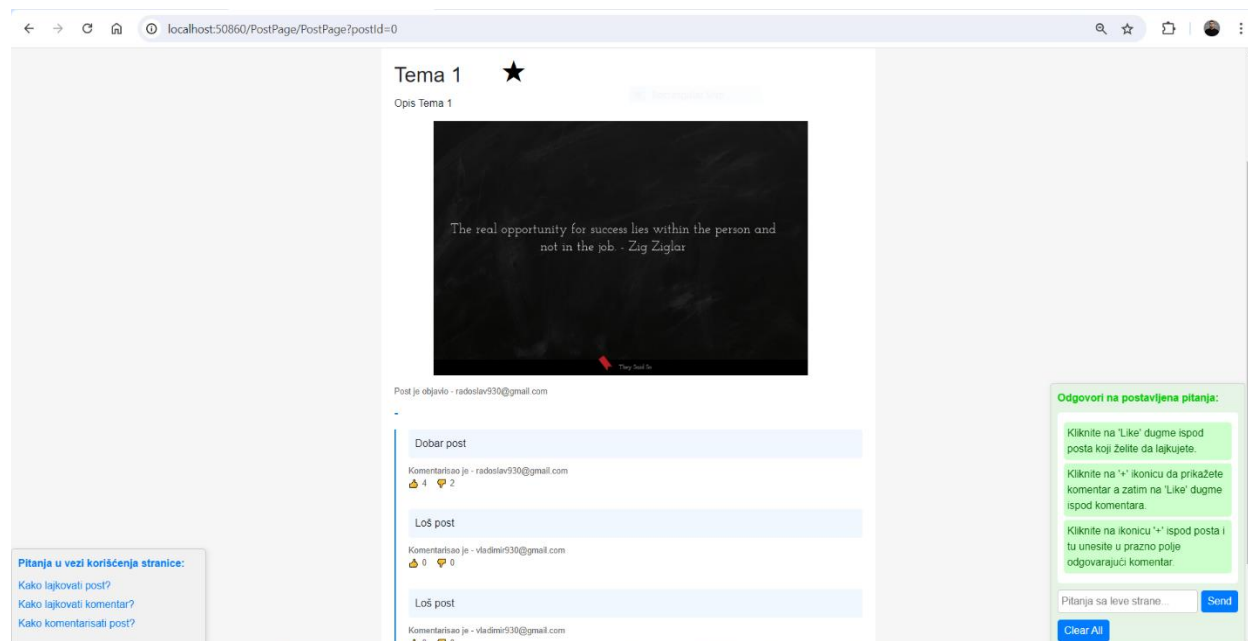
Showing 1 to 4 of 4 cached items

Activities:

- Clear completed
- Clear successful
- Deleted queue 'comments' successfully
- Successfully deleted table 'UserTableTemp'
- Successfully deleted table 'HealthStatusTableTemp'
- Successfully deleted table 'FavouritesTableTemp'

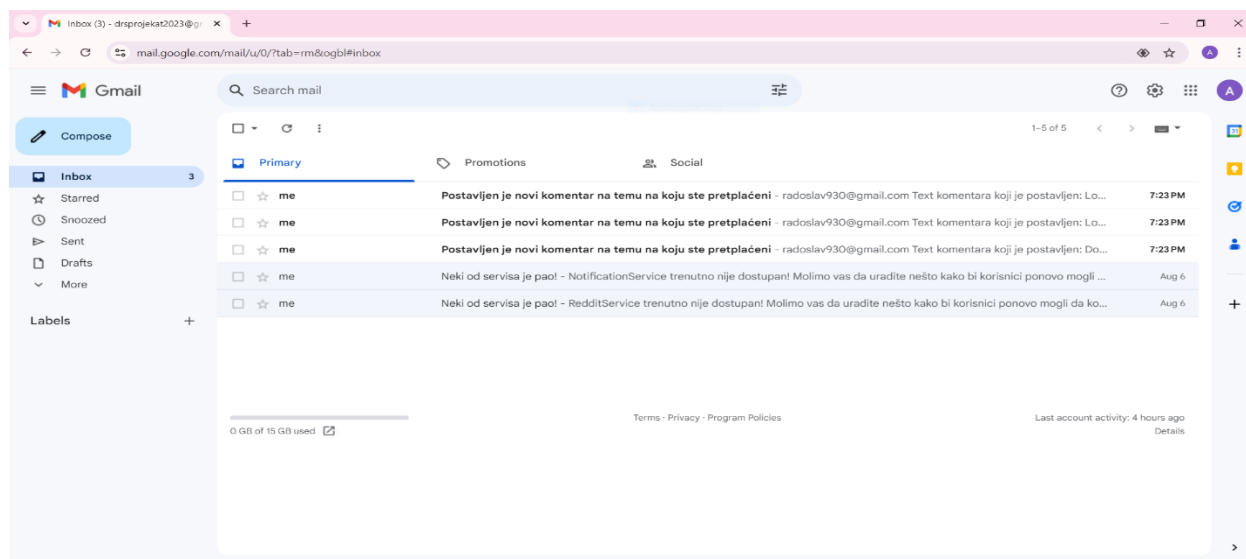
Slika 4.6.2. Tabela "PostTableTemp" polja Like/Dislike u bazi podataka

Pored toga, korisnik ima mogućnost da dodaje komentare na post na koji je ušao. Slika 4.6.3 prikazuje kako su postavljene komentari ispod posta od strane korisnika, kao i to da su korisnici izvršili like/unlike na komentare.



Slika 4.6.3. PostPage stranica komentari

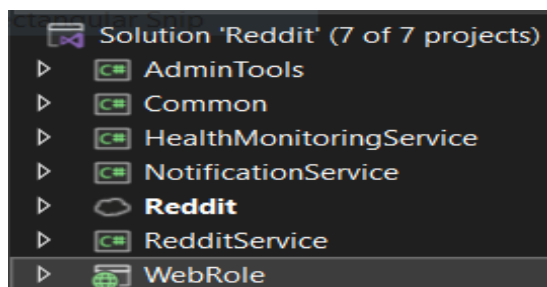
Svi komentari koje korisnici ostave ispod postova se upisuju u bazu podataka zajedno sa lajkovima/dislaikovima na tim komentarima. Slika 4.6.4 prikazuje tabelu "CommentTableTemp" gde se vide komentari koji su postavljени na određene postove, kao i broj lajkova/dislaikova na tim komentarima.



Slika 4.6.6. Gmail vlasnika posta

5. Implementacija

U ovom poglavlju će biti detaljno opisana implementacija sistema za upravljanje forumskim temama i komentarima, koji je razvijen koristeći CLOUD tehnologije na Microsoft Azure platformi. Projekat je kreiran u Microsoft Visual Studio 2022 kao Azure Cloud Service (Extended Support) i uključuje 6. ključnih komponenti, slika 5.1. prikazuje glavnu arhitekturu sistema: WebRole, RedditService, NotificationService, HealthMonitoringService, Common, AdminTools i Reddit (glavni projekat preko kojeg su kreirane WebRole i WorkerRole). Svaka od ovih komponenti ima specifičnu ulogu u funkcionalnosti sistema i njihova implementacija će biti detaljno prikazana.



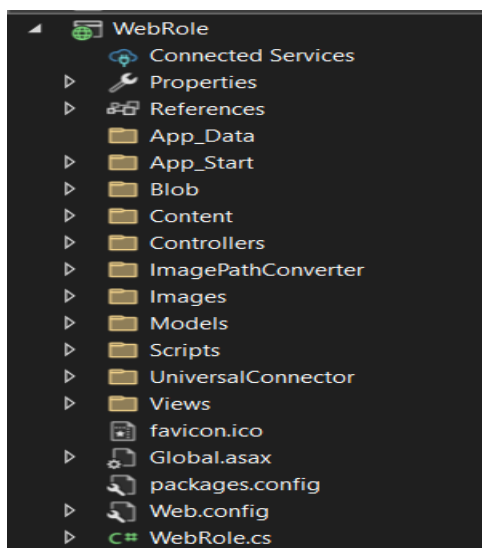
Slika 5.1. Arhitektura sistema

Cilj ovog poglavlja je da pruži sveobuhvatan pregled implementacije sistema, ističući ključne aspekte koji doprinose njegovoj skalabilnosti, pouzdanosti i efikasnosti.

Izvorni kod dostupan je na: https://github.com/MastilovicRadoslav/Razvoj_CLOUD_aplikacija_UPM

5.1. WebRole komponenta

WebRole komponenta je implementirana kao MVC aplikacija koja korisnicima pruža korisnički interfejs i komunicira sa ostalim servisima radi implementacije poslovne logike.



Slika 5.1.1. WebRole struktura

Slika 5.1.1. prikazuje strukturu WebRole komponente, koja uključuje raspored svih ključnih datoteka i foldera. Ispod foldera Views se nalaze:

1. WebRole.cs: Ova klasa nasleđuje RoleEntryPoint i definiše osnovne metode za pokretanje uloge (Role) u Azure okruženju. Metoda OnStart se koristi za inicijalizaciju i vraća baznu implementaciju. Klasa je prikazana na slici 5.1.2.

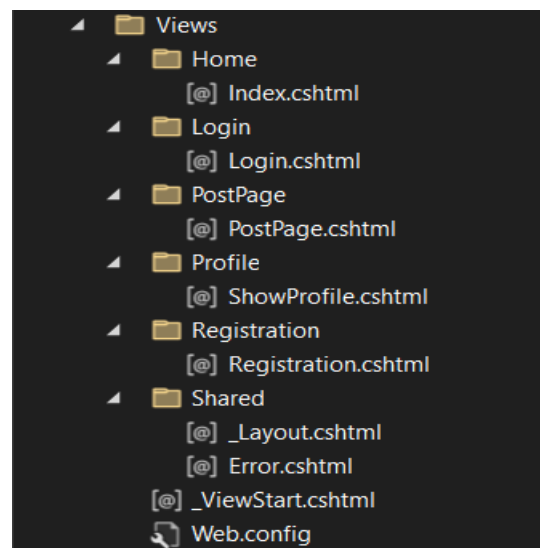
```
public class WebRole : RoleEntryPoint
{
    0 references
    public override bool OnStart()
    {
        // For information on handling configuration changes
        // see the MSDN topic at https://go.microsoft.com/fwlink/?LinkId=166357.

        return base.OnStart();
    }
}
```

Slika 5.1.2. WebRole struktura

2. Web.config: Ova XML datoteka sadrži konfiguracione postavke za ASP.NET aplikaciju, uključujući podešavanja za kompilaciju, runtime, sistemske dijagnostike, kao i bindinge za razne biblioteke i komponente koje aplikacija koristi.
3. packages.config: Ova datoteka sadrži spisak svih NuGet paketa i njihovih verzija koje su instalirane u projektu, omogućavajući upravljanje zavisnostima aplikacije.
4. Global.asax: Ova klasa inicijalizuje globalne postavke aplikacije, uključujući registraciju ruta, filtera i bundlova. Takođe, metoda InitBlobs se koristi za konfiguraciju i inicijalizaciju Azure Blob Storage kontejnera za čuvanje slika korisničkih profila i postova.

Slika 5.1.3. prikazuje strukturu foldera Views, koji sadrži sve .cshtml datoteke potrebne za prikaz korisničkog interfejsa.

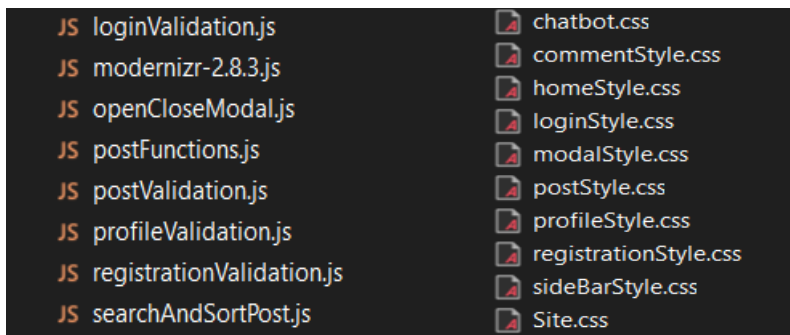


Slika 5.1.3. Views struktura

Datoteka **Registration.cshtml** sadrži HTML kod za registracionu formu, smeštenu u folderu Registration. Forma omogućava unos različitih ličnih podataka kao što su ime, prezime, adresa, grad, država, broj telefona, email, lozinka i profilna slika. Validacija unosa i dinamički prikaz odabrane slike realizovani su pomoću JavaScript funkcija ValidateAndSubmitReg() i previewImageReg(), koje se nalaze u registrationValidation.js u folderu Scripts (slika 5.1.4.). Stilizacija forme obavljena je korišćenjem CSS

datoteke `registrationStyle.css`, smeštene u folderu `Content` (slika 5.1.4.). Nakon validacije, podaci se šalju POST metodom na akciju `SubmitRegistration`, implementiranu u `RegistrationController` u folderu `Controllers`.

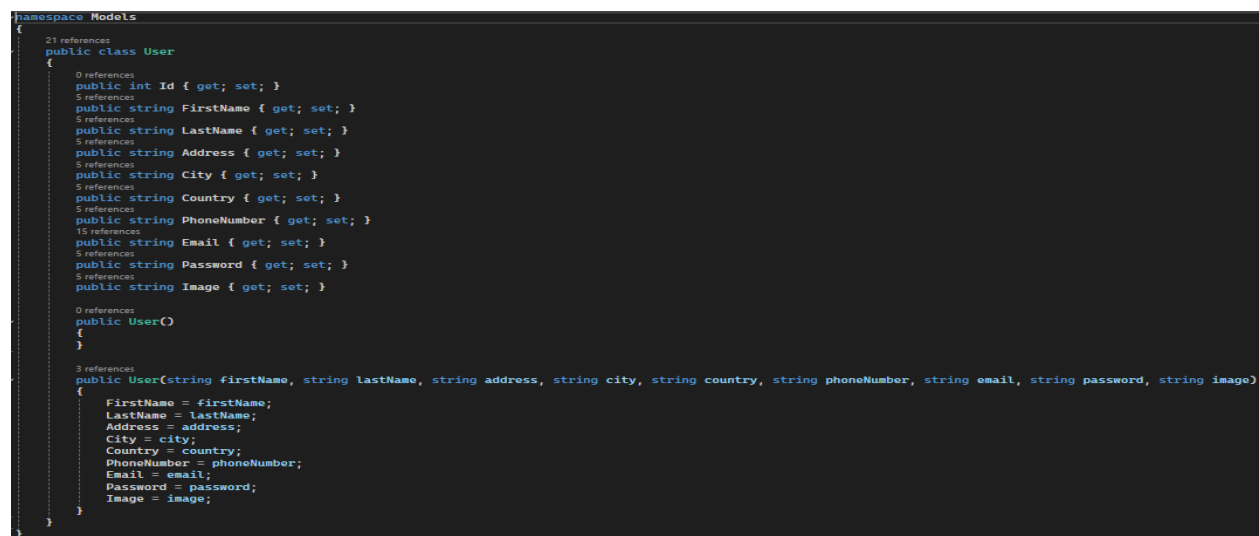
Datoteka `Login.cshtml`, smeštena u folderu `Login`, sadrži HTML kod za prijavnu formu. Forma omogućava unos emaila i lozinke kao autentifikacionih podataka. Za validaciju unosa i podnošenje forme koristi se JavaScript funkcija `ValidateAndSubmit()`, smeštena u `loginValidation.js` u folderu `Scripts` (slika 5.1.4.). Stilizacija forme obavljena je putem CSS datoteke `loginStyle.css`, koja se nalazi u folderu `Content` (slika 5.1.4.). Nakon validacije, podaci se šalju POST metodom na akciju `SubmitLogin` unutar `LoginController` u folderu `Controllers`.



Slika 5.1.4. Osnovna struktura foldera `Content` i `Scripts`

Slika 5.1.4. prikazuje osnovnu strukturu foldera `Scripts` i `Content`, u kojima su smeštene datoteke za validaciju, funkcionalnosti, i stilizaciju različitih delova aplikacije.

Datoteka **`ShowProfile.cshtml`**, smeštena u folderu `Profile`, sadrži HTML kod za prikaz i izmenu korisničkog profila unutar `WebRole` komponente aplikacije. Ova forma koristi model `User`, smešten u folderu `Models` (slika 5.1.5.), za prikaz i manipulaciju korisničkim podacima kao što su ime, prezime, adresa, grad, država, broj telefona, email, lozinka i profilna slika. Stilizacija forme obavljena je pomoću CSS datoteke `profileStyle.css`, smeštene u folderu `Content` (slika 5.1.4.), dok validaciju unosa i podnošenje forme obavlja JavaScript funkcija `ValidateAndSubmitEditProfile()`, smeštena u `profileValidation.js` u folderu `Scripts` (slika 5.1.4.). Podaci se šalju POST metodom na akciju `EditProfile` u `ProfileController` u folderu `Controllers`.



Slika 5.1.5. User model

Slika 5.1.5. prikazuje model User, koji se koristi za rad sa korisničkim podacima.

Datoteka **Index.cshtml**, koja se nalazi u folderu Home, sadrži HTML kod za prikaz početne stranice unutar WebRole komponente aplikacije. Ova stranica koristi model HomePagePostLists, smešten u folderu Models (slika 5.1.6.), za prikaz lista korisničkih postova (MyPosts - "Moje teme"), svih postova (AllPosts - "Sve teme"), i omiljenih postova (FavoritesPosts - "Omiljene teme"). Stilizacija stranice obavljena je pomoću više CSS datoteka: sideBarStyle.css, postStyle.css, modalStyle.css, i homeStyle.css, koje su smeštene u folderu Content (slika 5.1.4.). Interaktivne funkcionalnosti, kao što su validacija unosa, otvaranje i zatvaranje modalnih prozora, te pretraga i sortiranje postova, realizovane su pomoću JavaScript datoteka postFunctions.js, postValidation.js, openCloseModal.js, i searchAndSortPost.js, koje se nalaze u folderu Scripts (slika 5.1.4.). Nakon validacije, podaci se šalju POST metodom na odgovarajuće akcije unutar HomeController u folderu Controllers.

```
using System.Collections.Generic;

namespace Models
{
    4 references
    public class HomePagePostLists
    {
        2 references
        public List<Post> MyPosts { get; set; } = new List<Post>();
        11 references
        public List<Post> AllPosts { get; set; } = new List<Post>();
        1 reference
        public List<Post> FavoritesPosts { get; set; } = new List<Post>(); //DODATQ
    }
}
```

Slika 5.1.6. HomePagePostLists model

Slika 5.1.6. prikazuje model HomePagePostLists, koji se koristi za rad sa postovima i temama.

Datoteka **PostPage.cshtml**, smeštena u folderu PostPage, sadrži HTML kod za prikaz pojedinačnog posta i njegovih komentara unutar WebRole komponente aplikacije. Ova stranica koristi modele Post i Comment, koji se nalaze u folderu Models (slika 5.1.7.), za prikaz naslova, opisa, slike, broja lajkova i dislajkova, kao i komentara vezanih za određeni post. Stilizacija stranice realizovana je pomoću CSS datoteka sideBarStyle.css, postStyle.css, homeStyle.css, i commentStyle.css, smeštenih u folderu Content (slika 5.1.4.). Interaktivnost stranice, uključujući funkcionalnosti za dodavanje posta u omiljene, upravljanje lajkovima i dislajkovima, kao i dodavanje novih komentara, obezbeđena je putem JavaScript datoteka postFunctions.js, commentFunctions.js, i openCloseModal.js, smeštenih u folderu Scripts (slika 5.1.4.). Ove funkcionalnosti omogućavaju korisnicima jednostavnu interakciju sa sadržajem na stranici.

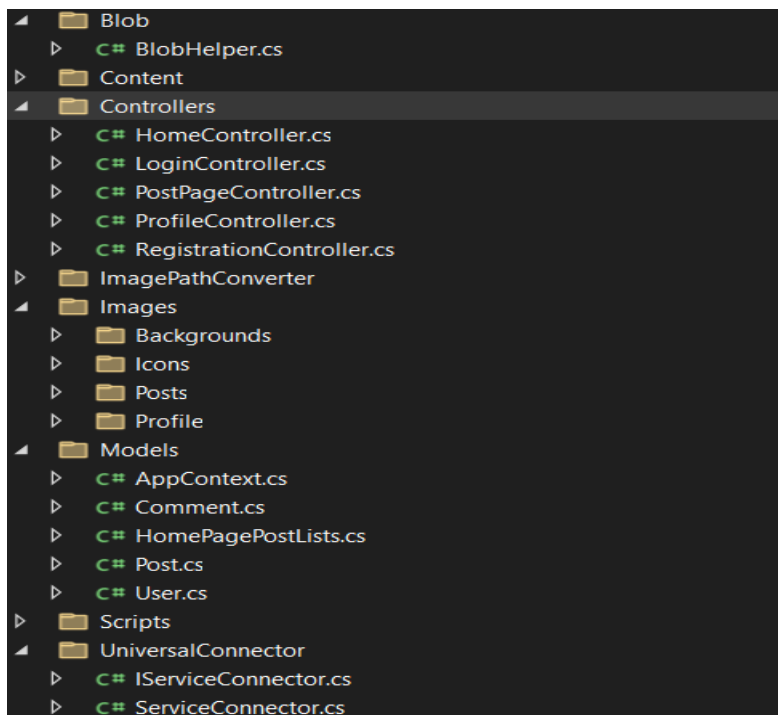
```
using System.Collections.Generic;

namespace Models
{
    22 references
    public class Post
    {
        5 references
        public int Id { get; set; }
        6 references
        public string Title { get; set; }
        4 references
        public string Description { get; set; }
        4 references
        public string Image { get; set; }
        4 references
        public List<Comment> Comments { get; set; }
        5 references
        public string UserEmail { get; set; }
        0 references
        public int UserId { get; set; }
        4 references
        public int Like { get; set; }
        4 references
        public int Unlike { get; set; }
        2 references
        public int FavoriteCount { get; set; }
        3 references
        public Post()
        {
            Comments = new List<Comment>();
        }
    }

    11 references
    public class Comment
    {
        3 references
        public string Text { get; set; }
        3 references
        public int PostId { get; set; }
        3 references
        public string UserEmail { get; set; }
        4 references
        public int Like { get; set; }
        4 references
        public int Unlike { get; set; }
        2 references
        public Comment()
        {
        }
        1 reference
        public Comment(string text, int postId, string userEmail, int like, int unlike)
        {
            Text = text;
            PostId = postId;
            UserEmail = userEmail;
            Like = like;
            Unlike = unlike;
        }
    }
}
```

Slika 5.1.7. Post i Comment modeli

Slika 5.1.7. prikazuje modele Post i Comment, koji se koriste za rad sa postovima i komentarima.



Slika 5.1.8. Struktura Controllers, Blob, Images, Models i UniversalConnector foldera

Na slici 5.1.8. prikazana je struktura foldera Controllers, koji se iznad spominjao, a koji ćemo u nastavku detaljnije opisati šta radi svaki od kontrolera. Uz to, vide se i strukture ostalih foldera kao što su Blob, Images, Models i UniversalConnector. Neki od njih su se već spominjali i prikazivali u kodu, dok ćemo neke tek prikazati jer se koriste u kontrolerima.

RegistrationController je ključni deo aplikacije smešten u folderu Controllers, koji upravlja procesom registracije korisnika unutar WebRole komponente. Ovaj kontroler koristi modele User (slika 5.1.5.), smešten u folderu Models, i UserData, koji se nalazi u komponenti Common u folderu Entities, kako bi kreirao nove korisnike na osnovu podataka unetih u registracionu formu. Za prilagođavanje putanje do profilne slike korisnika koristi se klasa ProfilePathConverter (slika 5.1.9.) iz foldera ImagePathConverter, dok klasa BlobHelper (slika 5.1.15.) iz foldera Blob omogućava učitavanje slike u Azure Blob Storage pod nazivom "redditprofileimages".

```
using System.IO;

namespace WebRole.ImagePathConverter
{
    2 references
    public class ProfilePathConverter
    {
        2 references
        public string ReplacePath(string path)
        {
            string fileName = Path.GetFileName(path);
            string newFormat = $"/Images/Profile/{fileName}";
            return newFormat;
        }
    }
}
```

Slika 5.1.9. ProfilePathConverter

Ovaj kontroler koristi ServiceConnector (slika 5.1.12.), koji nasleđuje interfejs IServiceConnector iz foldera UniversalConnector za uspostavljanje TCP veze sa UserService servisom, koji se nalazi na adresi "net.tcp://localhost:10100/UserService" (slika 3.2.3.). IUserService interfejs, smešten u komponenti Common u folderu Interfaces, omogućava pristup podacima korisnika kroz metode servisa. Detaljna implementacija logike biće objašnjena u narednim delovima. Slika 5.1.10. prikazuje povezivanje na UserService za korišćenje metoda.

```
// Connect to the user service
ServiceConnector<IUserService> serviceConnector = new ServiceConnector<IUserService>();
serviceConnector.Connect("net.tcp://localhost:10100/UserService");
IUserService userService = serviceConnector.GetProxy();
```

Slika 5.1.10. Povezivanje na UserService

Unos podataka iz registracione forme se validira i obrađuje u metodi SubmitRegistration, koja kreira novog korisnika na osnovu dostavljenih informacija. Ukoliko je priložena profilna slika, ona se učitava u Blob Storage uz pomoć BlobHelper (slika 5.1.15.), nakon čega se korisnik dodaje u internu kolekciju korisnika ApplicationContext.Users (slika 5.1.11.), koja će se kasnije koristiti za potrebe aplikacione logike, poput prikaza i izmene profila u ProfileController. Istovremeno, korisnik se dodaje u bazu podataka korišćenjem UserService servisa putem metode userService.AddUser(newUser).

```
using System.Collections.Generic;

namespace Models
{
    17 references
    public static class ApplicationContext
    {
        2 references
        public static List<User> Users { get; set; } = new List<User>();

        public static HomePagePostLists homePagePostLists = new HomePagePostLists();
    }
}
```

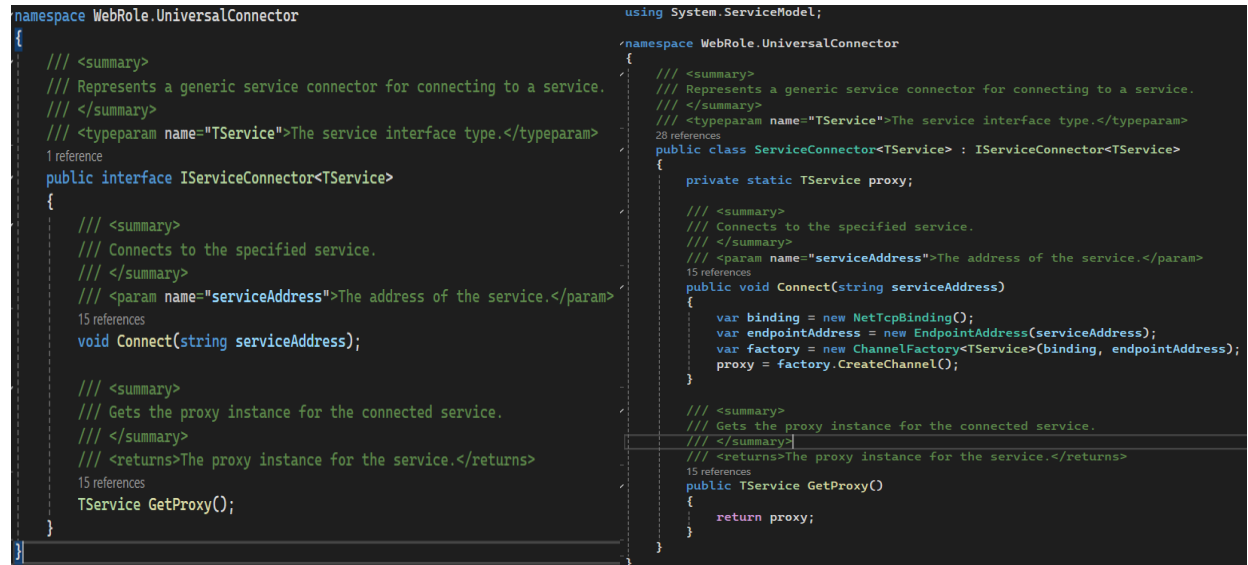
Slika 5.1.11. ApplicationContext

Ukoliko registracija uspe, korisnik se preusmerava na stranicu za prijavu, a u suprotnom, vraća se na stranicu za registraciju, uz odgovarajuće debug informacije zapisane u konzoli.

LoginController je ključni deo aplikacije smešten u folderu Controllers, koji upravlja procesom autentifikacije korisnika unutar WebRole komponente. Povezivanje na UserService za korišćenje metoda ovog servisa realizovano je na isti način kao u RegistrationController, što je prikazano i objašnjeno na slikama 5.1.10. i 5.1.12. Ovim je omogućeno korišćenje metoda UserService-a. Kontroler koristi metodu SubmitLogin za obradu podataka unetih u prijavnu formu. Korisnički email i lozinka se prikupljaju iz zahteva, nakon čega se putem UserService servisa preuzima korisnik sa odgovarajućom email adresom pomoću metode userService.GetUser(email).

Validacija lozinke obavlja se korišćenjem klase PasswordHasher iz projekta Common u folderu Cryptography (slika 5.1.13.), koja služi za verifikaciju korisničke lozinke. Ako su email i lozinka ispravni, kreira se novi User objekat (iz foldera Models, slika 5.1.5.), koji se smešta u sesiju (Session["LoggedInUser"]). Nakon uspešne prijave, korisnik se preusmerava na početnu stranicu aplikacije.

ProfileController je ključni deo aplikacije smešten u folderu Controllers, koji upravlja procesom prikaza i izmene korisničkog profila unutar WebRole komponente. Povezivanje na UserService, kako bi se koristile metode servisa, realizovano je na isti način kao i kod RegistrationController i LoginController, što je prikazano i objašnjeno na slikama 5.1.10. i 5.1.12. U ovom kontroleru koristi se metoda userService.UpdateUser za ažuriranje podataka o korisniku.



```

namespace WebRole.UniversalConnector
{
    /// <summary>
    /// Represents a generic service connector for connecting to a service.
    /// </summary>
    /// <typeparam name="TService">The service interface type.</typeparam>
    1 reference
    public interface IServiceConnector<TService>
    {
        /// <summary>
        /// Connects to the specified service.
        /// </summary>
        /// <param name="serviceAddress">The address of the service.</param>
        15 references
        void Connect(string serviceAddress);

        /// <summary>
        /// Gets the proxy instance for the connected service.
        /// </summary>
        /// <returns>The proxy instance for the service.</returns>
        15 references
        TService GetProxy();
    }
}

using System.ServiceModel;

namespace WebRole.UniversalConnector
{
    /// <summary>
    /// Represents a generic service connector for connecting to a service.
    /// </summary>
    /// <typeparam name="TService">The service interface type.</typeparam>
    28 references
    public class ServiceConnector<TService> : IServiceConnector<TService>
    {
        private static TService proxy;

        /// <summary>
        /// Connects to the specified service.
        /// </summary>
        /// <param name="serviceAddress">The address of the service.</param>
        15 references
        public void Connect(string serviceAddress)
        {
            var binding = new NetTcpBinding();
            var endpointAddress = new EndpointAddress(serviceAddress);
            var factory = new ChannelFactory<TService>(binding, endpointAddress);
            proxy = factory.CreateChannel();
        }

        /// <summary>
        /// Gets the proxy instance for the connected service.
        /// </summary>
        /// <returns>The proxy instance for the service.</returns>
        15 references
        public TService GetProxy()
        {
            return proxy;
        }
    }
}

```

Slika 5.1.12. IServiceConnector i ServiceConnector

Metoda ShowProfile koristi se za prikaz korisničkog profila. Ova metoda preuzima trenutno prijavljenog korisnika iz sesije (Session["LoggedInUser"]) i prikazuje njegove podatke na stranici. Ako je korisnik prijavljen, pretražuju se podaci iz interne kolekcije AppContext.Users (slika 5.1.11.) kako bi se dobio ažurirani profil korisnika, a zatim se ti podaci prikazuju na stranici.

Metoda EditProfile koristi se za obradu izmene korisničkog profila. Novi podaci, uključujući ime, prezime, adresu, grad, državu, broj telefona, email, lozinku i putanju do slike, preuzimaju se iz forme. Ako su svi podaci validni, kreira se novi User objekat (iz foldera Models, slika 5.1.5.) i ažurira se korisnička sesija. Potom se putem UserService servisa ažuriraju podaci korisnika u bazi podataka korišćenjem metode userService.UpdateUser(updateUser.Email, updateUser.Data). U slučaju uspešnog ažuriranja profila, korisnik se preusmerava nazad na stranicu za prikaz profila, dok se u slučaju neuspešnog ažuriranja prikazuju odgovarajuće debug informacije u konzoli, a korisnik ostaje na istoj stranici sa nepromenjenim podacima.

HomeController je centralni deo aplikacije smešten u folderu Controllers, koji upravlja prikazom i obradom postova unutar WebRole komponente. HomeController takođe koristi isto povezivanje za servise (slika 5.1.12.), koje je univerzalno za sve kontrolere, za uspostavljanje TCP veze sa PostService, CommentService, i FavouritesService servisima. Ovi servisi omogućavaju obradu i manipulaciju podacima vezanim za postove, komentare i omiljene postove korisnika. Konekcija sa ovim servisima je detaljno prikazana na slici 5.1.13.

- PostService: TCP veza se uspostavlja sa PostService servisom, koji se nalazi na adresi "net.tcp://localhost:10101/PostService". Interfejs IPostService, smešten u komponenti Common u folderu Interfaces, pruža pristup metodama za rad sa postovima.

- CommentService: Veza sa CommentService servisom uspostavlja se putem adrese "net.tcp://localhost:10102/CommentService". Interfejs ICommentService, koji se takođe nalazi u komponenti Common u folderu Interfaces, pruža pristup metodama za rad sa komentarima.
- FavouritesService: TCP veza se uspostavlja sa FavouritesService servisom, koji se nalazi na adresi "net.tcp://localhost:10103/FavouritesService". Interfejs IFavouritesService, koji je smešten u komponenti Common u folderu Interfaces, omogućava rad sa omiljenim postovima korisnika.

Metoda Index koristi se za prikaz početne stranice aplikacije. Ako je korisnik prijavljen, uspostavlja se veza sa PostService, CommentService, i FavouritesService servisima (slika 5.1.13.) kako bi se preuzeli svi postovi metodom postService.GetAllPosts(), komentari metodom commentService.GetAllComments(), i omiljeni postovi iz baze podataka metodom favouritesService.GetAllFavourites(). Nakon preuzimanja podataka, svi postovi se organizuju u tri liste: AllPosts (svi postovi), MyPosts (postovi korisnika) i FavoritesPosts (omiljeni postovi), koje se prikazuju na početnoj stranici. Organizacija (sortiranje) omiljenih postova obavljena je pomoću funkcije GetSortedFavoritesByPostId, koja postove sortira prema broju puta koliko su dodati u favorite.

```
// Connect to the post service
ServiceConnector<IPostService> serviceConnector = new ServiceConnector<IPostService>();
serviceConnector.Connect("net.tcp://localhost:10101/PostService");
IPostService postService = serviceConnector.GetProxy();

// Connect to the comment service
ServiceConnector<ICommentService> serviceConnectorComment = new ServiceConnector<ICommentService>();
serviceConnectorComment.Connect("net.tcp://localhost:10102/CommentService");
ICommentService commentService = serviceConnectorComment.GetProxy();

// Connect to the favourites service - DODATO
ServiceConnector<IFavouritesService> serviceConnectorFavourites = new ServiceConnector<IFavouritesService>();
serviceConnectorFavourites.Connect("net.tcp://localhost:10103/FavouritesService");
IFavouritesService favouritesService = serviceConnectorFavourites.GetProxy();
```

Slika 5.1.13. Povezivanje na PostService, CommentService, FavouritesService

Metoda AddNewPost obrađuje dodavanje novog posta. Podaci uneti u formu za kreiranje posta, uključujući naslov, opis i sliku, prikupljaju se iz zahteva, a zatim se validiraju i obrađuju. PostPathConverter klasa (slika 5.1.14.) iz foldera ImagePathConverter koristi se za prilagođavanje putanje do slike pre nego što se ona po potrebi učita u Azure Blob Storage pod nazivom "redditpostimages" pomoću BlobHelper klase (slika 5.1.15.) iz foldera Blob. Novi post se zatim dodaje u bazu podataka putem PostService servisa (na koji se prethodno povezalo kao na slici 5.1.13.) korišćenjem metode postService.AddPost(postdata).

```
using System.IO;

namespace WebRole.ImagePathConverter
{
    2 references
    public class PostPathConverter
    {
        2 references
        public string ReplacePath(string path)
        {
            string fileName = Path.GetFileName(path);
            string newFormat = $"/Images/Posts/{fileName}";
            return newFormat;
        }
    }
}
```

Slika 5.1.14. PostPathConverter

```

using Microsoft.Azure;
using Microsoft.WindowsAzure.Storage;
using Microsoft.WindowsAzure.Storage.Blob;
using System;
using System.Drawing;
using System.IO;

namespace WebRole.Blob
{
    5 references
    public class BlobHelper
    {
        readonly CloudStorageAccount storageAccount = CloudStorageAccount.Parse(CloudConfigurationManager.GetSetting("BlobDataConnectionString")); //Connection String
        readonly CloudBlobClient blobStorage;

        2 references
        public BlobHelper()
        {
            blobStorage = storageAccount.CreateCloudBlobClient();
        }

        0 references
        internal Image DownloadImage(String containerName, String blobName)
        {
            CloudBlobContainer container = blobStorage.GetContainerReference(containerName);
            CloudBlockBlob blob = container.GetBlockBlobReference(blobName);

            using (MemoryStream ms = new MemoryStream())
            {
                blob.DownloadToStream(ms);
                return new Bitmap(ms);
            }
        }

        2 references
        public string UploadImage(Image image, String containerName, String blobName)
        {
            CloudBlobContainer container = blobStorage.GetContainerReference(containerName);
            CloudBlockBlob blob = container.GetBlockBlobReference(blobName);

            using (MemoryStream memoryStream = new MemoryStream())
            {
                image.Save(memoryStream, System.Drawing.Imaging.ImageFormat.Bmp);
                memoryStream.Position = 0;
                blob.Properties.ContentType = "image/bmp";
                blob.UploadFromStream(memoryStream);
                return blob.Uri.ToString();
            }
        }
    }
}

```

Slika 5.1.15. BlobHelper

Metoda OpenPost koristi se za obradu zahteva za otvaranje pojedinačnog posta kada korisnik želi da otvori neki post iz liste "Sve teme". Ova metoda šalje ID posta do PostPageController-a kako bi se prikazao odgovarajući post na PostPage.cshtml stranici.

Metoda DeletePost omogućava brisanje postojećeg posta. Ovaj proces se realizuje pomoću metode postService.RemovePost(postId) unutar PostService servisa, nakon čega se odgovarajući postovi brišu iz lista AllPosts ili MyPosts, koje se nalaze u klasi AppContext u listi homePagePostLists() (slika 5.1.11.).

Metoda PostSort omogućava korisnicima da sortiraju postove prema određenim kriterijumima iz liste homePagePostLists() u klasi AppContext (slika 5.1.11.).

Metoda PostSearch omogućava pretragu postova po nazivu iz liste homePagePostLists() u klasi AppContext (slika 5.1.11.). Sortirani i pretraženi postovi prikazuju se na početnoj stranici korisnicima.

Metoda AddToFavorites koristi se za dodavanje posta u listu omiljenih postova. Ovaj proces se realizuje putem FavouiritesService servisa (na koji se prethodno povezalo kao na slici 5.1.13.), koji dodaje post u listu omiljenih postova korisnika metodom favouritesService.AddToFavourites(newFav).

PostPageController je ključni deo aplikacije smešten u folderu Controllers, koji upravlja prikazom pojedinačnih postova i obradom komentara unutar WebRole komponente.

PostPageController se povezuje kao i ostali kontroleri univerzalno preko Universal Connector-a (slika 5.1.12.), za uspostavljanje TCP veze sa CommentService i PostService servisima. Ovi servisi omogućavaju obradu i manipulaciju podacima vezanim za komentare i postove korisnika. Konekcija sa ovim servisima detaljno je prikazana na slici 5.1.13., jer se povezivanje sa istim servisima obavlja na isti način u svim kontrolerima.

Metoda `PostPage` koristi se za prikaz odabranog posta na osnovu njegovog ID-a. Podaci o postu preuzimaju se iz interne kolekcije `AppContext.homePagePostLists.AllPosts` (slika 5.1.11.), a zatim se prikazuju korisniku na odgovarajućoj stranici.

Metoda `NewComment` obrađuje zahteve za dodavanje novih komentara na postove. Kada korisnik unese komentar, kreira se objekat klase `Comment` iz foldera `Models` (slika 5.1.7.), koji se potom konvertuje u objekat `CommentData` koji se nalazi u projektu `Common` u folderu `Entities` i šalje `CommentService` servisu putem metode `commentService.AddComment(commentData)`. Komentar se takođe dodaje u odgovarajući `CloudQueue` (komentar queue) putem `QueueHelper` klase iz projekta `Common` u folderu `CommentQueue` (slika 5.1.16.).

```
using Microsoft.Azure;
using Microsoft.WindowsAzure.Storage;
using Microsoft.WindowsAzure.Storage.Queue;
using System;

namespace Common.Comment_queue
{
    2 references
    public class QueueHelper
    {
        2 references
        public static CloudQueue GetQueueReference(String queueName)
        {
            CloudStorageAccount storageAccount = CloudStorageAccount.Parse(CloudConfigurationManager.GetSetting("DataConnectionString"));
            CloudQueueClient queueClient = storageAccount.CreateCloudQueueClient();
            CloudQueue queue = queueClient.GetQueueReference(queueName);
            queue.CreateIfNotExists();

            return queue;
        }
    }
}
```

Slika 5.1.16. *QueueHelper* klasa

Metode `LikeComment` i `UnlikeComment` koriste se za upravljanje brojem lajkova i dislajkova za komentare. Kada korisnik lajkuje ili dislajkuje komentar, podaci se ažuriraju putem `CommentService` servisa korišćenjem metode `commentService.UpdateComment(commentData)`, koja ažurira broj lajkova ili dislajkova za određeni komentar.

Metode `LikePost` i `UnlikePost` koriste se za upravljanje brojem lajkova i dislajkova za postove. Podaci o lajkovima i dislajkovima se preuzimaju iz baze putem metode `postService.GetPost(postId)` i ažuriraju korišćenjem metode `postService.UpdatePost(postData)`.

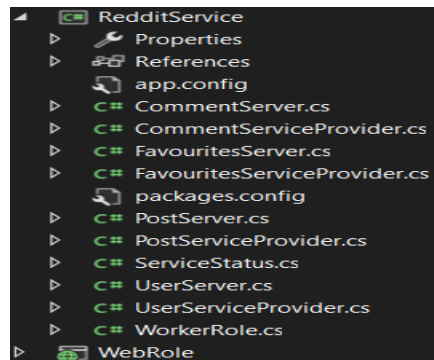
Sve interakcije sa bazom podataka, bilo da se radi o komentarima ili postovima, obavljaju se putem prethodno uspostavljenih konekcija na `CommentService` i `PostService` servise, kako je prikazano na slici 5.1.13. Na ovaj način, podaci o lajkovima, dislajkovima, i komentarima se ažuriraju u realnom vremenu i odmah reflektuju u korisničkom interfejsu.

Folder **Images** (slika 5.1.8.) sadrži slike organizovane u četiri podfoldere: `Backgrounds`, `Icons`, `Posts`, i `Profile`. Putanje do slika iz ovih foldera generišu se putem klasa `PostPathConverter` (slika 5.1.14.) i `ProfilePathConverter` (slika 5.1.9.).

Folder **App_Start** (slika 5.1.1.) sadrži ključne konfiguracione klase koje upravljaju ponašanjem aplikacije: `BundleConfig`: Upravlja grupisanjem i optimizacijom skripti i stilova, smanjujući vreme učitavanja stranica. `FilterConfig`: Definiše globalne filtere za obradu grešaka i druge zajedničke zadatke. `RouteConfig`: Upravlja rutiranjem URL-ova, određujući kako se URL-ovi mapiraju na kontrolere i akcije.

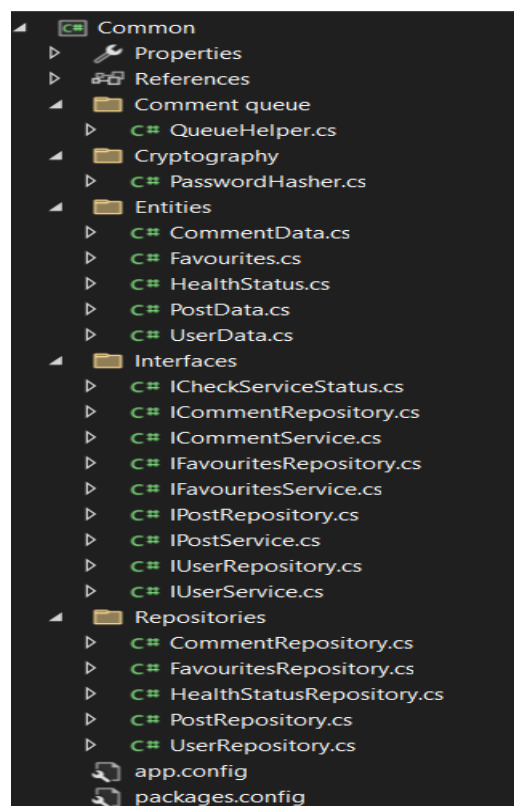
5.2. RedditService i Common komponenta

Svi servisi na koje se u WebRole u kontrolerima povezuje da bi se iz tih servisa koristila logika nalaze se u RedditService. Slika 5.2.1. prikazuje arhitekturu RedditService-a.



Slika 5.2.1. RedditService arhitektura

U Common projektu nalaze se sve komponente aplikacije koje su zajedničke i koje će koristiti iste podatke više servisa. Arhitektura Common-a je data na slici 5.2.2.



Slika 5.2.2. Common arhitektura

U folderu Comment queue se nalazi klasa **QueueHelper** klasa prikazana na slici 5.1.16. QueueHelper služi za upravljanje Azure Queue skladištem. Omogućava povezivanje sa Azure Storage nalogom i vraća referencu na određeni red (queue), kreirajući ga ako ne postoji. Upotrebljava se u PostPageController za kreiranje Queue-a u bazi podataka pod nazivom "comment" gdje se ubacuje komentar dodat na temu.

Takođe se koristi i u WorkerRole NotificationService-a gdje se provjerava da li ima komentara i za koju temu da bi se vlasnici tema obavestili da je komentar postavljen ali o tome kasnije u NotificationService poglavlju. Folder Interfaces služi za kreiranje interfejsa koji će se implementirati.

Interfejs **ICheckServiceStatus** definiše ugovor o usluzi za proveru statusa servisa putem WCF-a, uključujući jednu metodu CheckServiceStatus (slika 5.2.3.) koja vraća bool vrednost.

```
using System.ServiceModel;

namespace Common.Interfaces
{
    [ServiceContract]
    10 references
    public interface ICheckServiceStatus
    {
        [OperationContract]
        6 references
        bool CheckServiceStatus();
    }
}
```

Slika 5.2.3. ICheckServiceStatus interfejs

Ovaj interfejs će se implementirati u NotificationService i RedditService – u i koristiti kasnije za provjeru da li je neki od servisa pao, primjer implementacije je dat na slici 5.2.4. isti je u oba projekta.

```
using Common.Interfaces;

namespace RedditService
{
    1 reference
    public class ServiceStatus : ICheckServiceStatus
    {
        5 references
        public bool CheckServiceStatus()
        {
            return true;
        }
    }
}
```

Slika 5.2.4. Klasa ServiceStatus

Interfejsi ICommentRepository, IFavouritesRepository, IPostRepository i IUserRepository definišu osnovne metode za rad sa korisničkim podacima u bazi.

Interfejs **ICommentService** definiše metode: AddComment za dodavanje komentara, GetAllComments za preuzimanje svih komentara, GetComment za preuzimanje komentara po ID-u, i UpdateComment za ažuriranje komentara.

Interfejs **IFavouritesRepository** definiše metode Create za kreiranje omiljenih postova i ReadAll za preuzimanje svih omiljenih postova.

Interfejs **IPostRepository** definiše metode Create za kreiranje novog posta, Read za čitanje posta prema ID-u, ReadAll za preuzimanje svih postova, Delete za brisanje posta prema ID-u, i Update za ažuriranje postojećeg posta.

Interfejs **IUserRepository** definiše metode Create za kreiranje novog korisnika, Read za preuzimanje korisnika prema email adresi, ReadAll za preuzimanje svih korisnika, Update za ažuriranje korisničkih

podataka na osnovu email adrese, i Exists za proveru postojanja korisnika prema email adresi, za ovaj interfejs je dat primjer na slici 5.2.5.

```
using Common.Entities;
using System.Linq;

namespace Common.Interfaces
{
    1 reference
    public interface IUserRepository
    {
        2 references
        void Create(UserData user);
        2 references
        UserData Read(string email);
        4 references
        IQueryable<UserData> ReadAll();
        2 references
        void Update(string email, UserData user);
        2 references
        bool Exists(string email);
    }
}
```

Slika 5.2.5. IUserRepository

U folderu Repositories (slika 5.2.2.) nalaze se klase CommentRepository, FavouritesRepository, PostRepository i UserRepository koji implementiraju ove interfejse i koriste Azure Table Storage za skladištenje korisničkih podataka, realizujući sve navedene metode u skladu sa specifičnostima Azure Storage-a. Kasnije će se ove metode pozivati u RedditService klasama kako bi se komuniciralo sa bazom podataka. Kao primer je data klasa UserRepository na slici 5.2.6. ostale rade istu stvar za svoje potrebe.

```
namespace Common.Repositories
{
    3 references
    public class UserRepository : IUserRepository
    {
        private readonly CloudStorageAccount _storageAccount;
        private readonly CloudTable _table;

        1 reference
        public UserRepository()
        {
            string connectionString = CloudConfigurationManager.GetSetting("DataConnectionString");
            if (string.IsNullOrEmpty(connectionString))
            {
                throw new InvalidOperationException("Connection string for Azure Storage is not set.");
            }

            _storageAccount = CloudStorageAccount.Parse(connectionString);
            CloudTableClient tableClient = new CloudTableClient(new Uri(_storageAccount.TableEndpoint.AbsoluteUri),
                _storageAccount.Credentials);
            _table = tableClient.GetTableReference("UserTableTemp");
            _table.CreateIfNotExists();
        }

        4 references
        public IQueryable<UserData> ReadAll()
        {
            try
            {
                var results = from g in _table.CreateQuery<UserData>()
                    where g.PartitionKey == "User"
                    select g;

                return results;
            }
            catch (Exception ex)
            {
                throw new Exception($"{ex}");
            }
        }

        2 references
        public void Create(UserData newUser)
        {
            try
            {
                TableOperation insertOperation = TableOperation.Insert(newUser);
                TableResult result = _table.Execute(insertOperation);

                if (result.HttpStatusCode < 200 || result.HttpStatusCode >= 300)
                {
                    throw new InvalidOperationException("Failed to insert user into Azure Table Storage.");
                }
            }
            catch (Exception ex)
            {
                throw new Exception($"{ex}");
            }
        }

        2 references
        public bool Exists(string email)
        {
            return ReadAll().Where(s => s.RowKey == email).FirstOrDefault() != null;
        }

        2 references
        public UserData Read(string email)
        {
            return ReadAll().Where(p => p.RowKey == email).FirstOrDefault();
        }

        2 references
        public void Update(string email, UserData user)
        {
            if (string.IsNullOrEmpty(email) || user == null)
            {
                return;
            }

            try
            {
                var existingUser = (from g in _table.CreateQuery<UserData>()
                    where g.PartitionKey == "User" && g.RowKey == email
                    select g).FirstOrDefault();

                if (existingUser != null)
                {
                    existingUser.FirstName = user.FirstName;
                    existingUser.LastName = user.LastName;
                    existingUser.Address = user.Address;
                    existingUser.City = user.City;
                    existingUser.Country = user.Country;
                    existingUser.PhoneNumber = user.PhoneNumber;
                    existingUser.Email = user.Email;
                    existingUser.Password = user.Password;
                    existingUser.Image = user.Image;
                    TableOperation updateOperation = TableOperation.Replace(existingUser);
                    _table.Execute(updateOperation);
                }
                else
                {
                    return;
                }
            }
            catch (Exception)
            {
                return;
            }
        }
    }
}
```

Slika 5.2.6. UserRepository

Pored ovih postoji i klasa HealthStatusRepository (slika 5.2.7.) koja ce se koristiti u HealthMonitoringService servisu, koja omogućava rad sa tabelom HealthStatusTableTemp u Azure Table Storage-u. Klasa koristi Create metodu za dodavanje novog zapisa tipa HealthStatus u tabelu, dok se povezivanje sa Azure Storage-om vrši preko connection stringa. To je za ispisivanje poruka OK I NOT_OK za status servisa (slika 4.1.5).

```

using Common.Entities;
using Microsoft.Azure;
using Microsoft.WindowsAzure.Storage;
using Microsoft.WindowsAzure.Storage.Table;
using System;

namespace Common.Repositories
{
    3 references
    public class HealthStatusRepository
    {
        private readonly CloudStorageAccount _storageAccount;
        private readonly CloudTable _table;

        1 reference
        public HealthStatusRepository()
        {
            string connectionString = CloudConfigurationManager.GetSetting("HealthStatusConnectionString"); //Connection String
            if (string.IsNullOrEmpty(connectionString))
            {
                throw new InvalidOperationException("Connection string for Azure Storage is not set.");
            }

            _storageAccount = CloudStorageAccount.Parse(CloudConfigurationManager.GetSetting("HealthStatusConnectionString"));
            CloudTableClient tableClient = new CloudTableClient(new Uri(_storageAccount.TableEndpoint.AbsoluteUri), _storageAccount.Credentials);
            _table = tableClient.GetTableReference("HealthStatusTableTemp");
            _table.CreateIfNotExists();
        }

        4 references
        public void Create(HealthStatus newHealthStatus)
        {
            try
            {
                TableOperation insertOperation = TableOperation.Insert(newHealthStatus);
                TableResult result = _table.Execute(insertOperation);

                if (result.HttpStatusCode < 200 || result.HttpStatusCode >= 300)
                {
                    throw new InvalidOperationException("Failed to insert user into Azure Table Storage.");
                }
            }
            catch (Exception ex)
            {
                throw new Exception($"{ex}");
            }
        }
    }
}

```

Slika 5.2.7. Klasa HealthStatusRepository

U folderu Entities (slika 5.2.2.) nalaze se klase koje definišu modele podataka, a koje se koriste u različitim delovima aplikacije za rad sa Azure Table Storage-om. Svaka klasa predstavlja specifičan entitet i koristi TableEntity kao osnovnu klasu, omogućavajući tako direktan rad sa Azure Storage tabelama. Postoje:

CommentData: Ova klasa predstavlja podatke o komentarima koji se čuvaju u bazi. Sadrži attribute kao što su Id, Text (tekst komentara), PostId (ID posta na koji se komentar odnosi), UserEmail (email korisnika koji je postavio komentar), te brojevi lajkova i dislajkova (Like i Unlike). Klasa takođe uključuje logiku za generisanje jedinstvenog ID-a za svaki komentar.

Favourites: Ova klasa služi za upravljanje omiljenim postovima korisnika. Sadrži attribute kao što su Id, PostId (ID omiljenog posta) i UserEmail (email korisnika koji je dodao post u omiljene). Slično klasi CommentData, koristi logiku za generisanje jedinstvenog ID-a.

HealthStatus: Ova klasa koristi se za praćenje statusa servisa u aplikaciji. Sadrži attribute ServiceType (tip servisa) i Status (trenutni status servisa). Jedinstveni ID za svaki status generiše se na osnovu tipa servisa (RedditService ili NotificationService).

PostData: Ova klasa definiše podatke o postovima. Sadrži attribute kao što su Id, Title (naslov posta), Description (opis posta), Image (putanja slike posta), UserEmail (email korisnika koji je kreirao post), te brojevi lajkova i dislajkova (Like i Unlike). Klasa takođe uključuje logiku za generisanje jedinstvenog ID-a za svaki post.

UserData: Ova klasa predstavlja podatke o korisnicima. Sadrži osnovne podatke o korisnicima kao što su FirstName (ime), LastName (prezime), Address (adresa), City (grad), Country (država), PhoneNumber

(broj telefona), Email (email adresa), Password (lozinka) i Image (putanja slike profila). Koristi se za čuvanje i ažuriranje korisničkih podataka u bazi. Primjer je dat na slici 5.2.8. to jeste klasa UserData. Ona se na primjer koristi u IUserRepository, UserRepositoryu, IUserService, UserService kao i kontrolerima u WebApi projektu a to su ProfileController, RegisterController, LoginController. Tako da i sve ostale klase se koriste na određenim mjestima specifičnim za implemntaciju.

```
using Microsoft.WindowsAzure.Storage.Table;

namespace Common.Entities
{
    32 references
    public class UserData : TableEntity
    {
        10 references
        public string FirstName { get; set; }
        10 references
        public string LastName { get; set; }
        10 references
        public string Address { get; set; }
        10 references
        public string City { get; set; }
        10 references
        public string Country { get; set; }
        10 references
        public string PhoneNumber { get; set; }
        11 references
        public string Email { get; set; }
        15 references
        public string Password { get; set; }
        10 references
        public string Image { get; set; }

        2 references
        public UserData(string email)
        {
            PartitionKey = "User";
            RowKey = email;
        }

        1 reference
        public UserData() { }

        4 references
        public UserData(string firstName, string lastName, string address, string city,
            string country, string phoneNumber, string email, string password, string image) : this(firstName)
        {
            FirstName = firstName;
            LastName = lastName;
            Address = address;
            City = city;
            Country = country;
            PhoneNumber = phoneNumber;
            Email = email;
            Password = password;
            Image = image;
        }
    }
}
```

Slika 5.2.8. Klasa UserData

Svaka od ovih klasa omogućava jednostavnu manipulaciju podacima u aplikaciji, dok koriste Azure Table Storage za skladištenje i upravljanje ovim podacima.

Pored gore navedenih interfejsa u folderu Interfaces jos postoje:

Interfejs **ICommentService** definiše osnovne metode koje omogućavaju rad sa komentarima unutar aplikacije. Ovaj interfejs uključuje metode za dodavanje novog komentara (AddComment), dohvaćanje svih komentara (GetAllComments), preuzimanje specifičnog komentara na osnovu njegovog ID-a (GetComment), kao i za ažuriranje postojećeg komentara (UpdateComment).

Interfejs **IFavouritesService** omogućava rad sa omiljenim postovima korisnika. Ovaj interfejs obuhvata metode za dodavanje postova u listu omiljenih (AddToFavourites) i preuzimanje svih omiljenih postova (GetAllFavourites).

Interfejs **IPostService** pruža osnovne metode za rad sa postovima unutar aplikacije. Kroz ovaj interfejs se definišu metode za dodavanje novih postova (AddPost), preuzimanje svih postova (GetAllPosts), dohvaćanje specifičnog posta na osnovu ID-a (GetPost), brisanje posta (RemovePost), i ažuriranje postojećeg posta (UpdatePost).

Interfejs **IUserService** definiše metode za rad sa korisnicima u aplikaciji. Ovaj interfejs uključuje metode za dodavanje novih korisnika (AddUser), preuzimanje svih korisnika (GetAllUsers), dohvaćanje specifičnog korisnika na osnovu email adrese (GetUser), ažuriranje podataka o korisniku (UpdateUser), kao i proveru postojanja korisnika u bazi (Exists). Kao primjer dat je ovaj primjer na slici 5.2.9.

```
using Common.Entities;
using System.Collections.Generic;
using System.ServiceModel;

namespace Common.Interfaces
{
    [ServiceContract]
    11 references
    public interface IUserService
    {
        [OperationContract]
        2 references
        void AddUser(UserData user);
        [OperationContract]
        1 reference
        List<UserData> GetAllUsers();
        [OperationContract]
        2 references
        UserData GetUser(string email);
        [OperationContract]
        2 references
        void UpdateUser(string email, UserData user);
        [OperationContract]
        1 reference
        bool Exists(string email);
    }
}
```

Slika 5.2.9. Interfejs IUserService

Ovi interfejsi se implementiraju u RedditeServic-e servisu na njih se povezuje u kontrolerima I pruža im metode I logiku za upravljanje podacima

U **RedditService** servisu se kao što se vidi na arhitekturi na slici 5.2.1 nalaze klase CommentServer, FavouritesServer, PostServer, UserServer.

CommentServer klasa hostuje servis za rad sa komentarima u okviru aplikacije. Koristi ServiceHost za povezivanje sa TCP endpointom nazvanim "CommentService" i implementira ICommentService interfejs, koji definiše metode za upravljanje komentarima, kao što su dodavanje, ažuriranje i preuzimanje komentara. Poslovna logika za ovaj servis je implementirana u klasi CommentServiceProvider. Ključne metode u ovoj klasi su Open(), koja otvara servis za primanje zahteva, i Close(), koja ga zatvara.

FavouritesServer klasa omogućava hostovanje servisa za rad sa omiljenim postovima korisnika. Ovaj servis koristi TCP endpoint nazvan "FavouritesService" i implementira IFavouritesService interfejs, koji definiše metode kao što su dodavanje i preuzimanje omiljenih postova. Logika servisa se nalazi u klasi FavouritesServiceProvider. Slično kao i kod prethodnog servisa, Open() otvara servis, dok Close() zatvara konekciju.

PostServer klasa služi za hostovanje servisa koji upravlja postovima. Endpoint za ovaj servis naziva se "PostService", a klasa koristi ServiceHost za povezivanje sa IPostService interfejsom, koji pruža metode za dodavanje, ažuriranje, preuzimanje i brisanje postova. Poslovna logika je implementirana u klasi PostServiceProvider. Ova klasa takođe ima metode Open() za otvaranje servisa i Close() za njegovo zatvaranje.

UserServer klasa hostuje servis za rad sa korisničkim podacima. Endpoint je nazvan "UserService" on se gadj sa kontrolera kao na slici 5.9., a servis se povezuje preko ServiceHost klase koja implementira IUserService interfejs (slika 5.2.9.). Ovaj interfejs omogućava operacije kao što su dodavanje, ažuriranje i preuzimanje korisnika. Poslovna logika za ovaj servis nalazi se u klasi UserServiceProvider. Kao i kod

ostalim servisa, metode `Open()` i `Close()` služe za otvaranje i zatvaranje servisa. Primjer je dat za ovu klasu I nalazi se na slici 5.2.10. ostale sadrže isto samo za svoju logiku koja je iznad objašnjena.

```

namespace RedditService
{
    3 references
    public class UserServer
    {
        private readonly ServiceHost serviceHost;
        private readonly string endPointName = "UserService";

        1 reference
        public UserServer()
        {
            RoleInstanceEndpoint inputEndpoint = RoleEnvironment.CurrentRoleInstance.InstanceEndpoints[endPointName];
            string endpoint = string.Format("net.tcp://{0}/{1}", inputEndpoint.IPEndpoint, endPointName);
            serviceHost = new ServiceHost(typeof(UserServiceProvider));
            NetTcpBinding binding = new NetTcpBinding();
            serviceHost.AddServiceEndpoint(typeof(IUserService), binding, endpoint);
        }

        1 reference
        public void Open()
        {
            try
            {
                serviceHost.Open();
                Trace.TraceInformation(string.Format("Host for {0} endpoint type opened successfully at {1}", endPointName, DateTime.Now));
            }
            catch (Exception e)
            {
                Trace.TraceInformation("Host open error for {0} endpoint type. Error message is: {1}. ", endPointName, e.Message);
            }
        }

        1 reference
        public void Close()
        {
            try
            {
                serviceHost.Close();
                Trace.TraceInformation(string.Format("Host for {0} endpoint type closed successfully at {1}", endPointName, DateTime.Now));
            }
            catch (Exception e)
            {
                Trace.TraceInformation("Host close error for {0} endpoint type. Error message is: {1}. ", endPointName, e.Message);
            }
        }
    }
}

```

Slika 5.2.10. Klasa UserServer

Uz ove klase su implementirane i njihove poslovne logike koje se nazivaju: `CommentServiceProvider`, `FavouritesServiceProvider`, `PostServiceProvider`, `UserServiceProvider`.

Ove klase (`CommentServiceProvider`, `FavouritesServiceProvider`, `PostServiceProvider`, i `UserServiceProvider`) implementiraju logiku poslovnih servisa unutar aplikacije, koristeći odgovarajuće repozitorijume za interakciju s podacima. Svaka od ovih klasa implementira interfejs koji definiše metode potrebne za rad sa specifičnim entitetima (komentari, omiljeni postovi, postovi, korisnici).

CommentServiceProvider implementira `ICommentService` interfejs, pružajući funkcionalnosti kao što su dodavanje (`AddComment`), preuzimanje (`GetAllComments`, `GetComment`), i ažuriranje (`UpdateComment`) komentara. Interno koristi `CommentRepository` za upravljanje podacima o komentarima, pozivajući metode kao što su `Create`, `Read`, i `Update`.

FavouritesServiceProvider implementira `IFavouritesService` interfejs, omogućavajući dodavanje (`AddToFavourites`) i preuzimanje omiljenih postova (`GetAllFavourites`). Ova klasa koristi `FavouritesRepository` za upravljanje podacima, pozivajući metode `Create` i `ReadAll`.

PostServiceProvider implementira `IPostService` interfejs, pružajući funkcionalnosti za dodavanje (`AddPost`), preuzimanje (`GetAllPosts`, `GetPost`), brisanje (`RemovePost`), i ažuriranje (`UpdatePost`) postova. Koristi `PostRepository` za operacije nad podacima, uključujući metode `Create`, `Read`, `Delete`, i `Update`.

UserServiceProvider implementira `IUserService` interfejs i omogućava rad sa korisnicima putem metoda za dodavanje (`AddUser`), preuzimanje (`GetAllUsers`, `GetUser`), ažuriranje (`UpdateUser`), i proveru postojanja korisnika (`Exists`). Ova klasa koristi `UserRepository` za upravljanje korisničkim podacima (slika 5.2.6.), a za heširanje lozinki koristi klasu `PasswordHasher` (slika 5.12.) iz projekta `Common`. Primjer za ovu klasu dat je na slici 5.2.11.

```

namespace RedditService
{
    1 reference
    public class UserServiceProvider : IUserService
    {
        private readonly static UserRepository repository = new UserRepository();

        2 references
        public void AddUser(UserData user)
        {
            user.Password = PasswordHasher.HashPassword(user.Password);
            repository.Create(new UserData(user.Email) { FirstName = user.FirstName, LastName = user.LastName,
                Address = user.Address, City = user.City, Country = user.Country, PhoneNumber = user.PhoneNumber,
                Email = user.Email, Password = user.Password, Image = user.Image });
        }

        1 reference
        public List<UserData> GetAllUsers()
        {
            return repository
                .ReadAll()
                .Select(user => new UserData
                {
                    FirstName = user.FirstName,
                    LastName = user.LastName,
                    Address = user.Address,
                    City = user.City,
                    Country = user.Country,
                    PhoneNumber = user.PhoneNumber,
                    Email = user.Email,
                    Password = user.Password,
                    Image = user.Image
                }).ToList();
        }

        2 references
        public UserData GetUser(string email)
        {
            UserData user = repository.Read(email);
            return new UserData(user.FirstName, user.LastName, user.Address, user.City, user.Country,
                user.PhoneNumber, user.Email, user.Password, user.Image);
        }

        2 references
        public void UpdateUser(string email, UserData user)
        {
            user.Password = PasswordHasher.HashPassword(user.Password);
            repository.Update(email, new UserData(user.FirstName, user.LastName, user.Address,
                user.City, user.Country, user.PhoneNumber, user.Email, user.Password, user.Image));
        }

        1 reference
        public bool Exists(string email)
        {
            return repository.Exists(email);
        }
    }
}

```

Slika 5.2.11. Klasa UserServiceProvider

Svaka od ovih klasa sadrži poslovnu logiku specifičnu za svoj entitet i koristi repozitorijume za interakciju sa bazom podataka, osiguravajući da podaci budu pravilno upravljani i ažurirani.

```

namespace Common.Cryptography
{
    3 references
    public class PasswordHasher
    {
        2 references
        public static string HashPassword(string password)
        {
            return BCrypt.Net.BCrypt.HashPassword(password);
        }

        1 reference
        public static bool VerifyPassword(string password, string hashedPassword)
        {
            return BCrypt.Net.BCrypt.Verify(password, hashedPassword);
        }
    }
}

```

Slika 5.2.12. Klasa PasswordHasher

WorkerRole u aplikaciji RedditService (slika 5.2.13.) ima ključnu ulogu u upravljanju osnovnim servisima i povezivanju sa Health Monitoring sistemom putem definisanog internog endpointa "HealthCheck". Ovaj endpoint je definisan unutar klase i koristi se za povezivanje sa ServiceStatus servisom preko adrese `net.tcp://{endpoint.IPEndpoint}/Service`, što omogućava monitoring stanja aplikacije. Pri pokretanju aplikacije (**OnStart**). Metoda Run pokreće glavni servis i prati njegovo izvršavanje. Zapisuje da je servis pokrenut, zatim pokreće asinhronu metodu RunAsync koja obavlja posao dok ne primi signal

za zaustavljanje. Kada se izvršavanje završi, signalizira da je servis gotov. WorkerRole postavlja maksimalan broj istovremenih konekcija i osigurava sigurnosni protokol TLS 1.2 za komunikaciju. Takođe, pokreće i otvara četiri glavna servera:

```
public class WorkerRole : RoleEntryPoint
{
    private readonly CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
    private readonly ManualResetEvent runCompleteEvent = new ManualResetEvent(false);
    ServiceHost serviceHost;
    private readonly string internalEndpointName = "HealthCheck";
    private readonly static UserServer UserServer = new UserServer();
    private readonly static PostServer PostServer = new PostServer();
    private readonly static CommentServer CommentServer = new CommentServer();
    private readonly static FavouiritesServer FavouiritesServer = new FavouiritesServer();

    0 references
    public override void Run()
    {
        Trace.TraceInformation("RedditService is running");

        try
        {
            this.RunAsync(this.cancellationTokenSource.Token).Wait();
        }
        finally
        {
            this.runCompleteEvent.Set();
        }
    }

    0 references
    public override bool OnStart()
    {
        // Use TLS 1.2 for Service Bus connections
        ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;

        // Set the maximum number of concurrent connections
        ServicePointManager.DefaultConnectionLimit = 12;

        // For information on handling configuration changes
        // see the MSDN topic at https://go.microsoft.com/fwlink/?LinkId=166357.

        try
        {
            var endpoint = RoleEnvironment.CurrentRoleInstance.InstanceEndpoints[internalEndpointName];
            var endpointAddress = $"net.tcp://{endpoint.IPEndpoint}/Service"; //Gadja sa HealthMonitoringService
            serviceHost = new ServiceHost(typeof(ServiceStatus));
            NetTcpBinding binding = new NetTcpBinding();
            serviceHost.AddServiceEndpoint(typeof(ICheckServiceStatus), binding, endpointAddress);
            serviceHost.Open();
            Trace.WriteLine("Communication established!");
        }
        catch
        {
            serviceHost.Abort();
        }

        bool result = base.OnStart();
        UserServer.Open();
        PostServer.Open();
        CommentServer.Open();
        FavouiritesServer.Open();

        Trace.TraceInformation("RedditService has been started");

        return result;
    }

    0 references
    public override void OnStop()
    {
        Trace.TraceInformation("RedditService is stopping");
        this.cancellationTokenSource.Cancel();
        this.runCompleteEvent.WaitOne();

        base.OnStop();
        UserServer.Close();
        PostServer.Close();
        CommentServer.Close();
        FavouiritesServer.Close();

        Trace.TraceInformation("RedditService has stopped");
    }

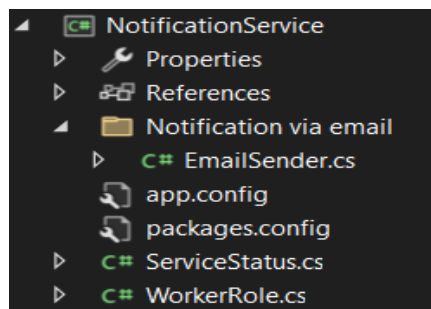
    1 reference
    private async Task RunAsync(CancellationToken cancellationToken)
    {
        // TODO: Replace the following with your own logic.
        while (!cancellationToken.IsCancellationRequested)
        {
            Trace.TraceInformation("Working");
            await Task.Delay(1000);
        }
    }
}
```

Slika 5.2.13. WorkerRole RedditService

UserServer, PostServer, CommentServer, i FavouiritesServer. Svaki od ovih servera omogućava obradu podataka korisnika, postova, komentara, i omiljenih postova kroz pripadajuće servise (kao što su IUserService, IPostService, ICommentService, i IFavouiritesService). Ovi serveri se povezuju putem TCP konekcija i rade unutar svog domena na osnovu specifičnih endpointa definisanih u aplikaciji. Klasa takođe sadrži metodu `RunAsync` koja se izvršava kontinuirano dok se ne prime signal za zaustavljanje aplikacije. Pri gašenju aplikacije (`OnStop`), svi otvoreni serveri se zatvaraju kako bi se osigurala pravilna dekomisija resursa i završavanje svih aktivnih konekcija. Sve ove komponente zajedno omogućavaju da RedditService funkcioniše kao robustan i skalabilan sistem za upravljanje korisnicima, postovima, komentarima, i omiljenim postovima, dok istovremeno omogućava praćenje zdravlja sistema kroz integraciju sa Health Monitoring servisom.

5.3. NotificationService komponenta

Kao što je ranije receno NotificationService komponenta zadužena je za automatizovano slanje obaveštenja korisnicima o aktivnostima unutar foruma. Kroz implementaciju ove komponente, sistem osigurava da svaki korisnik bude pravovremeno obavešten o novim komentarima na teme koje prati to jeste na teme na koje je preplacen – teme koje je favorizovao. NotificationService koristi Azure tehnologije za efikasno praćenje novih unosa i distribuciju obaveštenja putem e-maila, čime se značajno unapređuje korisničko iskustvo i omogućava veća interaktivnost unutar aplikacije. Slika 5.3.1. prikazuje arhitekturu NotificationService servisa.



Slika 5.3.1. NotificationService arhitektura

ServiceStatus klasa je implementirana na isti način kao kod **RedditService**-a. Implementacija je prikazana na slici 5.2.4. i služi, kako je rečeno, da proveri da li je servis pao ili da li je aktivan, što će biti objašnjeno u poglavlju 5.4.

U folderu **Notification via email** nalazi se klasa **EmailSender** (slika 5.3.2.), **EmailSender** klasa je ključna komponenta u sistemu za slanje e-mail obaveštenja korisnicima putem SMTP protokola. Ova klasa omogućava slanje e-mail poruka koristeći unapred definisane podatke za autentifikaciju i konfiguraciju.

Klasa sadrži jednu glavnu metodu, **SendEmail**, koja prima tri parametra: adresu primaoca (recipient), naslov poruke (subject), i telo poruke (body). Ovi parametri se koriste za kreiranje e-mail poruke koja će biti poslata korisnicima.

Unutar metode **SendEmail**, prvo se definiše e-mail adresa pošiljaoca (**fromMail**) i lozinka (**fromPassword**). Ovi podaci su hardkodirani u klasi i koriste se za autentifikaciju na SMTP serveru. Zatim se kreira objekat **MailMessage**, koji sadrži informacije o pošiljaocu, primaocu, naslovu, i telu poruke. Poruka je postavljena da bude u HTML formatu, što omogućava formatiranje sadržaja poruke na vizuelno privlačan način.

Za slanje poruke koristi se **SmtpClient**, koji je konfigurisan da koristi Gmail-ov SMTP server (**smtp.gmail.com**) na portu 587, sa omogućenim SSL-om radi sigurne komunikacije. Nakon konfiguracije, **smtpClient** pokušava da pošalje poruku. Ako je slanje uspešno, u Debug konzoli se ispisuje poruka "Email je uspešno poslat!". U slučaju greške tokom slanja, hvata se izuzetak i ispisuje odgovarajuća greška sa detaljima.

Ova klasa igra ključnu ulogu u obezbeđivanju pravovremenog informisanja korisnika putem e-maila, što je bitno za funkcionisanje sistema notifikacija. Prikaz klase je dat na slici 5.3.2.

```

namespace NotificationService.Notification_via_email
{
    2 references
    public class EmailSender
    {
        1 reference
        public void SendEmail(string recipient, string subject, string body)
        {
            string fromMail = "drsprojekat2023@gmail.com";
            string fromPassword = "xrnu nktr zprh vvqk";
            string toMail = recipient;

            MailMessage message = new MailMessage
            {
                From = new MailAddress(fromMail),
                Subject = subject
            };
            message.To.Add(new MailAddress(toMail));
            message.Body = body;
            message.IsBodyHtml = true;

            var smtpClient = new SmtpClient("smtp.gmail.com")
            {
                Port = 587,
                Credentials = new NetworkCredential(fromMail, fromPassword),
                EnableSsl = true,
            };

            try
            {
                smtpClient.Send(message);
                Debug.WriteLine("Email je uspešno poslat!");
            }
            catch (Exception ex)
            {
                Debug.WriteLine($"Došlo je do greške tokom slanja emaila: {ex.Message}");
            }
        }
    }
}

```

Slika 5.3.2. EmailSender klasa

Klasa **WorkerRole** predstavlja osnovni deo servisa za slanje notifikacija u Azure Cloud okruženju. Ova klasa nasleđuje **RoleEntryPoint**, što omogućava premošćivanje životnog ciklusa servisa kroz četiri ključne metode: **OnStart**, **Run**, **OnStop** i **RunAsync**.

Na početku se definišu ključni članovi klase (slika 5.3.3), uključujući **cancellationTokenSource**, koji služi za kontrolu toka izvršenja i prekidanje asinhronih zadataka, kao i **runCompleteEvent**, koji se koristi za signalizaciju kada je izvršenje kompletirano. Takođe se definiše **ServiceHost** objekat za komunikaciju sa **HealthMonitoringService**-om putem internog krajnje tačke (internal endpoint) nazvane **HealthCheck**. Ovaj endpoint je ključan jer omogućava povezivanje sa **HealthMonitoringService**-om, što omogućava proveru statusa servisa i detekciju eventualnih kvarova.

```

public class WorkerRole : RoleEntryPoint
{
    private readonly CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
    private readonly ManualResetEvent runCompleteEvent = new ManualResetEvent(false);
    ServiceHost serviceHost;
    private readonly string internalEndpointName = "HealthCheck";
    private readonly FavouritesServiceProvider favouritesService = new FavouritesServiceProvider();
    private readonly EmailSender emailSender = new EmailSender();
}

```

Slika 5.3.3. Ključni članovi klase

Metoda **OnStart** (slika 5.3.4.) se koristi za inicijalizaciju servisa. U njoj se podešava sigurnosni protokol na TLS 1.2 radi osiguravanja komunikacije sa Service Bus-om. Takođe se postavlja maksimalan broj istovremenih konekcija. Ovaj deo je kritičan za sigurnost i performanse sistema. U okviru **OnStart** metode, takođe se otvara komunikacioni kanal sa **HealthMonitoringService**-om putem internog endpoint-a. Ako se desi greška prilikom otvaranja kanala, servis se abortira kako bi se izbegli dodatni problemi.

```

0 references
public override bool OnStart()
{
    // Use TLS 1.2 for Service Bus connections
    ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;

    // Set the maximum number of concurrent connections
    ServicePointManager.DefaultConnectionLimit = 12;

    // For information on handling configuration changes
    // see the MSDN topic at https://go.microsoft.com/fwlink/?LinkId=166357.

    try
    {
        var endpoint = RoleEnvironment.CurrentRoleInstance.InstanceEndpoints[internalEndpointName];
        var endpointAddress = $"net.tcp://{endpoint.IPEndpoint}/Service"; //Gadja sa HealthMonitoringService
        serviceHost = new ServiceHost(typeof(ServiceStatus), new Uri(endpointAddress));
        NetTcpBinding binding = new NetTcpBinding();
        serviceHost.AddServiceEndpoint(typeof(ICheckServiceStatus), binding, endpointAddress);
        serviceHost.Open();
        Trace.TraceInformation("Communication established!");
    }
    catch
    {
        serviceHost.Abort();
    }

    bool result = base.OnStart();

    Trace.TraceInformation("NotificationService has been started");

    return result;
}

```

Slika 5.3.4. OnStart metoda

Metoda `Run` (slika 5.3.5.) predstavlja glavni izvršni tok servisa. U njoj se poziva asinhrona metoda `RunAsync`, koja obavlja glavnu logiku servisa. U slučaju greške tokom izvršenja, ona se hvata i ispisuje pomoću `Debug.WriteLine`, dok se na kraju izvršenja signalizira kompletiranje zadatka.

```

0 references
public override void Run()
{
    Trace.TraceInformation("NotificationService is running");

    try
    {
        this.RunAsync(this.cancellationTokenSource.Token).Wait();
    }
    catch (Exception ex)
    {
        Debug.WriteLine(ex);
    }
    finally
    {
        this.runCompleteEvent.Set();
    }
}

```

Slika 5.3.5. Run metoda

Metoda `OnStop` se koristi za sigurno zaustavljanje servisa. Prvo se prekida izvršenje putem `cancellationTokenSource`, a zatim se čeka da se izvršenje kompletira. Nakon toga, metoda poziva osnovnu implementaciju `OnStop` metode i ispisuje informaciju o zaustavljanju servisa pomoću `Trace.TraceInformation`.

```

0 references
public override void OnStop()
{
    Trace.TraceInformation("NotificationService is stopping");

    this.cancellationTokenSource.Cancel();
    this.runCompleteEvent.WaitOne();

    base.OnStop();

    Trace.TraceInformation("NotificationService has stopped");
}

```

Slika 5.3.6. OnStop metoda

Najvažniji deo ove klase je metoda `RunAsync` koja je data na slici 5.3.7., koja se izvršava u beskonačnoj petlji dok se ne dobije signal za prekid. U njoj se periodično proverava red poruka u Azure Storage-u, konkretno red pod nazivom "comments". Ako se pronađe poruka u redu, ona se preuzima i obrađuje. Metoda prvo čita sadržaj poruke, a zatim pretražuje bazu podataka za korisnike koji su pretplaćeni na odgovarajući post. Ako se pronađu pretplaćeni korisnici, svima njima se šalje email sa obaveštenjem o novom komentaru putem `EmailSender` klase.

Tokom izvršenja, sve ključne informacije i događaji se beleže pomoću `Trace.TraceInformation`, što omogućava praćenje toka izvršenja i dijagnostiku putem emulatora. Ovaj način ispisivanja poruka je od velikog značaja jer omogućava uvid u rad sistema, uključujući informacije o stanju poruka u redu i broju poslatih email-ova, što je posebno korisno za nadgledanje i otklanjanje grešaka u produkcionom okruženju.

```

1 reference
private async Task RunAsync(CancellationToken cancellationToken)
{
    // TODO: Replace the following with your own logic.
    while (!cancellationToken.IsCancellationRequested)
    {
        Trace.TraceInformation("Working");
        await Task.Delay(1000);

        CloudQueue queue = QueueHelper.GetQueueReference("comments");
        while (true)
        {
            CloudQueueMessage message = queue.GetMessage();
            if (message == null)
            {
                Trace.TraceInformation("Trenutno ne postoji poruka u redu.");
            }
            else
            {
                Trace.TraceInformation(String.Format("Poruka glasi: {0}", message.AsString));

                List<string> emails = new List<string>();
                CommentData commentData = CommentData.FromString(message.AsString);
                List<Favourites> allFavouritesFromTable = favouritesService.GetAllFavourites();
                foreach (var favourites in allFavouritesFromTable)
                {
                    if (favourites.PostId == commentData.PostId)
                    {
                        emails.Add(favourites.UserEmail);
                    }
                }

                if (emails.Count != 0)
                {
                    string subject = "Postavljen je novi komentar na temu na koju ste pretplaćeni";
                    StringBuilder bodyBuilder = new StringBuilder();
                    foreach (var email in emails)
                    {
                        bodyBuilder.AppendLine(email);
                    }
                    bodyBuilder.AppendLine();
                    bodyBuilder.AppendLine("Text komentara koji je postavljen: " + commentData.Text);
                    string body = bodyBuilder.ToString().TrimEnd();
                    body = body.Replace(Environment.NewLine, "<br/>");
                    emailSender.SendEmail("drsprojekat2023@gmail.com", subject, body);

                    Trace.TraceInformation("Informacije o poslatoj grupi notifikacija: ");
                    Trace.TraceInformation("Datum i vreme: " + DateTime.Now + " | ID komentara: " + commentData.Id + " | Broj poslatih mejlova: " + emails.Count + " |");

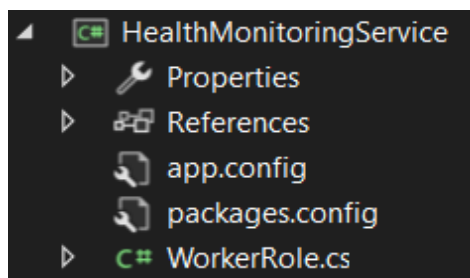
                    if (message.DequeueCount >= 1)
                    {
                        queue.DeleteMessage(message);
                    }
                }
            }
        }
        Thread.Sleep(5000);
    }
}

```

Slika 5.3.7. RunAsync metoda

5.4. HealthMonitoringService komponenta

HealthMonitoringService komponenta igra ključnu ulogu u obezbeđivanju stabilnosti i dostupnosti sistema. Ovaj servis je dizajniran da kontinuirano prati status svih ključnih komponenti aplikacije, kao što su RedditService i NotificationService, i da automatski reaguje u slučaju bilo kakvih problema. Kroz implementaciju HealthMonitoringService-a, sistem osigurava proaktivno otkrivanje i prijavljivanje kvarova, čime se minimiziraju rizici od potencijalnih zastoja i korisnicima garantuje visoka dostupnost usluge. Komponenta se integriše sa drugim servisima putem definisanih endpointa i koristi Azure tehnologije za praćenje i logovanje stanja sistema u realnom vremenu. Arhitektura HealthMonitoringService data je na slici 5.4.1.



Slika 5.4.1. HealthMonitoringService arhitektura

WorkerRole komponenta implementira ključnu funkcionalnost za praćenje statusa drugih servisa unutar sistema, konkretno RedditService i NotificationService. Ova komponenta funkcioniše kao servis za nadzor, koristeći politike ponovnog pokušaja i razmenu podataka sa drugim servisima putem WCF-a i SignalR-a.

Na početku implementacije, cancellationTokenSource služi za prekidanje asinhronih operacija i zaustavljanje servisa kada je to potrebno, dok runCompleteEvent omogućava da se čeka na završetak svih operacija pre nego što se rad servisa potpuno zaustavi. healthStatusRepo predstavlja repozitorijum koji evidentira status servisa u bazi podataka (slika 5.2.7.). serviceRedditProxy i serviceNotificationProxy su WCF proxy objekti koji omogućavaju komunikaciju sa udaljenim servisima (slika 5.2.3 i slika 5.2.4.). Promenljiva startTime beleži vreme početka rada servisa, što je ključno za simulaciju grešaka u komunikaciji. Promenljive redditServiceFailedOnce i notificationServiceFailedOnce prate da li je greška već simulirana za određeni servis. Sve ovo je prikazano na slici 5.4.2.

```
public class WorkerRole : RoleEntryPoint
{
    private readonly CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
    private readonly ManualResetEvent runCompleteEvent = new ManualResetEvent(false);
    private readonly HealthStatusRepository healthStatusRepo = new HealthStatusRepository(); //Connection String HealthStatusConnectionString
    private ICheckServiceStatus serviceRedditProxy;
    private ICheckServiceStatus serviceNotificationProxy;
    private DateTime startTime;
    private bool redditServiceFailedOnce = false;
    private bool notificationServiceFailedOnce = false;
}
```

Slika 5.4.2. WorkerRole

Metoda **OnStart** se poziva pri pokretanju role i definiše osnovne postavke za rad servisa. Obezbeđuje se sigurna komunikacija korišćenjem TLS 1.2 i postavlja se maksimalan broj istovremenih veza. Takođe,

uspostavljaju se veze sa `RedditService` i `NotificationService`, dok se beleži vreme pokretanja role kako bi se kasnije simulirale greške sto je prikazano na slici 5.4.3.

```
public override bool OnStart()
{
    // Use TLS 1.2 for Service Bus connections
    ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;

    // Set the maximum number of concurrent connections
    ServicePointManager.DefaultConnectionLimit = 12;

    // For information on handling configuration changes
    // see the MSDN topic at https://go.microsoft.com/fwlink/?LinkId=166357.

    ConnectToReddit();
    ConnectToNotificationService();

    // Record the start time
    startTime = DateTime.UtcNow;

    bool result = base.OnStart();

    Trace.TraceInformation("HealthMonitoringService has been started");

    return result;
}
```

Slika 5.4.3. `WorkerRole` funkcija `OnStart()`

Metoda `OnStop` (Slika 5.4.4.) služi za pravilno zaustavljanje servisa `HealthMonitoringService`. Ona prvo beleži u log da se servis zaustavlja, zatim otkazuje sve zadatke u toku, čeka da svi zadaci budu završeni, poziva osnovnu implementaciju `OnStop()` i na kraju beleži da je servis uspešno zaustavljen.

```
public override void OnStop()
{
    Trace.TraceInformation("HealthMonitoringService is stopping");

    this.cancellationTokenSource.Cancel();
    this.runCompleteEvent.WaitOne();

    base.OnStop();

    Trace.TraceInformation("HealthMonitoringService has stopped");
}
```

Slika 5.4.4. `WorkerRole` funkcija `OnStop()`

Metoda `Run` prikazana je na slici 5.4.5., ispisuje u Compute Emulatoru da je `HealthMonitoringService` uspešno pokrenut.

```

public override void Run()
{
    Trace.TraceInformation("HealthMonitoringService is running");

    try
    {
        this.RunWithRetryAsync(this.cancellationTokenSource.Token).Wait();
    }
    catch (Exception ex)
    {
        Debug.WriteLine(ex);
    }
    finally
    {
        this.runCompleteEvent.Set();
    }
}

```

Slika 5.4.5. WorkerRole funkcija Run()

Run metoda predstavlja glavni tok izvršenja i pokreće **RunWithRetryAsync** (slika 5.4.6.), koja koristi Polly biblioteku za implementaciju politike ponovnog pokušaja. Ova politika osigurava da se u slučaju grešaka, kao što su `CommunicationException` ili `TimeoutException`, izvršenje ponovi nakon određenog vremena.

```

1 reference
private async Task RunWithRetryAsync(Cancellation token)
{
    var retryPolicy = Policy
        .Handle<CommunicationException>()
        .Or<TimeoutException>()
        .WaitAndRetryAsync(
            retryCount: 3,
            sleepDurationProvider: retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)),
            onRetry: (exception, timeSpan, retryCount, context) =>
            {
                Trace.WriteLine($"Retry {retryCount} encountered a {exception.GetType().Name}. " +
                    $"Waiting {timeSpan} before next retry. Exception: {exception.Message}");
            });

    await retryPolicy.ExecuteAsync(async () =>
    {
        await RunAsync(token);
    });
}

```

Slika 5.4.6. WorkerRole funkcija RunWithRetryAsync

Metoda **RunAsync** (slika 5.4.7.) radi u beskonačnoj petlji dok se ne zatraži prekid rada. Unutar ove metode, `EnsureConnection` proverava da li su svi proxy objekti pravilno inicijalizovani, dok `MonitorWebRoleStatusAsync` periodično proverava status servisa i ažurira podatke u bazi.

```

1 reference
private async Task RunAsync(CancellationToken token)
{
    while (!token.IsCancellationRequested)
    {
        try
        {
            EnsureConnection();

            await MonitorWebRoleStatusAsync();
        }
        catch (CommunicationException ex)
        {
            Trace.WriteLine($"CommunicationException: {ex.Message}");
            throw;
        }
        catch (TimeoutException ex)
        {
            Trace.WriteLine($"TimeoutException: {ex.Message}");
            throw;
        }
        catch (Exception ex)
        {
            Trace.WriteLine($"Unexpected exception: {ex.Message}");
            throw;
        }
    }
}

```

Slika 5.4.7. WorkerRole funkcija RunAsync

Metoda `EnsureConnection` (slika 5.4.8.) igra ključnu ulogu u osiguravanju da su WCF proxy objekti za `RedditService` i `NotificationService` pravilno inicijalizovani. Ako proxy objekti nisu inicijalizovani, metode `ConnectToReddit` i `ConnectToNotificationService` se pozivaju da uspostave vezu sa odgovarajućim servisima. U slučaju simuliranih grešaka u komunikaciji, metod generiše log zapise koji se ispisuju u Compute Emulatoru, što omogućava jednostavno praćenje statusa servisa. Ukoliko dođe do greške, status servisa se ažurira u bazi podataka putem `healthStatusRepo`, a sistem koristi SignalR za obaveštavanje AdminHub o problemima sa uslugom

```

private void EnsureConnection()
{
    if (serviceRedditProxy == null)
    {
        ConnectToReddit();
    }
    // Simulate failure for RedditService after 30 seconds
    try
    {
        if (!redditServiceFailedOnce && (DateTime.UtcNow - startTime).TotalSeconds > 120)
        {
            redditServiceFailedOnce = true;
            throw new CommunicationException("RedditService is down (simulated)");
        }
        serviceRedditProxy.CheckServiceStatus();
    }
    catch (CommunicationException ex)
    {
        if (redditServiceFailedOnce)
        {
            Trace.WriteLine($"RedditService is down! {ex.Message} \n Reconnecting...");
            healthStatusRepo.Create(new HealthStatus("RedditService") { ServiceType = "RedditService", Status = "NOT_OK" });
            var connection = new HubConnection("http://localhost:8088");
            var myHub = connection.CreateHubProxy("AdminHub");
            connection.Start().Wait();
            string message = "1";
            myHub.Invoke("SendMessage", "RedditService", message).Wait();
            ConnectToReddit();
        }
    }
    if (serviceNotificationProxy == null)
    {
        ConnectToNotificationService();
    }
    // Simulate failure for NotificationService after 1 minute
    try
    {
        if (!notificationServiceFailedOnce && (DateTime.UtcNow - startTime).TotalSeconds > 150)
        {
            notificationServiceFailedOnce = true;
            throw new CommunicationException("NotificationService is down (simulated)");
        }
        serviceNotificationProxy.CheckServiceStatus();
    }
    catch (CommunicationException ex)
    {
        if (notificationServiceFailedOnce)
        {
            Trace.WriteLine($"NotificationService is down! {ex.Message} \n Reconnecting...");
            healthStatusRepo.Create(new HealthStatus("NotificationService") { ServiceType = "NotificationService", Status = "NOT_OK" });
            var connection = new HubConnection("http://localhost:8088");
            var myHub = connection.CreateHubProxy("AdminHub");
            connection.Start().Wait();
            string message = "1";
            myHub.Invoke("SendMessage", "NotificationService", message).Wait();
            ConnectToNotificationService();
        }
    }
}

```

Slika 5.4.8. WorkerRole funkcija EnsureConnection

SignalR (slika 5.4.8.) omogućava real-time komunikaciju između sistema za nadzor i administrativnog interfejsa. U kontekstu ove metode, kada se otkrije da je neki od servisa nedostupan, uspostavlja se konekcija prema SignalR hubu putem HubConnection objekta, koji se povezuje na određeni SignalR hub na serveru, koristeći URL poput `http://localhost:8080`. Nakon uspešne uspostave veze, SignalR omogućava slanje poruka direktno ka AdminHub, koristeći metod `Invoke` za pozivanje udaljene metode `SendMessage` na hubu. Na ovaj način, administrativni sistem je odmah obavešten o potencijalnim problemima, omogućavajući pravovremene intervencije i dodatne akcije. Ova funkcionalnost će kasnije biti detaljno objašnjena i prikazana u kontekstu kompletne arhitekture rešenja, sa posebnim osvrtom na lokalne konekcije i njihovu važnost tokom razvoja i testiranja.

Metoda **MonitorWebRoleStatusAsync** (slika 5.4.9.) periodično proverava dostupnost servisa i ažurira njihov status u bazi podataka. Ako servisi funkcionišu ispravno, njihov status se beleži kao "OK". U suprotnom, kada se simulira greška, status servisa se menja u "NOT_OK", a administrativni sistem je obavešten putem SignalR konekcije.

```
1 reference
private async Task MonitorWebRoleStatusAsync()
{
    bool serviceRedditAvailable = serviceRedditProxy.CheckServiceStatus();
    if (serviceRedditAvailable)
    {
        Trace.WriteLine("RedditService is running!");
        healthStatusRepo.Create(new HealthStatus("RedditService") { ServiceType = "RedditService", Status = "OK" });
    }

    bool notificationServiceAvailable = serviceNotificationProxy.CheckServiceStatus();
    if (notificationServiceAvailable)
    {
        Trace.WriteLine("NotificationService is running!");
        healthStatusRepo.Create(new HealthStatus("NotificationService") { ServiceType = "NotificationService", Status = "OK" });
    }

    await Task.Delay(5000);
}
```

Slika 5.4.9. WorkerRole funkcija `MonitorWebRoleStatusAsync`

Metoda **ConnecToReddit** (slika 5.4.10.) uspostavlja WCF konekciju sa `RedditService` koristeći TCP binding. Endpoint adresa se dinamički preuzima iz Azure Role okruženja pomoću `RoleEnvironment.Roles` (slika 3.2.3. prikazuje definisan endpoint za "HealthCheck")., što omogućava precizno određivanje IP adrese i porta na koji se servis povezuje. Nakon što se adresa preuzme, kreira se WCF `ChannelFactory`, koja inicijalizuje proxy objekat `serviceRedditProxy`. Ovaj proxy objekat omogućava daljinsko pozivanje metoda, poput `CheckServiceStatus`, čime se osigurava kontinuirano praćenje zdravlja servisa.

```
3 references
public void ConnecToReddit()
{
    var endpoint = RoleEnvironment.Roles["RedditService"].Instances[0].InstanceEndpoints["HealthCheck"];
    var address = new EndpointAddress($"net.tcp://{endpoint.IPEndpoint}/Service");
    var binding = new NetTcpBinding();
    ChannelFactory<ICheckServiceStatus> factory = new ChannelFactory<ICheckServiceStatus>(binding, address);
    serviceRedditProxy = factory.CreateChannel();
}
```

Slika 5.4.10. WorkerRole funkcija `ConnecToReddit`

Metoda **ConnecToNotificationService** (slika 5.4.11.) funkcioniše na sličan način, uspostavljaajući WCF konekciju sa `NotificationService`. Kao i kod prethodne metode, adresa i port se dinamički preuzimaju iz Azure Role okruženja (slika 3.2.6. prikazuje definisan endpoint za "HealthCheck"). Korišćenjem ove

dinamičke metode povezivanja, sistem ostaje fleksibilan i sposoban da se automatski prilagodi promenama u mrežnoj infrastrukturi, osiguravajući stabilnu komunikaciju sa svim ključnim servisima.

```
3 references
public void ConnectToNotificationService()
{
    var binding = new NetTcpBinding();
    var endpoint = RoleEnvironment.Roles["NotificationService"].Instances[0].InstanceEndpoints["HealthCheck"];
    var address = new EndpointAddress($"net.tcp://{endpoint.IPEndpoint.Address}:{endpoint.IPEndpoint.Port}/Service");
    ChannelFactory<ICheckServiceStatus> factory = new ChannelFactory<ICheckServiceStatus>(binding, address);
    serviceNotificationProxy = factory.CreateChannel();
}
```

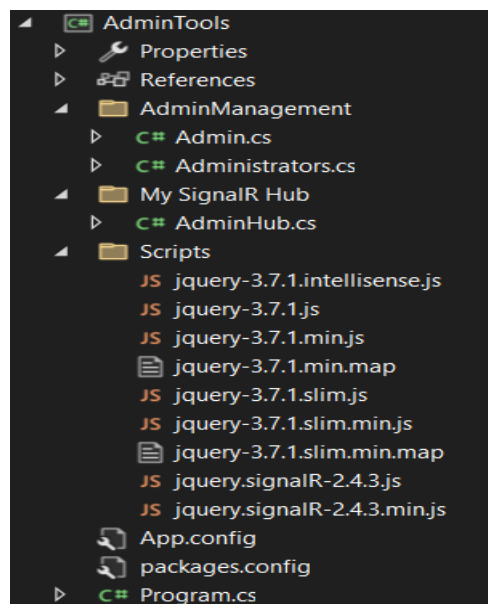
Slika 5.4.11. WorkerRole funkcija ConnectToNotificationService

5.5. AdminTools i SignalR biblioteka

AdminTools komponenta je ključna za administraciju sistema, omogućavajući administratorima da upravljaju obaveštavanjem korisnika o statusu servisa u realnom vremenu. Kroz integraciju sa SignalR bibliotekom, AdminTools omogućava momentalno slanje obaveštenja korisnicima o potencijalnim problemima i padovima servisa, čime se osigurava visoka dostupnost i pouzdanost sistema.

AdminTools komponenta, implementirana kao konzolna aplikacija, omogućava administratorima da se prijave na sistem, dodaju ili uklanjaju email adrese za obaveštavanje i prate status servisa putem jednostavnog interfejsa. SignalR biblioteka omogućava dvosmernu komunikaciju između servera i klijenata u realnom vremenu, čime se korisnicima pruža trenutna povratna informacija o promenama u statusu sistema. Ova funkcionalnost je od ključne važnosti za održavanje stabilnosti i pouzdanosti sistema, posebno u okruženjima gde je neophodna brza reakcija na potencijalne probleme.

Kroz ovaj prikaz, detaljno ćemo opisati način na koji su AdminTools i SignalR integrisani u postojeći sistem, funkcionalnosti koje pružaju, kao i prednosti koje donose u kontekstu upravljanja i monitoringa nad sistemom. Arhitektura komponente AdminTools i SignalR biblioteke data je na slici 5.5.1.



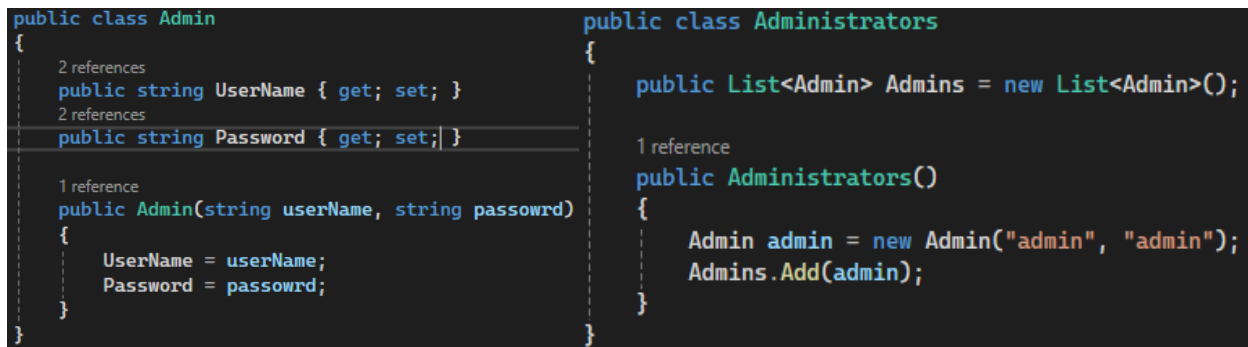
Slika 5.5.1. AdminTools i SignalR arhitektura

AdminManagement folder sadrži klase koje omogućavaju osnovnu funkcionalnost administracije unutar sistema, uključujući kreiranje, autentifikaciju i upravljanje administratorskim nalogima.

Klasa **Admin** je osnovni model za predstavljanje administratorskih naloga. Ova klasa sadrži dva svojstva: **UserName**: Korisničko ime administratora i **Password**: Lozinka administratora. Konstruktor klase **Admin** omogućava inicijalizaciju ova dva svojstva prilikom kreiranja novog administratorskog naloga, omogućavajući jednostavno kreiranje administratora sa zadatim korisničkim imenom i lozinkom.

Klasa **Administrators** je odgovorna za upravljanje kolekcijom administratorskih naloga. Ova klasa sadrži listu objekata klase **Admin** koja se koristi za skladištenje i upravljanje svim administratorima unutar sistema. **Admins**: Lista koja sadrži sve instance klase **Admin**.

Konstruktor klase **Administrators** automatski kreira listu i dodaje jednog podrazumevanog administratora sa korisničkim imenom i lozinkom "admin", "admin". Slika ove dvije klase data je na slici 5.5.2.



```

public class Admin
{
    2 references
    public string UserName { get; set; }
    2 references
    public string Password { get; set; }

    1 reference
    public Admin(string userName, string password)
    {
        UserName = userName;
        Password = password;
    }
}

public class Administrators
{
    public List<Admin> Admins = new List<Admin>();

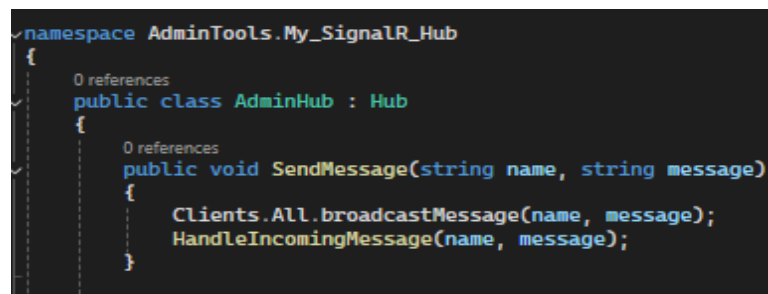
    1 reference
    public Administrators()
    {
        Admin admin = new Admin("admin", "admin");
        Admins.Add(admin);
    }
}

```

Slika 5.5.2. AdminTools i SignalR arhitektura

Ove dve klase, **Admin** i **Administrators**, zajedno pružaju osnovu za upravljanje administratorskim nalogima u okviru **AdminTools** aplikacije. Dok **Admin** modeluje pojedinačni administratorski nalog, **Administrators** omogućava kreiranje i upravljanje celokupnim skupom ovih naloga.

Klasa **AdminHub** predstavlja ključnu komponentu u okviru **AdminTools** aplikacije, zaduženu za upravljanje komunikacijom između servera i klijenata putem **SignalR** tehnologije. Ova klasa omogućava dvosmernu komunikaciju, što je esencijalno za aplikacije koje zahtevaju real-time ažuriranje i obaveštavanje korisnika o promenama statusa servisa.



```

namespace AdminTools.My_SignalR_Hub
{
    0 references
    public class AdminHub : Hub
    {
        0 references
        public void SendMessage(string name, string message)
        {
            Clients.All.broadcastMessage(name, message);
            HandleIncomingMessage(name, message);
        }
    }
}

```

Slika 5.5.3. Metoda SendMessage

Klasa **AdminHub** nasleđuje klasu **Hub** iz Microsoftovog **SignalR**-a, čime se omogućava direktna komunikacija sa svim klijentima koji su trenutno povezani na hub. Metoda **SendMessage** ima dvostruku funkciju: **broadcastMessage** - Ova metoda šalje poruku svim klijentima koji su povezani na hub. Poruka

sadrži ime i tekst poruke, čime se osigurava da svi klijenti dobiju informaciju o promeni. `HandleIncomingMessage` - Nakon slanja poruke svim klijentima, ova metoda se koristi za dalju obradu poruke, kako bi se proverilo da li poruka zahteva dodatne akcije, kao što je obaveštavanje administratora. Nasledjivanje I metoda `SendMessage` prikazani sun a slici 5.5.3.

```
1 reference
private void HandleIncomingMessage(string name, string message)
{
    if (message.Equals("1"))
    {
        Console.WriteLine($"Obaveštava se admin da {name} nije dostupan!\n");
        string subject = "Neki od servisa je pao!";
        string body = $"{name} trenutno nije dostupan!\nMolimo vas da uradite nešto kako bi korisnici ponovo mogli da koriste platformu.";
        SendEmail(subject, body);
    }
}
```

Slika 5.5.4. Metoda `SendMessage`

`HandleIncomingMessage` metoda igra ključnu ulogu u detekciji problema sa servisima. U ovom primeru, ukoliko poruka sadrži vrednost "1", to signalizira da neki servis nije dostupan. Nakon što se ovo detektuje, generiše se odgovarajuća poruka u konzoli i priprema se email koji će biti poslat administratorima. Metoda poziva `SendEmail` kako bi poslala email obaveštenje o padu servisa. Ova funkcionalnost omogućava brzo reagovanje administratora na potencijalne probleme, osiguravajući visoku dostupnost sistema. Na slici 5.5.4. prikazana je metoda `HandleIncomingMessage`.

```
1 reference
private void SendEmail(string subject, string body)
{
    string fromMail = "drsprojekat2023@gmail.com";
    string fromPassword = "xrnu nktr zprh vvqk";
    List<string> toMails = AdminTools.Program.EmailAddresses;

    foreach (var toMail in toMails)
    {
        MailMessage message = new MailMessage
        {
            From = new MailAddress(fromMail),
            Subject = subject,
            Body = body,
            IsBodyHtml = true
        };
        message.To.Add(new MailAddress(toMail));

        var smtpClient = new SmtpClient("smtp.gmail.com")
        {
            Port = 587,
            Credentials = new NetworkCredential(fromMail, fromPassword),
            EnableSsl = true,
        };

        try
        {
            smtpClient.Send(message);
            Console.WriteLine($"Email uspešno poslat na {toMail}!");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Došlo je do greške tokom slanja emaila na {toMail}: {ex.Message}");
        }
    }
}
```

Slika 5.5.5. Metoda `SendMessage`

`SendEmail` metoda koristi SMTP protokol za slanje email obaveštenja. Parametri kao što su adresa pošiljaoca, lozinka, i lista primaoca, inicijalizuju se na početku. Za svakog primaoca se kreira `MailMessage` objekat, koji se zatim šalje koristeći `SmtpClient`. Ova metoda osigurava da svi relevantni administratori budu obavešteni o padu servisa i da imaju sve potrebne informacije kako bi mogli da reaguju na problem.

Integracija sa EnsureConnection metodom:

Klasa AdminHub je integrisana sa metodom EnsureConnection (slika 5.4.8.), koja se nalazi u okviru HealthMonitoringService aplikacije. Kada dođe do pada servisa, kao što su RedditService ili NotificationService, EnsureConnection metoda koristi SendMessage metod iz klase AdminHub da pošalje obavještenje putem SignalR-a i pokrene proces slanja emaila administratorima. Na ovaj način, administratori su uvek informisani o statusu servisa i mogu brzo reagovati na eventualne probleme.

Ovaj detaljan opis klase AdminHub pruža uvid u njenu funkcionalnost i integraciju sa ostatkom sistema, omogućavajući bolju razumevanje uloge koju ova klasa igra u obezbeđivanju stabilnosti i dostupnosti aplikacije.

Klasa Program u projektu AdminTools predstavlja glavnu ulaznu tačku aplikacije i osigurava inicijalizaciju i upravljanje administrativnim alatima, uključujući autentifikaciju administratora, upravljanje email adresama, i pokretanje SignalR servera. Ova klasa je od suštinske važnosti za pravilno funkcionisanje aplikacije, jer omogućava administratorima da se prijave, upravljaju listom email adresa, i započnu komunikaciju sa drugim komponentama sistema.

```
public class Program
{
    public static List<string> EmailAddresses = new List<string>();

    0 references
    static void Main()
    {
        bool isAuthenticated = false;
        Administrators administrators = new Administrators();

        Console.WriteLine("*****");
        Console.WriteLine("Dobrodošli u administratorsku konzolu. Molimo vas prijavite se!");
        Console.WriteLine("*****\n");

        do
        {
            Console.Write("Unesite korisničko ime: ");
            string userName = Console.ReadLine();
            Console.Write("Unesite lozinku: ");
            string password = Console.ReadLine();

            foreach (var admin in administrators.Admins)
            {
                if (admin.UserName == userName && admin.Password == password)
                {
                    isAuthenticated = true;
                    break;
                }
            }

            if (isAuthenticated)
            {
                Console.WriteLine("\nUspešna prijava!\n");
                ManageEmailAddresses();

                string url = "http://localhost:8080";
                using (WebApp.Start<Startup>(url))
                {
                    Console.WriteLine($"Server je dostupan na: {url}\n");
                    Console.ReadLine();
                }
            }
            else
            {
                Console.WriteLine("\nPogrešili ste korisničko ime ili lozinku! Pokušajte ponovo.\n");
            }
        } while (!isAuthenticated);
    }
}
```

Slika 5.5.6. Metoda Main

Glavna metoda Main (slika 5.5.6.) je srce aplikacije i upravlja sledećim ključnim funkcionalnostima:

1. Autentifikacija: Prvi deo metode služi za autentifikaciju administratora. Korisniku se traži da unese svoje korisničko ime i lozinku, a uneti podaci se proveravaju u odnosu na listu administratora definisanu u klasi Administrators. Ako su kredencijali tačni, korisnik se uspešno prijavljuje u sistem.

2. Upravljanje email adresama: Nakon uspešne prijave, metoda `ManageEmailAddresses` omogućava administratoru da upravlja listom email adresa. Ova funkcionalnost je važna jer omogućava da se definišu primaoci obaveštenja o problemima sa servisima.
3. Pokretanje SignalR servera: Nakon što su administratori prijavljeni i email adrese postavljene, aplikacija pokreće SignalR server na lokalnom hostu na portu 8080, koristeći klasu `Startup` za konfiguraciju. Server ostaje aktivan dok se ne pritisne Enter, čime se omogućava povezivanje sa ostalim delovima sistema.

Metoda `ManageEmailAddresses` (slika 5.5.7.) pruža administratorima alat za dodavanje, uklanjanje, ažuriranje i pregled email adresa koje će biti korišćene za slanje obaveštenja. Ova metoda je implementirana kroz petlju koja se završava kada administrator unese komandu "exit". Ovo omogućava administratorima fleksibilnost u upravljanju email adresama.

```
private static void ManageEmailAddresses()
{
    string command = "";
    while (command != "exit")
    {
        Console.WriteLine("1. Unesite 'add' za dodavanje email adrese");
        Console.WriteLine("2. Unesite 'remove' za uklanjanje email adrese");
        Console.WriteLine("3. Unesite 'update' za ažuriranje email adrese");
        Console.WriteLine("4. Unesite 'list' za prikaz svih email adresa");
        Console.WriteLine("5. Unesite 'exit' za izlaz");
        Console.WriteLine("Unos: ");
        command = Console.ReadLine().ToLower();

        switch (command)
        {
            case "add":
                Console.WriteLine("Unesite email adresu za dodavanje: ");
                string emailToAdd = Console.ReadLine();
                if (!EmailAddresses.Contains(emailToAdd))
                {
                    EmailAddresses.Add(emailToAdd);
                    Console.WriteLine($"Email adresa {emailToAdd} je dodata.");
                }
                else
                {
                    Console.WriteLine($"Email adresa {emailToAdd} već postoji.");
                }
                break;

            case "remove":
                Console.WriteLine("Unesite email adresu za uklanjanje: ");
                string emailToRemove = Console.ReadLine();
                if (EmailAddresses.Contains(emailToRemove))
                {
                    EmailAddresses.Remove(emailToRemove);
                    Console.WriteLine($"Email adresa {emailToRemove} je uklonjena.");
                }
                else
                {
                    Console.WriteLine($"Email adresa {emailToRemove} nije pronađena.");
                }
                break;

            case "update":
                Console.WriteLine("Unesite email adresu koju želite da ažurirate: ");
                string emailToUpdate = Console.ReadLine();
                if (EmailAddresses.Contains(emailToUpdate))
                {
                    case "update":
                        Console.WriteLine("Unesite email adresu koju želite da ažurirate: ");
                        string emailToUpdate = Console.ReadLine();
                        if (EmailAddresses.Contains(emailToUpdate))
                        {
                            Console.WriteLine("Unesite novu email adresu: ");
                            string newEmail = Console.ReadLine();
                            int index = EmailAddresses.IndexOf(emailToUpdate);
                            EmailAddresses[index] = newEmail;
                            Console.WriteLine($"Email adresa {emailToUpdate} je ažurirana na {newEmail}.");
                        }
                        else
                        {
                            Console.WriteLine($"Email adresa {emailToUpdate} nije pronađena.");
                        }
                        break;

                    case "list":
                        Console.WriteLine("Trenutne email adrese:");
                        foreach (var email in EmailAddresses)
                        {
                            Console.WriteLine(email);
                        }
                        break;

                    case "exit":
                        Console.WriteLine("Izlazak iz menadžera email adresa.");
                        break;

                    default:
                        Console.WriteLine("Nepoznata komanda.");
                        break;
                }
            }
        }
    }
}
```

Slika 5.5.7. Metoda `ManageEmailAddresses`

Klasa `Startup` sadrži metodu `Configuration` koja konfiguriše SignalR middleware unutar OWIN pipeline-a. Ova konfiguracija omogućava aplikaciji da koristi SignalR za real-time komunikaciju sa klijentima. U kontekstu projekta AdminTools, ovo znači da će SignalR server biti aktivan i spreman za slanje i primanje poruka između servera i klijenata. Ova klasa prikazana je na slici 5.5.8.

```
1 reference
public class Startup //Startup pokrece AdminHub na onom gore definisanom portu
{
    0 references
    public void Configuration(IApplicationBuilder app)
    {
        app.MapSignalR();
    }
}
```

Slika 5.5.8. Klasa `Startup`

Klasa **Program** u okviru projekta AdminTools igra ključnu ulogu u celokupnoj arhitekturi aplikacije. Od autentifikacije administratora do upravljanja email adresama i pokretanja SignalR servera, ova klasa osigurava da aplikacija radi glatko i da su svi administrativni zadaci pravilno obavljani. Posebno je važno napomenuti da je potrebno ručno izaći iz aplikacije (komanda exit) kako bi se oslobodio port za buduće konekcije, što omogućava pravilno funkcionisanje metode EnsureConnection u okviru drugih servisa aplikacije.

6. Zaključak

Implementacija aplikacije za upravljanje forumskim temama i komentarima korišćenjem Microsoft Azure tehnologija uspešno je demonstrirala sposobnost Cloud platforme da omogući skalabilnost, pouzdanost i fleksibilnost neophodnu za modernu web aplikaciju. Kroz detaljan pregled korišćenih tehnika i tehnologija, kao i kroz konkretne primere implementacije različitih funkcionalnosti sistema, prikazana je sveobuhvatna slika o prednostima i izazovima rada u Cloud okruženju.

Primena Microsoft Azure Cloud Service-a, zajedno sa komponentama kao što su WebRole, WorkerRole, i Azure Storage, omogućila je razvijanje distribuiranog sistema sposobnog da obradi veliki broj korisničkih zahteva, skalira resurse prema potrebi i obezbedi visoku dostupnost aplikacije. Korišćenje Visual Studio 2022 kao razvojnog okruženja pružilo je sveobuhvatan skup alata za efikasno upravljanje kodom, debugovanje i testiranje, što je značajno doprinelo uspešnoj realizaciji projekta.

Jedan od ključnih aspekata implementacije bio je razvoj NotificationService-a i HealthMonitoringService-a, koji su omogućili efikasno slanje obaveštenja korisnicima i stalnu proveru dostupnosti sistema. Kroz integraciju sa SignalR bibliotekom, omogućeno je real-time obaveštavanje korisnika, što dodatno poboljšava korisničko iskustvo i pouzdanost sistema.

Detaljan pregled funkcionalnosti aplikacije, uključujući registraciju korisnika, logovanje, upravljanje korisničkim profilima, kreiranje i upravljanje temama i komentarima, kao i integraciju chatbot-a za pomoć korisnicima, pokazuje visoku funkcionalnost i intuitivnost korisničkog interfejsa. Sve ove funkcionalnosti su podržane efikasnim sistemom za skladištenje podataka korišćenjem Azure Storage servisa, što omogućava sigurno i pouzdano čuvanje i upravljanje velikim količinama podataka.

Predlozi za dalja usavršavanja uključuju:

1. **Unapređenje sigurnosnih aspekata aplikacije:** Implementacija naprednih sigurnosnih protokola i dodatnih mehanizama za zaštitu podataka, kao što su dvostruka autentifikacija (2FA) i enkripcija osjetljivih podataka u mirovanju i prilikom prenosa.
2. **Optimizacija performansi:** Dodatna optimizacija performansi aplikacije, uključujući poboljšanje vremena odziva i efikasnije korišćenje resursa. Ovo se može postići kroz implementaciju naprednih tehnika keširanja i optimizaciju baze podataka.
3. **Razvoj mobilne verzije aplikacije:** Kreiranje mobilne aplikacije koja bi omogućila korisnicima da pristupaju forumu sa svojih pametnih telefona i tableta, pružajući im veću fleksibilnost i dostupnost.
4. **Implementacija naprednih analitičkih alata:** Uvođenje analitičkih alata koji bi omogućili detaljno praćenje i analizu korisničkog ponašanja, identifikaciju trendova i boljeg razumevanja potreba korisnika. Ovi podaci mogu biti korišćeni za dalji razvoj i prilagođavanje funkcionalnosti aplikacije.
5. **Integracija sa drugim Cloud servisima:** Razmatranje integracije sa dodatnim Azure servisima kao što su Azure Cognitive Services za implementaciju naprednih funkcionalnosti poput prepoznavanja slika i prirodnog jezika, kao i Azure Machine Learning za prediktivnu analitiku.

6. **Poboljšanje korisničkog iskustva:** Kontinuirano unapređenje korisničkog interfejsa na osnovu povratnih informacija korisnika, što bi uključivalo poboljšanje navigacije, responzivnosti dizajna i dodavanje novih korisnih funkcionalnosti.

U budućem radu na ovom projektu, fokus bi mogao biti na dodatnom unapređenju sigurnosnih aspekata aplikacije, optimizaciji performansi i dodavanju novih funkcionalnosti koje bi dodatno poboljšale korisničko iskustvo. Mogućnosti za proširenje aplikacije uključuju integraciju sa drugim Cloud servisima, razvoj mobilne verzije aplikacije, kao i implementaciju naprednih analitičkih alata za bolje praćenje i razumevanje korisničkog ponašanja.

Zaključno, rad na ovom projektu pružio je dragoceno iskustvo u korišćenju Cloud tehnologija za razvoj složenih web aplikacija, istovremeno demonstrirajući potencijal i fleksibilnost Microsoft Azure platforme u podršci modernih poslovnih procesa. Ova implementacija predstavlja čvrstu osnovu za dalji razvoj i unapređenje, čime se dodatno potvrđuje značaj Cloud tehnologija u savremenom IT okruženju.

Literatura

- [1] **Microsoft Visual Studio [2022]**. Pristup: 26.06.2024. Ključne reči: Visual Studio, razvojno okruženje, Microsoft. Dostupno na: <https://visualstudio.microsoft.com/>.
- [2] **HTML (HyperText Markup Language)**. Pristup: 27.06.2024. Ključne reči: HTML, jezik za strukturisanje, veb stranice. Dostupno na: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [3] **CSS (Cascading Style Sheets)**. Pristup: 29.06.2024. Ključne reči: CSS, stilizacija, veb stranice. Dostupno na: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [4] **JavaScript**. Pristup: 01.07.2024. Ključne reči: JavaScript, interaktivnost, programiranje. Dostupno na: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [5] **Azure Cloud Service**. Pristup: 02.07.2024. Ključne reči: Azure, cloud, Microsoft. Dostupno na: <https://azure.microsoft.com/en-us/services/cloud-services/>.
- [6] **Azure Storage Service Explorer**. Pristup: 05.07.2024. Ključne reči: Azure Storage, Explorer, Microsoft. Dostupno na: <https://azure.microsoft.com/en-us/features/storage-explorer/>.
- [7] **Microsoft Azure Compute Emulator**. Pristup: 12.07.2024. Ključne reči: Azure, Compute Emulator, Microsoft. Dostupno na: <https://docs.microsoft.com/en-us/azure/cloud-services/cloud-services-choose-emulator>.
- [8] **Microsoft Azure Storage Emulator**. Pristup: 27.07.2024. Ključne reči: Azure, Storage Emulator, Microsoft. Dostupno na: <https://docs.microsoft.com/en-us/azure/storage/common/storage-use-emulator>.
- [9] **ChatGPT**. Pristup: [26.06.2024 – 26.08.2024]. Ključne reči: ChatGPT, OpenAI, AI pomoćnik. Dostupno na: <https://chat.openai.com/>.
- [10] **GitHub Repository: Razvoj CLOUD aplikacija UPM**. Pristup: [26.06.2024 – 26.08.2024]. Dostupno na: https://github.com/MastilovicRadoslav/Razvoj_CLOUD_aplikacija_UPM.
- [11] **Razvoj Cloud aplikacija u pametnim mrežama**. Pristup: [26.06.2024 – 26.08.2024]. Ključne reči: Razvoj Cloud aplikacija, pametne mreže. Dostupno na: <https://www.eepsi.ftn.uns.ac.rs/group/razvoj-cloud-aplikacija-u-pametnim-mrezama/discussion>.

Biografija

Lične informacije:

Podaci: Ime i prezime: Radoslav Mastilović

Datum rođenja: 02.01.2001

Mjesto rođenja: Trebinje, Republika Srpska, Bosna i Hercegovina

Obrazovanje:

Osnovna škola: Sveti Sava, Gacko

Srednja škola: Pero Slijepčević, Gacko (Mašinski tehničar za kompjutersko konstruisanje)

Trenutno: Student četvrte godine na Fakultetu Tehničkih Nauka, Univerzitet u Novom Sadu

Naučna oblast: Elektrotehničko i računarsko inženjerstvo

Departman (smjer): Primenjeno softversko inženjerstvo (PSI)

Porodična pozadina:

Živim sa svojim roditeljima i starijim bratom i sestrom. Moj otac je penzioner, a majka je nezaposlena. Brat je diplomirao na Visokoj brodarskoj školi u Beogradu i trenutno radi u Rudniku i Termoelektrani u Gacku. Moja sestra studira Srpski jezik i književnost na Univerzitetu u Istočnom Sarajevu i paralelno se usavršava i radi u IT struci.

Hobiji i interesovanja:

U slobodno vrijeme, uživam u sportu i gledanju filmova. Takođe, volim da čitam knjige iz oblasti nauke, tehnologije i istorije, posebno one koje istražuju nove inovacije u softverskom inženjerstvu. Pored toga, veliki je moj interes za politiku, i redovno pratim događaje i razvoje u ovoj oblasti.

Ključna dokumentacijska informacija

Redni broj, RBR :			
Identifikacioni broj, IBR :			
Tip dokumentacije, TD :	Monografska publikacija		
Tip zapisa, TZ :	Tekstualni štampani dokument		
Vrsta rada, VR :	Diplomski rad		
Autor, AU :	Radoslav Mastilović		
Mentor, MN :	Prof. Dr Srdjan Vukmirović		
Naslov rada, NR :	Implementacija CLOUD tehnologija za razvoj aplikacije za upravljanje forumskim temama i komentarima na primeru Reddita		
Jezik publikacije, JP :	Srpski		
Jezik izvoda, JL :	Srpski / Engleski		
Zemlja publikovanja, ZP :	Srbija		
Ule geografsko područje, UGP :	Vojvodina		
Godina, GO :	2024		
Izdavač, IZ :	Autorski reprint		
Mesto i adresa, MA :	Novi Sad, Fakultet tehničkih nauka, Trg Dositeja Obradovića 6		
Fizički opis rada, FO : (поглавља/страна/ цитата/табела/слика/графика/прилога)	Br. Poglavlja 6/ stranica 83/ citata 0/ tabela 0/ slika 110/ grafikona 0/ priloga 0		
Naučna oblast, NO :	Primenjeno softversko inženjersvo – E3		
Naučna disciplina, ND :	Razvoj Cloud aplikacija u pametnim mrežama		
Predmetna odrednica / Ključne reči, PO :	Azure Cloud aplikacija, WebRole, WorkerRole, Console Application, Class Library, ASP.NET MVC		
UDK			
Čuva se, ČU :	Biblioteka Fakulteta tehničkih nauka, Trg Dositeja Obradovića 6, Novi Sad		
Važna napomena, VN :			
Izvod, IZ :	U radu je opisana Azure Cloud aplikacija za upravljanje forumskim temama I komentarima na primeru Reddita. U prvom poglavlju opisan je uvod, u drugom korištene tehnike i tehnologije, u trećem specifikacija sistema šta će sve sadržati i kako će funkcionisati, u četvrtom implementacija sistema, u petom prikaz implementirano sistema i na kraju u šestom zaključak.		
Datum prihvatanja teme, DP :			
Datum odbrane, DO :			
Članovi komisije, KO :	Predsednik:	Prof. Dr Srdjan Vukmirović, FTN Novi Sad	Potpis mentora
	Član:	Doc. Dr Nemanja Nedić, FTN Novi Sad	
	Član:	Prof. Dr Darko Čapko, FTN Novi Sad	
	Član, mentor:	Prof. Dr Srdjan Vukmirovic, FTN Novi Sad	

Key words documentation

Accession number, ANO :			
Identification number, INO :			
Document type, DT :	Monographic publication		
Type of record, TR :	Textual material		
Contents code, CC :	BSc thesis		
Author, AU :	Radoslav Mastilović		
Mentor, MN :	Prof. Dr Srdjan Vukmirović		
Title, TI :	Implementation of CLOUD technologies for the development of an application for managing forum topics and comments on the example of Reddit		
Language of text, LT :	Serbian		
Language of abstract, LA :	Serbian/English		
Country of publication, CP :	Serbia		
Locality of publication, LP :	Vojvodina		
Publication year, PY :	2024		
Publisher, PB :	Author's reprint		
Publication place, PP :	Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	No. Chapters 6/ page 83/ quotes 0/ table 0/ figure 110/ graph 0/ attachment 0		
Scientific field, SF :	Applied software engineering - E3		
Scientific discipline, SD :	Development of Cloud applications in smart networks		
Subject/Key words, S/KW :	Azure Cloud Application, WebRole, WorkerRole, Console Application, Class Library, ASP.NET MVC		
UC			
Holding data, HD :	Library of the Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad		
Note, N :			
Abstract, AB :	The paper describes an Azure Cloud application for managing forum topics and comments on the example of Reddit. In the first chapter the introduction is described, in the second the techniques and technologies used, in the third the specification of the system, what it will contain and how it will function, in the fourth the implementation of the system, in the fifth a presentation of the implemented system and finally in the sixth the conclusion.		
Accepted by the Scientific Board on, ASB :			
Defended on, DE :			
Defended Board, DB :	President:	Prof. Dr. Srdjan Vukmirović, FTN Novi Sad	Menthor's sign
	Member:	Doc. Dr Nemanja Nedić, FTN Novi Sad	
	Member:	Prof. Dr Darko Čapko, FTN Novi Sad	
	Member, Mentor:	Prof. Dr Srdjan Vukmirovic, FTN Novi Sad	